MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

Módulo 9 - Tarea 1



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

1. Análisis inicial de la muestra con PEStudio	...	6
1.1 Información general:	...	6
1.2 INDICATORS	...	6
1.3 FOOTPRINTING - Huellas Dactilares	...	7
1.4 DOS-HEADER	...	7
1.5 DOS-STUB	...	8
1.6 RICH HEADER	...	8
1.7 FILE HEADER	...	9
2. Analizamos la muestra en VirusTotal	...	9
2.1 Vemos el informe en Tria.Ge	...	11
3. Primer análisis de la muestra	...	12
3.1 El tipo de fichero	...	12
3.2 Clasificación técnica	...	12
3.3 Evidencias principales	...	13
3.4 Mensaje DOS stub habitual	...	13
3.5 La cabecera PE	...	13
3.6 Secciones detectadas	...	13
3.7 Conclusiones	...	13
4. Analizamos si la muestra está empaquetada	...	14
4.1 Conclusiones del empaquetamiento	...	15
5. Analizamos la muestra con Detect It Easy	...	15
5.1 Analizamos la entropía con DIE:	...	16
5.2 Investigamos `.reloc` con la herramienta PEview:	...	17
6. Decodificación de cadenas ofuscadas usando FLOSS	...	18
7. Las secciones	...	20
7.1 La sección .rsrc	...	20
8. Los directorios	...	21
9. Los imports	...	22
10. Los strings	...	23
10.1 Hallazgos principales de los strings	...	23
10.2 Imports visibles en la tabla de importación	...	24
10.3 Búsqueda en los strings de IoCs	...	24
10.4 Conclusión del análisis de strings	...	25
11. Aplicamos Capa Rules	...	26
11.1 Capacidades destacadas identificadas	...	30
11.2 Funciones relevantes identificadas	...	31
11.3 Interpretación de comportamiento	...	31
11.4 Correspondencia con MITRE ATT&CK	...	32
11.5 Matices importantes	...	32
11.6 Funciones que vamos a renombrar en Ghidra	...	32

11.7 Orden de las funciones que vamos a revisar en Ghidra ... 33
11.8 Conclusión de las reglas Capa ... 33
12. Funciones ... 34
12.1 Función `0x402FA0` | `x_install_and_start_threads` ... 34
Creación de directorios ... 34
Obtiene la ruta del ejecutable actual ... 34
Copia el malware como svchost.exe ... 34
Prepara la ruta del screenshot ... 35
Oculta la consola ... 35
Configura persistencia ... 35
Captura screenshot inicial ... 36
Creación del hilo del keylogger ... 36
Crea un segundo hilo ... 37
Espera a que los hilos terminen ... 37
Volvemos a renombrar la función ... 37
Conclusiones ... 37
12.2 Función 0x405580 - x_set_run_key_persistence ... 39
Resumen de la función 405580 ... 39
Qué recibe como parámetros ... 39
Construye el inicio del comando ... 39
Añade el tipo y el dato del registro ... 40
Ejecuta el comando con system ... 40
Limpieza de objetos std::string ... 41
Esta función confirma persistencia clara ... 41
Conclusiones ... 41
12.3 Función 0x402A00 - x_keylogger_loop ... 42
Prólogo de la función ... 42
Inicialización del buffer/stream ... 42
Entrada al bucle de keylogging ... 42
Bucle interno: Recorre códigos de tecla ... 42
Procesamiento de teclas ... 43
¿Por qué no hay RET? ... 43
Conclusiones: ... 44
12.4 Función 0x405AD0 - x_capture_screenshot ... 45
Resumen de la función 405AD0 ... 45
Obtiene el DC de la pantalla ... 45
Crea un DC compatible en memoria ... 45
Crea un bitmap compatible de 1920x1080 ... 45
Selecciona el bitmap en el DC de memoria ... 46
Captura la pantalla con BitBlt ... 46

- Guarda la captura como BMP ... 46
- Limpieza de recursos GDI ... 47
- Sobre RET 0x18 ... 47
- Conclusiones ... 47
- 12.5 Función 0x405850 - x_write_bmp_or_file_data ... 48
- Resumen funcional ... 48
- Creación del fichero BMP ... 48
- Escribe la cabecera BMP ... 48
- Indicadores que se confirman ... 49
- Conclusiones ... 49
- 12.6 Función 0x402C40 ... 50
- Resumen funcional ... 50
- Construye la ruta base del screenshot ... 50
- Espera 50 segundos ... 50
- Construye un nombre numerado ... 50
- La extensión .bmp ...
- Llama a x_capture_screenshot ... 51
- Vuelve a dormir y repite ... 51
- Renombramos la función ... 51
- Relación con el flujo general del malware ... 51
- Conclusiones ... 51
- 12.7 Función 0x401550 ... 51
- 13. Referencias Cruzadas ... 52
- 13.1 XREFs a svchost.exe ... 52
- 0x00408670 ... 53
- 0x004086E8 ... 54
- 13.2 XREFs a Firewall ... 55
- 13.3 XREF a keylogs.txt ... 57
- 14. Técnicas Anti-Análisis ... 58
- 15. Finalizando el análisis estático ... 59
- 15.1 Import GetCurrentProcess ... 59
- 15.2 Import GetCurrentProcessId ... 59
- 16. Análisis Dinámico ... 60
- 16.1 Preparamos la máquina virtual ... 60
- 16.2 Ejecutamos la muestra ... 60
- 16.3 Paramos la muestra ... 61
- 16.4 Comprobaciones ... 61
- Persistencia en el registro ... 61
- Revisamos `keylogs.txt` ... 62
- Revisamos carpeta de screenshots ... 62

Confirmamos que `svchost.exe` copiado es la misma muestra del malware	...	63
Analizamos la comparación de los Shots realizados por RegShot	...	64
Revisamos los procesos	...	64
16.5 Validamos la persistencia tras reinicio	...	67
17. Recopilación de Técnicas MITRE usadas	...	68
18. Indicadores de Compromiso	...	68
19. Análisis dinámico avanzado: x32dbg	...	69
19.1 Persistencia	...	69
19.2 Keylogger	...	72
19.3 Captura de pantalla	...	75
20. Conclusiones	...	76

1. Análisis inicial de la muestra con PEStudio

1.1 Información general:

Sección	Campo	Valor
file	SHA256	AD9427B659A7B77E08566627B82C21AF23F35FA3047356113CFEE764A890AD92
file	First 32 bytes — hex	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00
file	First 32 bytes — text	MZ.....@.....
file	Info	Size: 40960 bytes; Entropy: 6.221
file	Type	Executable, 32-bit, console
file	Version	n/a
file	Description	n/a
entry-point	First 32 bytes — hex	E8 01 04 00 00 E9 74 FE FF FF 55 8B EC 8B 45 08 56 8B 48 3C 03 C8 0F B7 41 14 8D 51 18 03 D0 0F
entry-point	Location	0x000062E4 — section[.text]
file	Signature	Microsoft Linker 14.27
stamps	Compiler	Sun Oct 18 16:09:37 2020 UTC
stamps	Debug	Sun Oct 18 16:09:37 2020 UTC
stamps	Resource	n/a
stamps	Import	n/a
stamps	Export	n/a
names	File name	c:\users\usuario\desktop\master_malware_lab1.exe
debug	File	C:\Users\Admin\Desktop\MALWARE Labs\MASTER_Malware_Lab1\Release\MASTER_Malware_Lab1.pdb

Conclusión: Binario PE32 de consola, compilado con Visual Studio/MSVC, sin metadatos legítimos relevantes, no empaquetado de forma evidente y con ruta PDB expuesta que facilita el análisis estático.

1.2 INDICATORS

Sección	Campo	Valor
file	name	c:\users\usuario\desktop\master_malware_lab1.exe
file	signature	Microsoft Linker 14.27
file	sha256	AD9427B659A7B77E08566627B82C21AF23F35FA3047356113CFEE764A890AD92
file	info	size: 40960 bytes; entropy: 6.221
file	type	executable, 32-bit, console
virustotal	score	No se pudo resolver el nombre de servidor o su dirección
stamp	compiler	Sun Oct 18 16:09:37 2020
languages	names	English-US
resources	info	count: 1; size: 381 bytes; file-ratio: 0.93%
manifest	general	name: n/a; description: n/a; severity: asInvoker

Sección	Campo	Valor
file	version	n/a
entry-point	location	0x000062E4 — section: .text
certificate	—	n/a
imports	flag	CopyFileW; GetAsyncKeyState; GetCurrentProcess; GetCurrentProcessId; GetCurrentThreadId; WriteFile
imphash	md5	962E2DBBC5BB67A0F87FCD14DE36453F
exports	—	n/a
overlay	—	n/a

Conclusión: los indicadores de PEStudio apuntan a un ejecutable `PE32` no firmado, sin overlay ni exports, con recursos mínimos y con imports sospechosos como: `CopyFileW; GetAsyncKeyState; GetCurrentProcess; GetCurrentProcessId...`.

1.3 FOOTPRINTING - Huellas Dactilares

Sección	Campo	Valor
file	sha256	AD9427B659A7B77E08566627B82C21AF23F35FA3047356113CFEE764A890AD92
dos-stub	sha256	EBFBB9BC5FA6B4B4E8B64C3EA3D97EDFEB11B3EBDE005FE63FD58D074F81F86D
dos-header	sha256	9A032ADF039736A7CC5306E76BB5CB8C9426CF0E4613647E488C0CB3E44A7BD6
rich-header	sha256	0314F0F21BEA0FF87373BFB0D0A96FBF254320CB37FE159B4873F14C0E00ECE6
section	.text > sha256	8D86FADC0B0E41A060E00FE9206F962DE5A9FF4B67A960DE02862F928561DFD6
section	.rdata > sha256	0EE327063C47F9A7AC8D6BB96B60C5C97231CD2C6F28E3FE8CAEC811E3F369B4
section	.data > sha256	71D776C9BF050969EBB2BFF994E3F32E45345E585F75E8AC76F8D6B99B4DE6A7
section	.rsrc > sha256	E19CB20DBBC9477BB9D5F4675629F3BFA179975001EDA72616F048D46702F466
section	.reloc > sha256	61B681E520209648D877C46172BA575DD96CFA4E679FA0A99530F554C4B95E18
manifest	sha256	4BB79DCEA0A901F7D9EAC5AA05728AE92ACB42E0CB22E5DD14134F4421A3D8DF
debug	RSDS > sha256	93DE24C020C461809D04BF055839CEA0BAA5DC305D443CC4706C7870F43C9A54
debug	vcFeature > sha256	EF06E0DC71B1AFE4607EBF3EA5C8943D16D324A32C71C17ABBD2F0E2C82820A8
debug	PGO > sha256	BA2DF318794DD7B1215FC206795BC82B6B91DF85DB82A1A2152091C5C57F4E95
special	imphash > md5	962E2DBBC5BB67A0F87FCD14DE36453F

1.4 DOS-HEADER

Sección	Campo	Valor
dos-header	sha256	9A032ADF039736A7CC5306E76BB5CB8C9426CF0E4613647E488C0CB3E44A7BD6
dos-header	size	0x40 — 64 bytes
dos-header	location	0x00000000 - 0x00000040
dos-header	entropy	4.624
file	ratio	0.00 %
exe-header	offset	0x000000F8 — e_lfanew

Conclusión: el DOS header es válido y no muestra indicios de manipulación, empaquetado u ocultación. El campo `e_lfanew` del DOS header apunta a `0x000000F8`, indicando que la cabecera `PE` comienza en ese offset. Este valor es coherente con una estructura `PE` válida, ya que en dicha posición se espera encontrar la firma `PE\0\0`.

1.5 DOS-STUB

El DOS stub es la pequeña porción de código que se ejecutaría si el fichero se intentara abrir en un entorno DOS real:

Sección	Campo	Valor
dos-stub	sha256	EBFBB9BC5FA6B4B4E8B64C3EA3D97EDFEB11B3EBDE005FE63FD58D074F81F86D
dos-stub	location	0x00000040 - 0x000000F8
dos-stub	size	0xB8 — 184 bytes
dos-stub	entropy	5.123
file	ratio	0.45 %
dos-stub	first 32 bytes — hex	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F
dos-stub	first 32 bytes — text	.....!....L..!This program canno
dos-stub	message	!This program cannot be run in DOS mode.

Conclusión: el DOS stub es estándar y contiene el mensaje habitual de compatibilidad DOS. No se observan datos anómalos, payloads ocultos ni indicios de manipulación en esta región del fichero.

1.6 RICH HEADER

El Rich Header contiene metadatos generados por el toolchain de Microsoft durante la compilación/enlazado del binario. En esta muestra aparecen componentes asociados principalmente a Visual Studio 2015 / MSVC 14.0, como:

Componente	Herramienta / Firma
Implib900	Visual Studio 2008 - 9.0
Utc1900_C	Visual Studio 2015 - 14.0
Masm1400	Visual Studio 2015 - 14.0
Utc1900_CPP	Visual Studio 2015 - 14.0
Implib1400	Visual Studio 2015 - 14.0
Implib1400	Visual Studio 2015 - 14.0
Import	Visual Studio -
Utc1900_LTCG_CPP	Visual Studio 2015 - 14.0
Cvtres1400	Visual Studio 2015 - 14.0
Linker1400	Visual Studio 2015 - 14.0

Sección	Campo	Valor
rich-header	location	0x00000080 - 0x000000F8
rich-header	size	0x00000078 — 120 bytes
rich-header	checksum-builtin	0x61E35C71
rich-header	checksum-real	0x61E35C71
rich-header	sha256	0314F0F21BEA0FF87373BFB0D0A96FBF254320CB37FE159B4873F14C0E00ECE6

Conclusión: el Rich Header es coherente con un binario compilado con herramientas de Microsoft Visual Studio, principalmente `MSVC 14.0 / Visual Studio 2015`. No se observan indicios claros de alteración del `Rich Header`. La coincidencia entre `checksum-builtin` y `checksum-real` sugiere que esta estructura es válida.

1.7 FILE HEADER

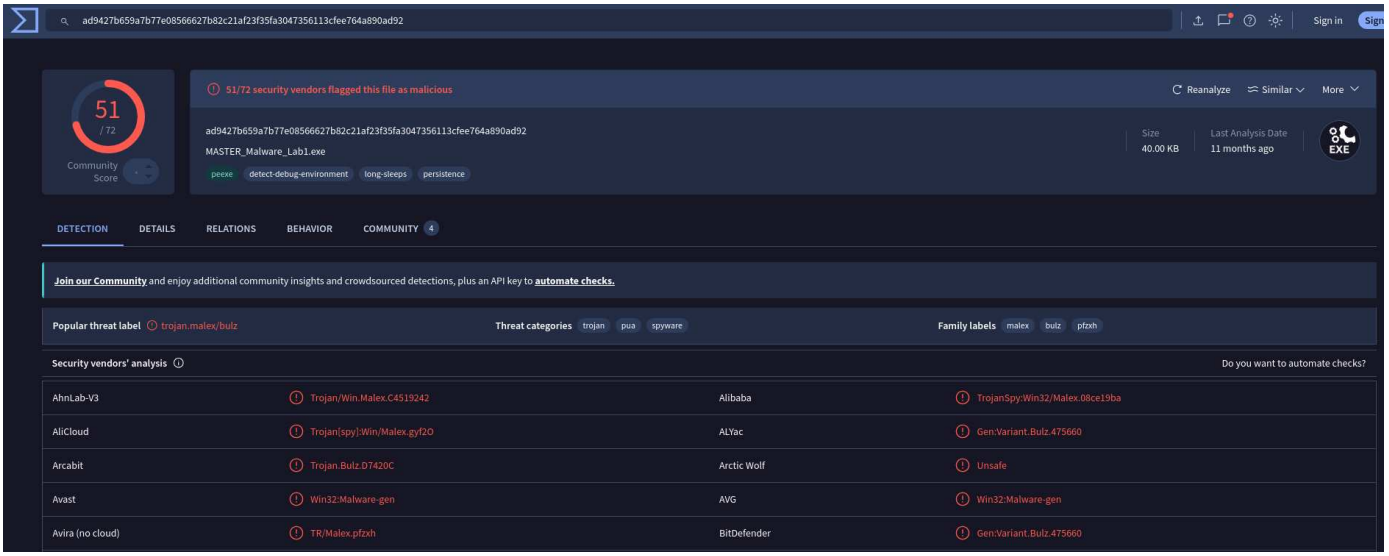
Sección	Campo	Valor hexadecimal	Valor interpretado
characteristics	characteristics	0x0102	—
characteristics	dynamic-link-library	0x0000	false
characteristics	32-bit words support	0x0100	true
characteristics	file-can-be-executed	0x0002	true
characteristics	system-image	0x0000	false
characteristics	large-address-aware	0x0000	false
characteristics	debug-stripped	0x0000	false
characteristics	line-stripped-from-file	0x0000	false
characteristics	local-symbols-stripped-from-file	0x0000	false
characteristics	relocation-stripped	0x0000	false
characteristics	uniprocessor	0x0000	false
characteristics	bytes-of-machine-words-reversed-Low	0x0000	false
characteristics	bytes-of-machine-words-reversed-Hi	0x0000	false
characteristics	media-run-from-swap	0x0000	false
characteristics	network-run-from-swap	0x0000	false

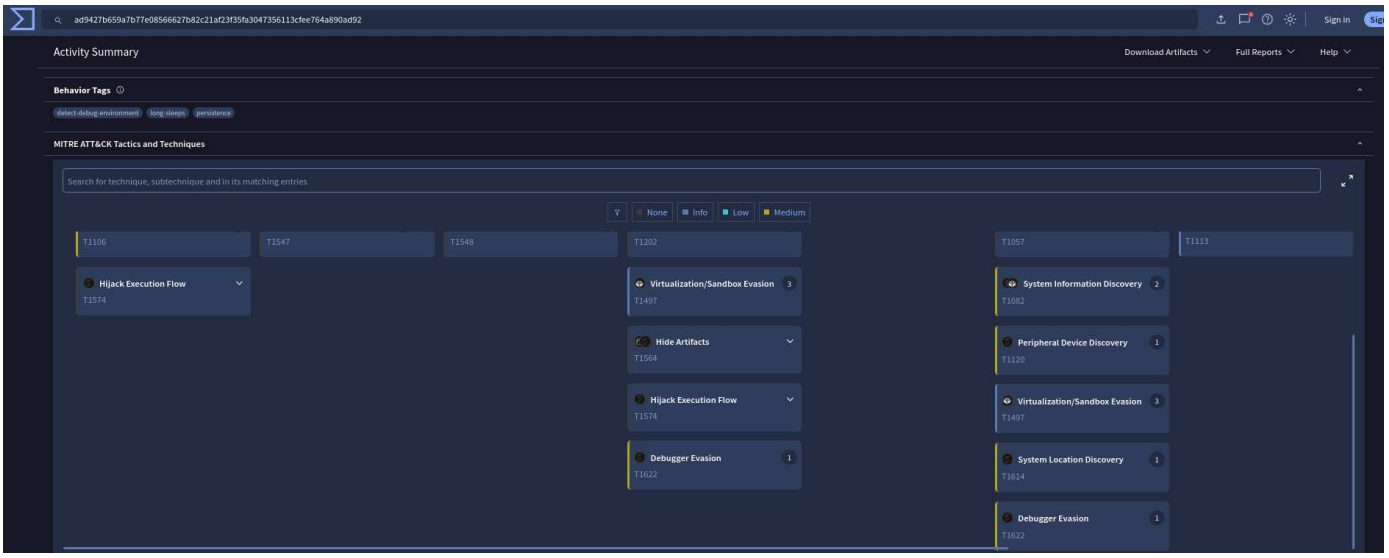
Conclusión: el File Header confirma que la muestra es un ejecutable PE32 válido, no DLL, no imagen de sistema y con soporte de relocalización e información de depuración no completamente eliminada. No se observan flags anómalos en esta cabecera.

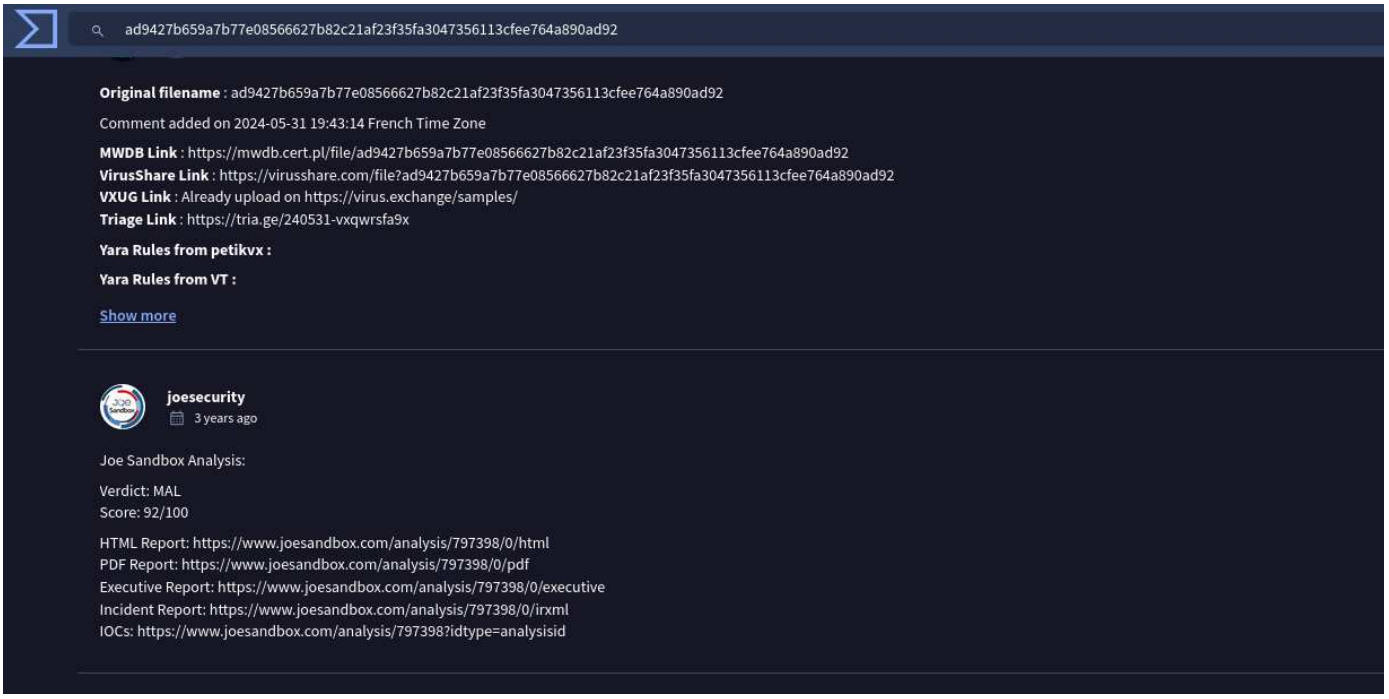
---

2. Analizamos la muestra en VirusTotal

Informe en VirusTotal





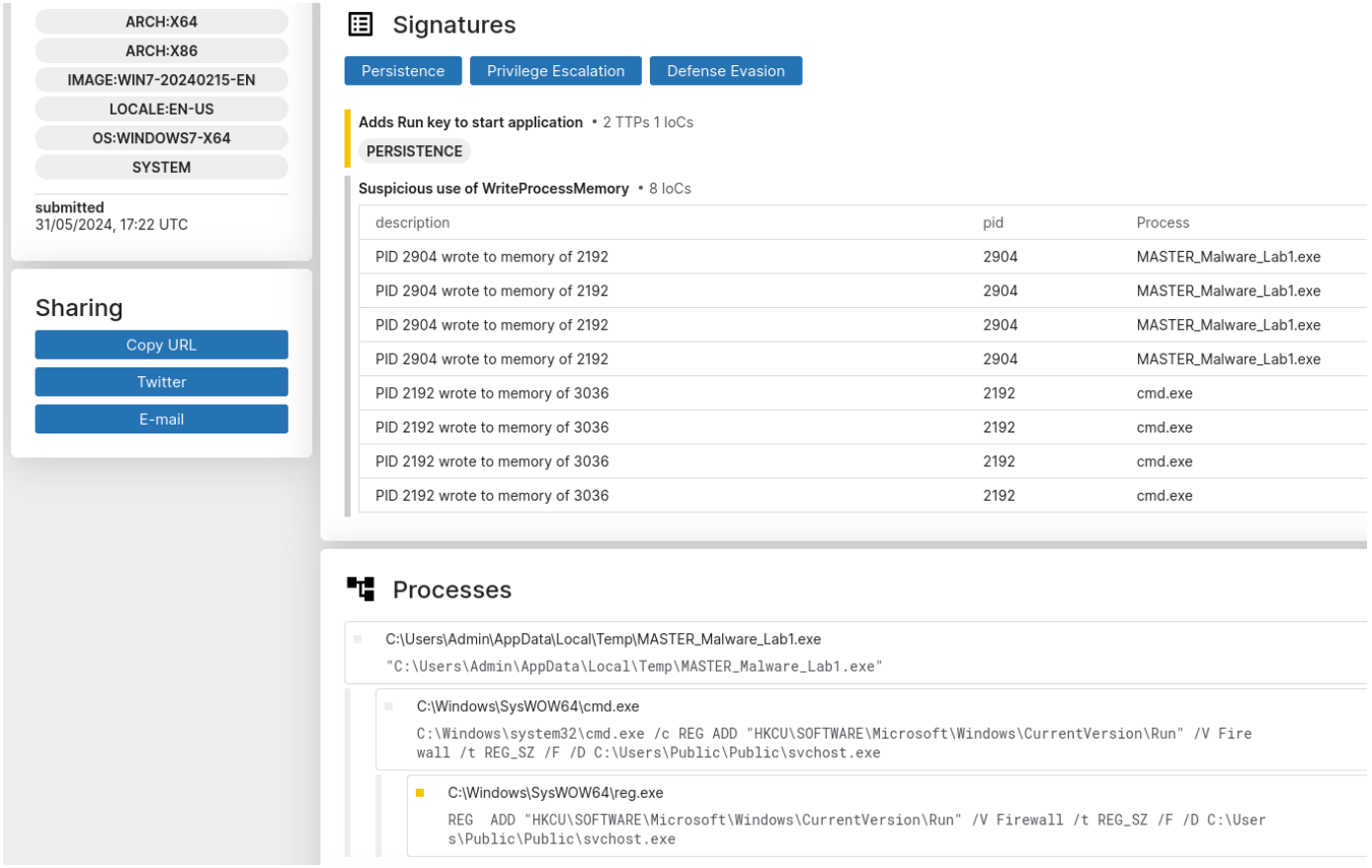


donde:

- Comprobamos que hay varios informes ya realizados de la muestra:
  - Vemos un informe en `Tria.Ge`.
  - Si seguimos el informe en Joe SandBox se comprueba un error. El informe que aparece, no se corresponde con nuestra muestra.

2.1 Vemos el informe en Tria.Ge

Informe en tria.ge



La captura de Triage confirma dinámicamente varias conclusiones del análisis estático:

Hallazgo en Triage	**Interpretación**
Adds Run key to start application	Confirma persistencia mediante clave `Run`.
Proceso `cmd.exe` ejecuta `REG ADD`	La muestra usa `system()` o equivalente para lanzar un comando de registro.
Comando observado	`REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D C:\Users\Public\Public\svchost.exe`
Valor creado	`Firewall`
Ejecutable persistente	`C:\Users\Public\Public\svchost.exe`
Proceso inicial	`MASTER_Malware_Lab1.exe` desde `AppData\Local\Temp`
Proceso hijo	`cmd.exe`
Proceso posterior	`reg.exe`
Firma “Suspicious use of WriteProcessMemory”	Indica escritura en memoria de otro proceso; habría que revisar si es inyección real o comportamiento asociado a creación de procesos/runtime.

El análisis dinámico en Triage confirma la técnica de persistencia identificada durante el análisis estático. La muestra ejecuta `cmd.exe` , que a su vez lanza `reg.exe` para añadir una entrada en `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run` con el nombre `Firewall`, apuntando a `C:\Users\Public\Public\svchost.exe`.

---

3. Primer análisis de la muestra

3.1 El tipo de fichero

El fichero corresponde a un ejecutable **PE de Windows de 32 bits**, compilado como aplicación de consola.

Entre la información que aporta la herramienta PEStudio, destacamos:

Atributo	Análisis
Tipo de fichero	Ejecutable Windows PE
Arquitectura	32 bits
Plataforma objetivo	Windows
Subsistema probable	Consola
Formato	PE32
Cabecera inicial	`MZ`
Firma PE	`PE00`
Máquina	`Intel-386` / `0x014C`
Extensión observada	`.exe`
Nombre del fichero	`master_malware_lab1.exe`
Tamaño	40960 bytes
Entropía global	6.221
Secciones	5
Sección de entrada	`.text`
Entry point	`0x000062E4`
DLL	No
Ejecutable	Sí
Certificado digital	No presente
Overlay	No presente

3.2 Clasificación técnica

Categoría	Resultado
Tipo general	Binario ejecutable nativo
Familia de formato	Portable Executable — PE
Compatibilidad	Windows x86
Modo de ejecución	Aplicación de usuario
Tipo de interfaz	Consola
Empaquetado aparente	No hay indicios claros de packer por los datos dados
Compilador / linker	Microsoft Linker 14.27 / Visual Studio
Fecha de compilación	Sun Oct 18 16:09:37 2020 UTC

3.3 Evidencias principales

El fichero empieza con la firma `MZ`, típica de ejecutables DOS/Windows `PE`:

Indicador	Valor
Primeros bytes	`4D 5A 90 00...`
Texto interpretado	`MZ...`

`MZ` es la firma mágica de los archivos ejecutables en DOS/Windows (archivos .exe). Llamada así por **Mark Zbikowski**, ingeniero de Microsoft. Indica que el archivo es un ejecutable `PE` (Portable Executable) o que al menos comienza con una cabecera compatible con DOS.

3.4 Mensaje DOS stub habitual

Campo	Valor
DOS stub message	`This program cannot be run in DOS mode.`

Esto confirma que se trata de un ejecutable PE estándar de Windows.

3.5 La cabecera PE

Campo	Valor	Interpretación
`e_lfanew`	`0x000000F8`	Offset donde empieza la cabecera PE
Signature	`0x00004550`	Firma `PE00`
Machine	`0x014C`	Intel 386 / x86
Sections count	`0x0005`	5 secciones
Characteristics	`0x0102`	Ejecutable de 32 bits
File can be executed	`true`	Es ejecutable
32-bit words support	`true`	Binario de 32 bits
Dynamic-link-library	`false`	No es una DLL

3.6 Secciones detectadas

Sección	Interpretación
`.text`	Código ejecutable
`.rdata`	Datos de solo lectura, imports, strings o constantes
`.data`	Datos modificables
`.rsrc`	Recursos del ejecutable
`.reloc`	Información de relocalización

La presencia de .text, .rdata, .data, .rsrc y .reloc es típica de un ejecutable PE compilado con Visual Studio.

3.7 Conclusiones

La muestra analizada corresponde a un fichero ejecutable Portable Executable de Windows en formato PE32, orientado a arquitectura Intel x86. El binario está compilado como aplicación de consola, no presenta firma digital, no contiene `overlay` y posee cinco secciones estándar: `.text`, `.rdata`, `.data`, `.rsrc` y `.reloc`. El punto de entrada se encuentra en la sección `.text`, lo que sugiere una estructura `PE` convencional sin indicios evidentes de empaquetado a partir de los metadatos observados.

---

4. Analizamos si la muestra está empaquetada

No hay indicios fuertes de empaquetado clásico:

Evidencia de la muestra	Valor	Interpretación
Tipo	PE32, consola, x86	Estructura normal de ejecutable Windows
Entropía global	`6.221`	Moderada. No es típica de un packer fuertemente comprimido
Entry point	`0x000062E4`, sección `.text`	Normal. Muchos packers usan secciones no estándar o EP anómalo
Secciones	`.text`, `.rdata`, `.data`, `.rsrc`, `.reloc`	Nombres estándar de Visual Studio
Overlay	`n/a`	No hay datos añadidos al final del fichero
Rich Header	Visual Studio 2015 / linker MSVC	Compatible con binario compilado normalmente
Imports sospechosos	`CopyFileW`, `GetAsyncKeyState`, `WriteFile`	Sugieren comportamiento malicioso, pero no empaquetado por sí mismos

La entropía 6.221 puede indicar que el binario contiene código compilado normal, recursos o datos algo densos, pero no prueba empaquetado. En muchos ejecutables empaquetados la entropía suele ser más alta, especialmente en secciones completas.

Cosas que conviene analizar para determinar si está empaquetado:

Pestaña / campo	Qué buscar
`sections`	Entropía por sección, permisos, tamaños raw/virtual
`indicators`	Alertas de packer, entropy, suspicious imports
`imports`	Si importa muchas APIs normales o solo unas pocas APIs de carga dinámica
`strings`	Cantidad y calidad de cadenas legibles
`resources`	Recursos grandes o con entropía elevada
`overlay`	Presencia de datos añadidos. Hemos comprobado que no tiene overlay
`debug`	Rutas PDB, información de compilación
`rich-header`	Coherencia con compilador normal

donde:

- Una muestra no empaquetada suele tener muchas imports claras, secciones estándar y strings legibles.
- Una muestra empaquetada suele mostrar pocas imports, strings pobres y una sección de alta entropía que actúa como contenedor.

Herramientas interesantes para usar en la muestra:

Herramienta	Uso
Detect It Easy / DIE	Detección rápida de packers y compiladores
PEStudio	Revisión de indicadores PE
PE-bear	Inspección manual de cabeceras y secciones
CFF Explorer	Análisis de estructura PE
capa	Identificación de capacidades del malware
FLOSS	Extracción de strings ofuscadas
strings / floss	Comparar strings normales frente a strings recuperadas
Ghidra / IDA Free	Revisión del código desensamblado
x32dbg / x64dbg	Ver si se desempaqueta en memoria

Herramienta	Uso
Process Monitor	Observar actividad en ejecución controlada
ProcDump / Scylla	Volcado de memoria de un binario desempaquetado

4.1 Conclusiones del empaquetamiento

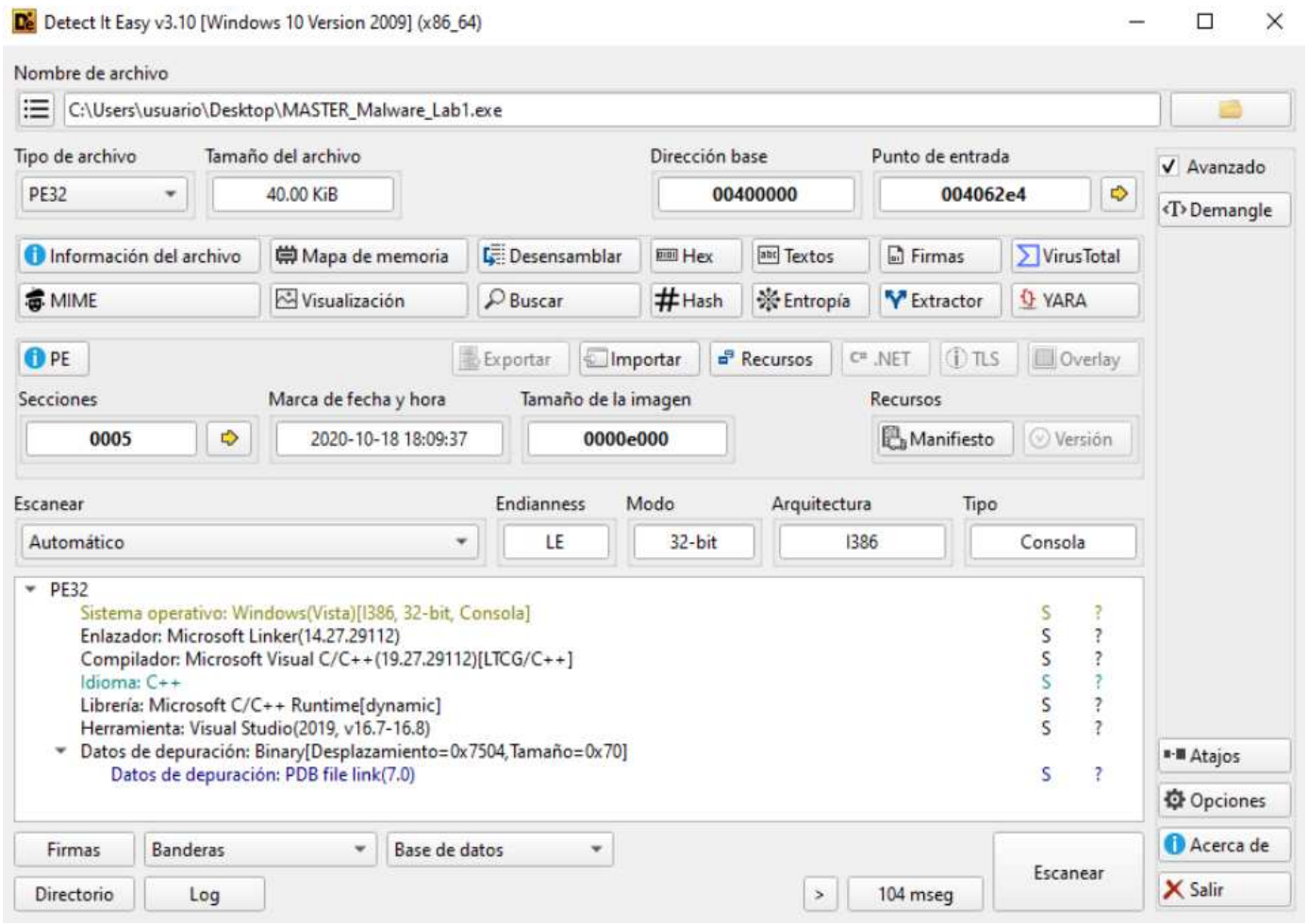
Para determinar si la muestra se encuentra empaquetada u ofuscada, se revisaron los indicadores estructurales del formato `PE`, incluyendo entropía, secciones, punto de entrada, imports, overlay y metadatos de compilación. La muestra presenta secciones estándar de Visual Studio, punto de entrada ubicado en `.text`, ausencia de overlay y una entropía global moderada de `6.221`, por lo que no se observan indicios fuertes de empaquetado clásico.

No obstante, la presencia de un conjunto reducido de imports y de funciones como `GetAsyncKeyState`, `CopyFileW` y `WriteFile` requiere un análisis dinámico para confirmar si existe resolución de APIs en tiempo de ejecución, desempaquetado en memoria u ofuscación de strings.

No parece empaquetada de forma evidente, pero conviene confirmar con `Detect It Easy`, entropía por sección, `FLOSS` y análisis dinámico en `x32dbg`.

---

5. Analizamos la muestra con Detect It Easy



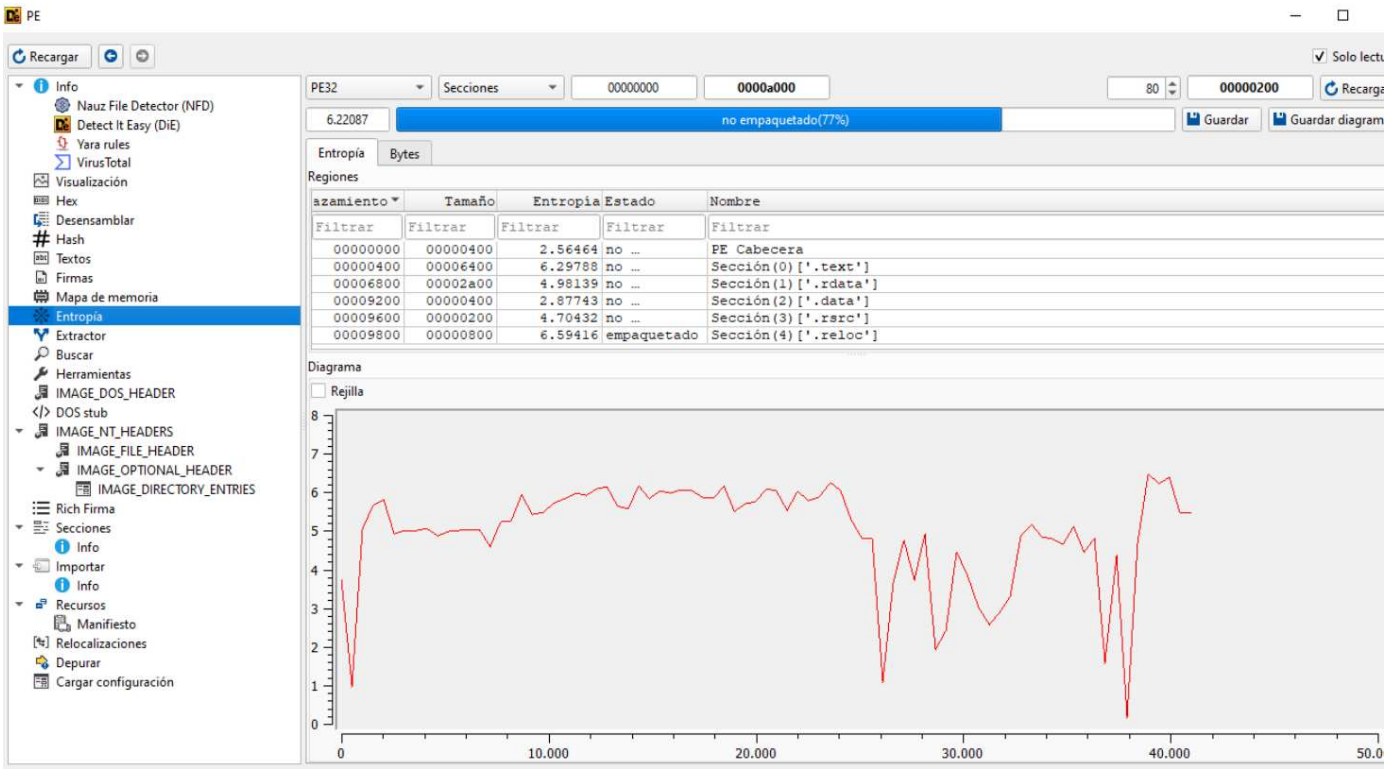
La muestra no parece empaquetada ni protegida con un packer conocido según `Detect It Easy`. DIE identifica el binario como un PE32 compilado con Microsoft Visual C/C++ y enlazado con Microsoft Linker, sin detectar firmas de packers como UPX, Themida, VMProtect, ASPack u otros protectores.

Evidencias que confirma Detect It Easy:

Indicador	Valor observado	Interpretación
Tipo de fichero	`PE32`	Ejecutable Windows de 32 bits
Arquitectura	`I386`	Binario x86
Modo	`32-bit`	Compatible con Windows 32-bit
Tipo	`Consola`	Aplicación de consola
Tamaño	`40.00 KiB`	Tamaño pequeño, pero no implica packing por sí solo
Secciones	`0005`	Número normal para un ejecutable PE compilado
Punto de entrada	`004062E4`	Punto de entrada normal dentro del rango del PE
Enlazador	`Microsoft Linker 14.27.29112`	Indica compilación con toolchain de Microsoft
Compilador	`Microsoft Visual C/C++ 19.27... [LTCG/C++]`	Código C/C++ compilado normalmente
Librería	`Microsoft C/C++ Runtime [dynamic]`	Uso de runtime dinámico estándar
Herramienta	`Visual Studio 2019, v16.7-16.8`	Toolchain legítimo/normal
Datos de depuración	`PDB file link`	Presencia de información de depuración, poco habitual en malware muy ofuscado
Overlay	Botón deshabilitado / sin overlay visible	No se observa overlay añadido

DIE no muestra ninguna firma de packer/protector. En su lugar, detecta `toolchain` de compilación normal de Microsoft.

5.1 Analizamos la entropía con DIE:



Región / sección	Offset	Tamaño	Entropía	Estado según DIE	Interpretación
PE Cabecera	`00000000`	`00000400`	`2.56464`	no empaquetado	Normal. Las cabeceras PE suelen tener baja entropía.
`.text`	`00000400`	`00006400`	`6.29788`	no empaquetado	Normal/moderada para código compilado.
`.rdata`	`00006800`	`00002A00`	`4.98139`	no empaquetado	Normal para datos de solo lectura, imports, strings, constantes.

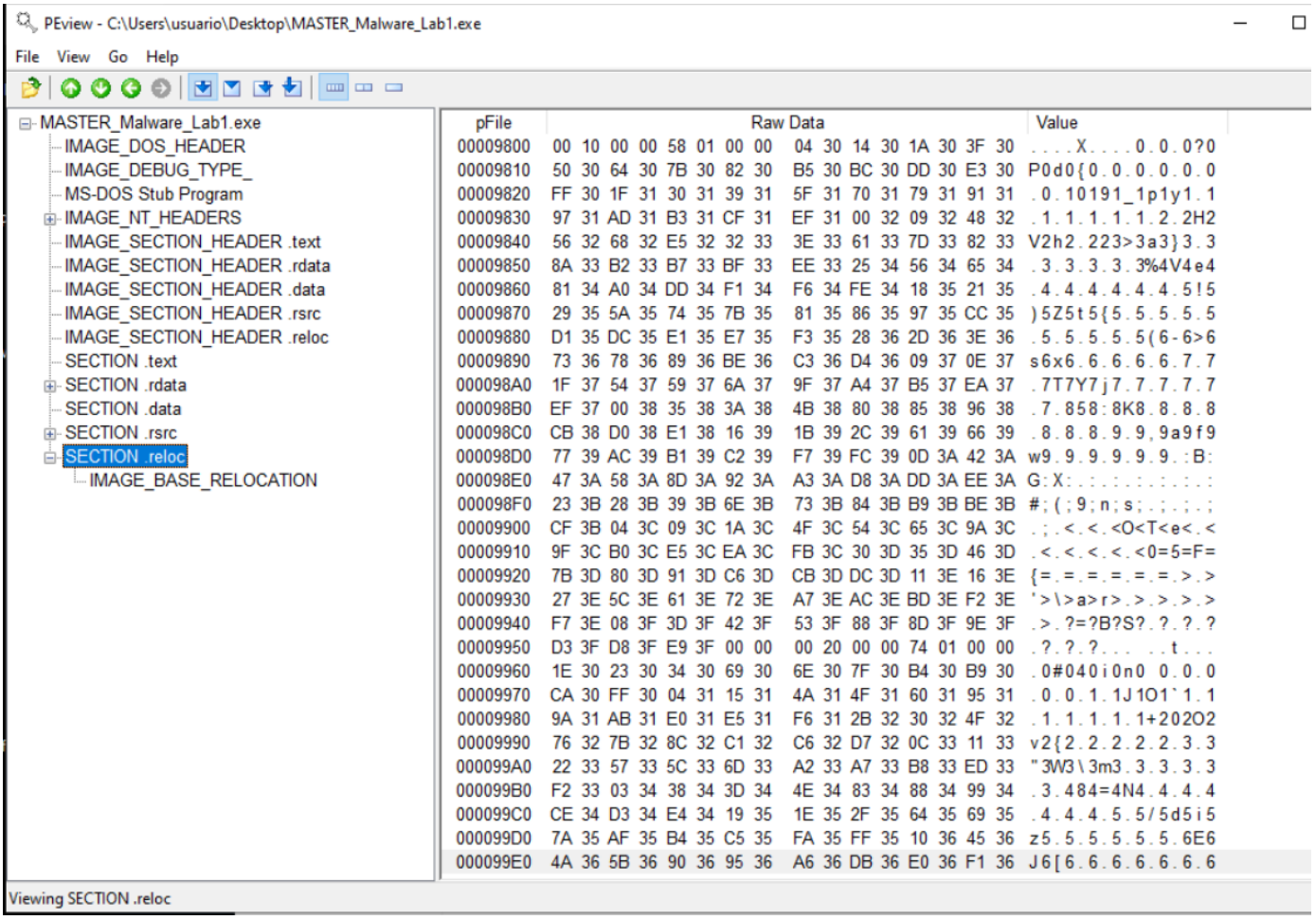
Región / sección	Offset	Tamaño	Entropía	Estado según DIE	Interpretación
`.data`	`00009200`	`00000400`	`2.87743`	no empaquetado	Baja entropía, compatible con datos inicializados.
`.rsrc`	`00009600`	`00000200`	`4.70432`	no empaquetado	Normal para recursos pequeños.
`.reloc`	`00009800`	`00000800`	`6.59416`	**empaquetado**	**Sospechoso de forma aislada, pero no concluyente.**

La sección `.reloc` contiene información de relocalización del `PE`. Puede tener una entropía relativamente elevada dependiendo de cómo estén distribuidos los datos, y además es una sección pequeña. En secciones pequeñas, las heurísticas de entropía pueden producir falsos positivos.

El análisis de entropía realizado con Detect It Easy muestra una entropía global de 6.22087, clasificando la muestra como `no empaquetada`. La mayoría de las secciones presentan valores de entropía compatibles con un ejecutable `PE32` compilado normalmente. **La sección .`reloc` aparece marcada como `empaquetado` con una entropía de 6.59416**, aunque por su pequeño tamaño y función dentro del formato `PE`, este indicador aislado no permite concluir la presencia de un packer. No se observan secciones con entropía extremadamente alta ni patrones típicos de empaquetadores conocidos.

Conclusión final: La muestra no presenta evidencias claras de empaquetado. La marca sobre `.reloc` debe tratarse como un indicador débil o posible falso positivo, no como prueba de packing.

5.2 Investigamos `.reloc` con la herramienta PEview:



La sección `.reloc` ha sido revisada con `PEview`. Aunque `Detect It Easy` marcó esta sección como `empaquetada` por su entropía, `PEview` identifica correctamente la estructura `IMAGE_BASE_RELOCATION`. Los datos iniciales corresponden a bloques de relocalización válidos, con campos `VirtualAddress` y `SizeOfBlock` coherentes y entradas compatibles con relocalizaciones `PE32` tipo `HIGHLOW`. Por tanto, la alerta de entropía sobre `.reloc` se considera un falso positivo local y no una evidencia de empaquetado.

---

6. Decodificación de cadenas ofuscadas usando FLOSS

FireEye Labs Obfuscated String Solver (FLOSS) es una herramienta diseñada para identificar y extraer automáticamente cadenas ofuscadas de malware. Puede ayudarnos a determinar las cadenas que los autores de malware quieren ocultar de las herramientas de extracción de cadenas.

FLOSS también se puede utilizar como la utilidad de cadenas para extraer cadenas legibles (ASCII y Unicode). Para descargar FLOSS para Windows o Linux: https://github.com/fireeye/flare-floss.

Instalamos FLOSS en un entorno virtual Python:

```
sudo apt update
sudo apt install -y python3 python3-pip python3-venv

mkdir -p ~/miENV-FLOSS
cd ~/miENV-FLOSS

python3 -m venv venv-floss
source ~/miENV-FLOSS/venv-floss/bin/activate

python -m pip install --upgrade pip setuptools wheel
pip install flare-floss

floss --version
which floss
```

donde:

- Creamos un entorno virtual Python.
- Instalamos FLOSS con `pip`.
- Comprobamos que ha sido instalado correctamente.

Ahora analizamos la muestra: Dentro del entorno virtual Python ejecutamos:

```
cd ~/lugar-donde-esté-la-muestra-de-malware
floss MASTER_Malware_Lab1.exe > MASTER_Malware_Lab1-FLOSS.txt
```

donde:

- Se obtiene el fichero MASTER_Malware_Lab1-FLOSS.txt.

Destacamos de los resultados obtenidos con FLOSS:

```
FLARE FLOSS RESULTS (version 3.1.1)

+------------------------+-------------------------------------------------------------------------------
---------+
| file path              | MASTER_Malware_Lab1.exe
|
| identified language    | unknown
|
| extracted strings      |
|
|  static strings        | 414 (6896 characters)
|
|   language strings     |   0 (   0 characters)
|
|  stack strings         | 1
|
|  tight strings         | 0
```

```
|
|  decoded strings       | 0
|
+------------------------+-------------------------------------------------------------------------------
---------+


 ________________________________
  FLOSS STATIC STRINGS (414)
 ________________________________

+----------------------------------+
| FLOSS STATIC STRINGS: ASCII (412) |
+----------------------------------+

!This program cannot be run in DOS mode.
ax$pa}\
aRichq\
...
...


+----------------------------------+
| FLOSS STATIC STRINGS: UTF-16LE (2) |
+----------------------------------+

@jjj
C:\Users\Public\Public\svchost.exe


 ____________________________
  FLOSS STACK STRINGS (1)
 ____________________________

ineIGenu


 ____________________________
  FLOSS TIGHT STRINGS (0)
 ____________________________

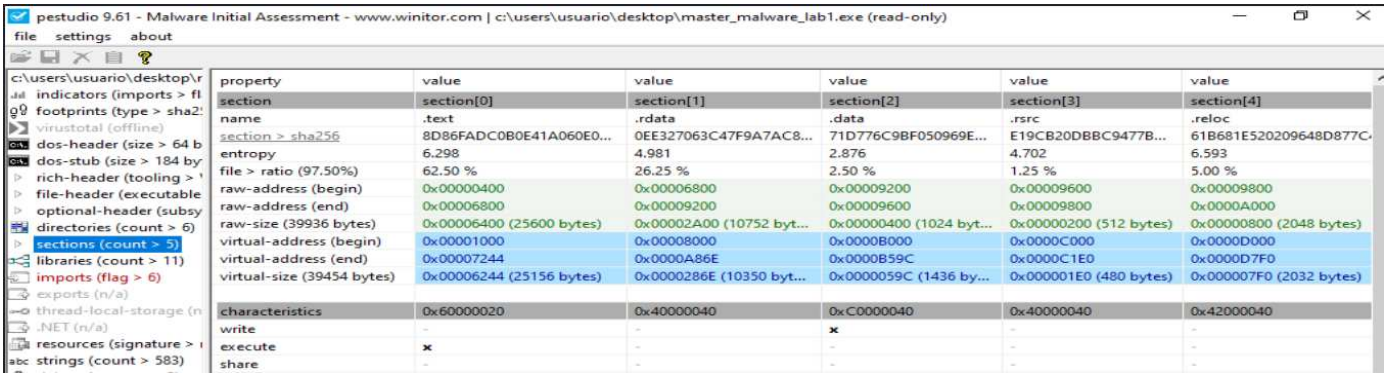


 ________________________________
  FLOSS DECODED STRINGS (0)
 ________________________________
```

- donde aparte de los strings que analizaremos en un apartado posterior, obtenemos que:
  - La muestra no parece usar ofuscación de strings relevantes.
  - FLOSS no ha encontrado cadenas decodificadas ni rutinas evidentes de descifrado de strings.
  - `decoded strings: 0`: FLOSS no ha identificado strings ofuscadas que sean decodificadas en tiempo de ejecución.

---

7. Las secciones



Las secciones tienen una estructura normal para un `PE32` compilado con Visual Studio. No se observan nombres extraños, secciones `RWX` ni indicios fuertes de packing.

Sección	Entropía	Tamaño raw	Permisos	Interpretación
`.text`	`6.298`	`0x6400` / 25600 bytes	Ejecutable	Contiene el código. Entropía normal-media/alta para código compilado.
`.rdata`	`4.981`	`0x2A00` / 10752 bytes	Lectura	Datos de solo lectura: strings, imports, constantes, RTTI/C++ runtime.
`.data`	`2.876`	`0x400` / 1024 bytes	Lectura/escritura	Datos globales modificables. Entropía baja, normal.
`.rsrc`	`4.702`	`0x200` / 512 bytes	Lectura	Recursos. Tamaño pequeño, sin indicios de payload embebido grande.
`.reloc`	`6.593`	`0x800` / 2048 bytes	Lectura / descartable	Relocalizaciones. Entropía algo elevada, pero ya vimos que contiene estructuras `IMAGE_BASE_RELOCATION` válidas.

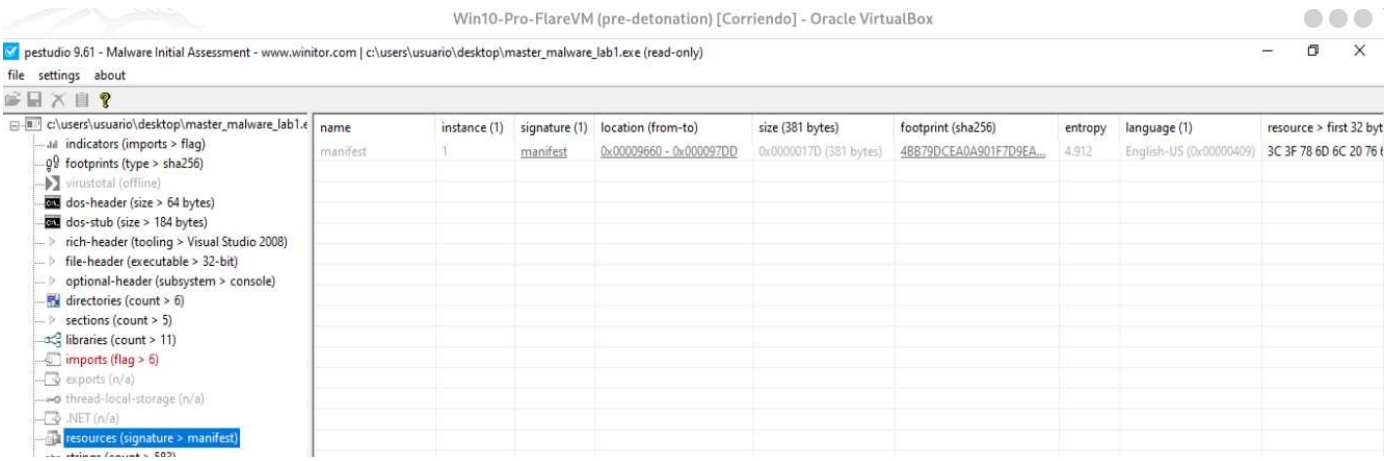
Observaciones relevantes:

- La sección principal es `.text`. Es la única sección marcada como ejecutable, lo que es normal.
- La sección `.data` es la única con permiso de escritura, lo cual también es esperable. No se observa una sección con permisos simultáneos de `write + execute`, que sería sospechoso.
- La sección `.reloc` tiene una entropía relativamente alta (6.593), pero ya analizamos esta sección y comprobamos que no hay evidencias de empaquetado.

7.1 La sección .rsrc

Destacamos esta sección ya que es donde podemos encontrar:

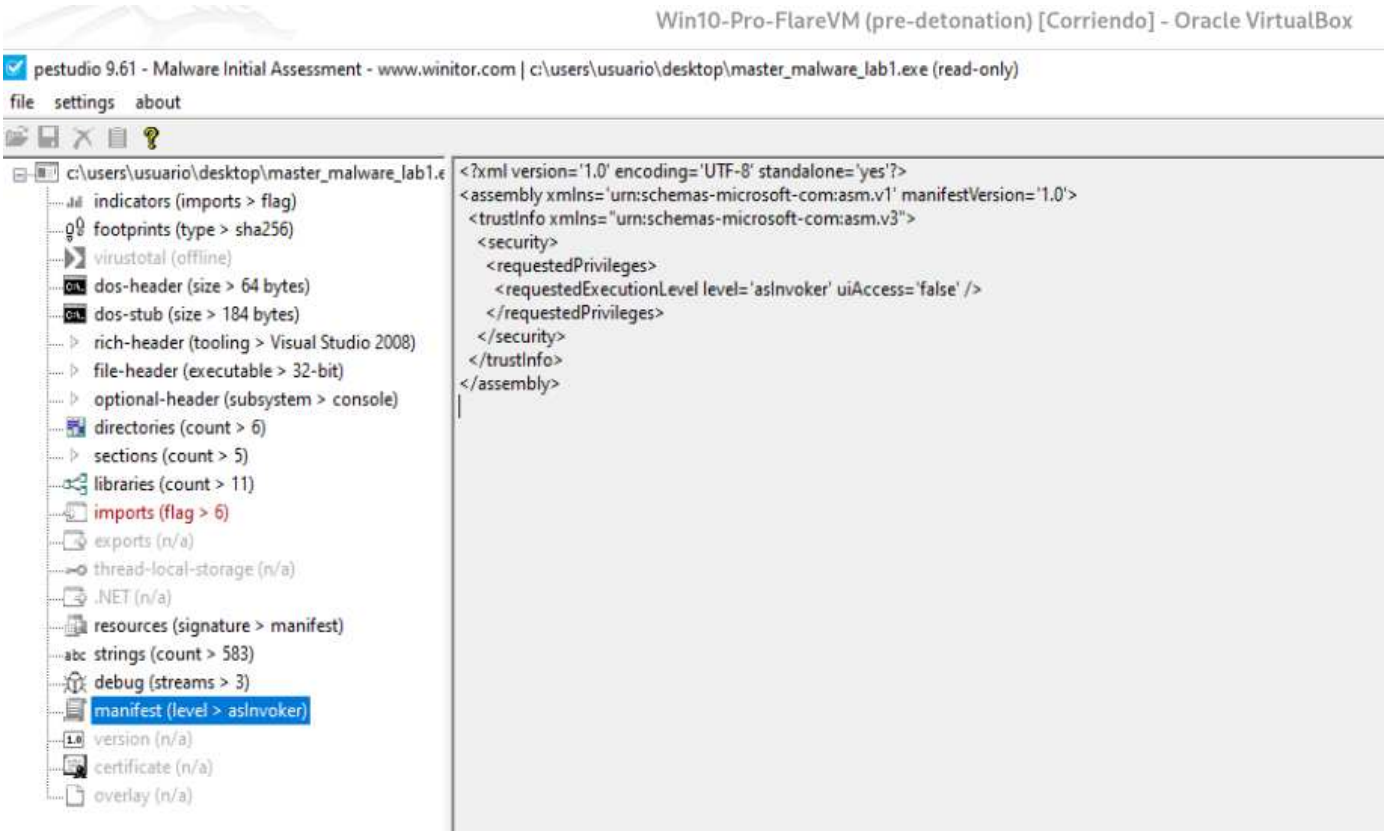
- Recursos normales, como un manifiesto.
- Para comprobar si oculta datos relevantes: payloads, configuración, iconos modificados, blobs cifrados, scripts, DLLs, etc.



donde:

- La muestra parece tener un recurso pequeño asociado al manifiesto, sin indicios claros de payload embebido.

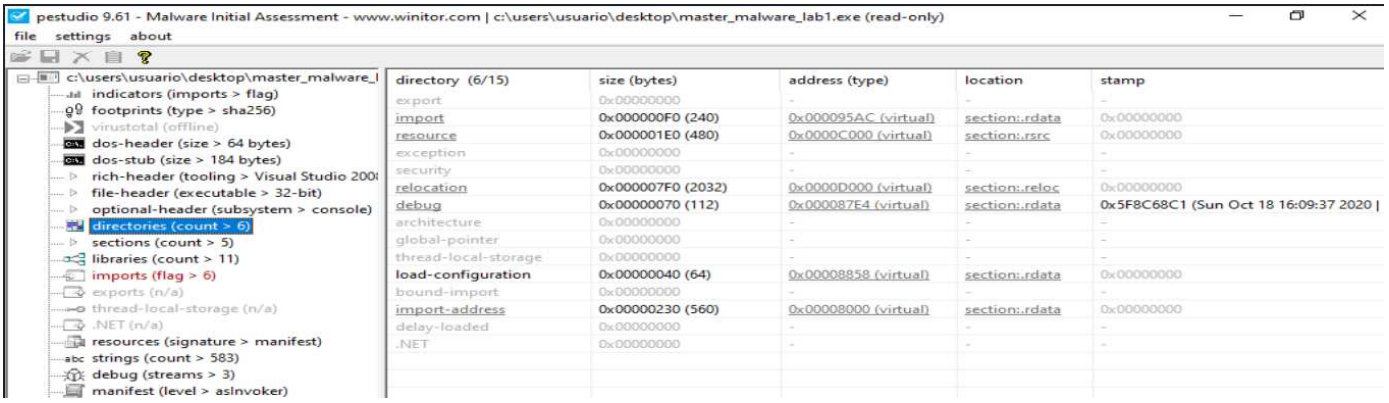
Analizamos el manifiesto:



Campo	Valor	Interpretación
`requestedExecutionLevel`	`asInvoker`	El binario se ejecuta con los privilegios del usuario que lo lanza.
`uiAccess`	`false`	No solicita acceso especial a la interfaz gráfica ni a ventanas elevadas.
Elevación UAC	No solicitada	No intenta pedir permisos de administrador mediante el manifest.
Tipo de recurso	Manifest XML	Recurso normal en ejecutables Windows.
Indicio de payload embebido	No	No hay datos binarios, script ni configuración oculta en este recurso.

El análisis del recurso manifest muestra que la muestra declara `requestedExecutionLevel` como `asInvoker` y `uiAccess` como `false`. Esto indica que el ejecutable no solicita elevación de privilegios mediante `UAC` y se ejecuta con los permisos del usuario actual. Este comportamiento es coherente con la persistencia observada en `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`, que no requiere privilegios administrativos. No se identifican payloads, scripts ni configuraciones ocultas dentro del recurso `manifest`.

8. Los directorios



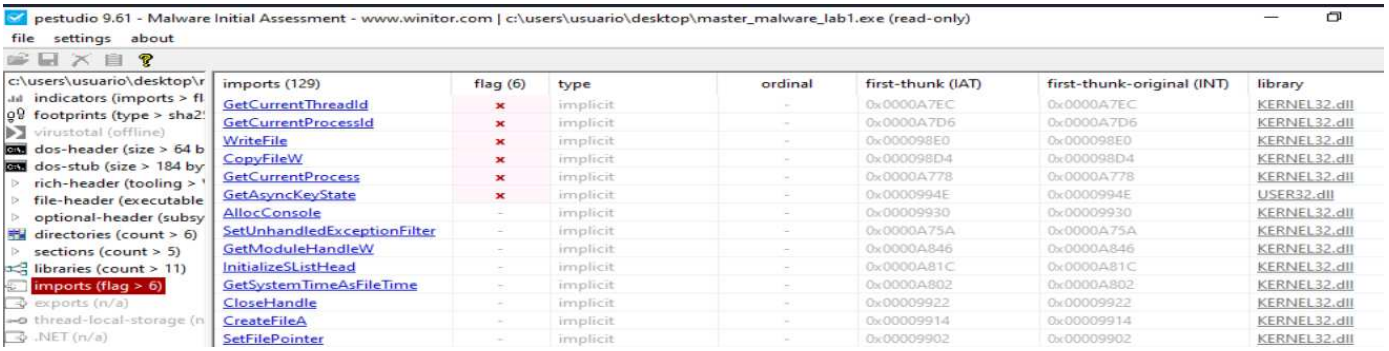
La tabla de directorios de PEStudio muestra qué directorios del `PE` están presentes y dónde se ubican:

Directorio	Tamaño	Ubicación	Interpretación
export	0x00000000	—	No exporta funciones. Normal para un .exe.
import	0x000000F0 / 240 bytes	.rdata	Contiene la tabla de imports: APIs como GetAsyncKeyState, WriteFile, CopyFileW, BitBlt, etc.
resource	0x000001E0 / 480 bytes	.rsrc	Contiene recursos, principalmente el manifiesto. No parece contener payload grande.
exception	0x00000000	—	Sin tabla de excepciones PE relevante. En x86 es habitual.
security	0x00000000	—	No hay certificado digital / Authenticode.
relocation	0x000007F0 / 2032 bytes	.reloc	Tabla de relocalización presente. Ya vimos que parece legítima.
debug	0x00000070 / 112 bytes	.rdata	Información de depuración presente, incluyendo ruta PDB.
architecture	0x00000000	—	No usado normalmente.
global-pointer	0x00000000	—	No aplica en este PE.
thread-local-storage	0x00000000	—	No usa TLS callbacks. Importante: no hay indicio de ejecución oculta vía TLS.
load-configuration	0x00000040 / 64 bytes	.rdata	Configuración de carga del binario, típica en compilaciones modernas MSVC.
bound-import	0x00000000	—	No usa bound imports.
import-address	0x00000230 / 560 bytes	.rdata	IAT presente; tabla usada para resolver llamadas a APIs importadas.
delay-loaded	0x00000000	—	No hay imports cargados en diferido.
.NET	0x00000000	—	No es binario .NET; es PE nativo.

Observaciones relevantes:

- La muestra no tiene exportaciones, lo cual confirma que se trata de un ejecutable normal, no una DLL ni un componente pensado para ser cargado por otro programa.
- El directorio de imports y la IAT están en .rdata, algo normal en binarios compilados con Visual Studio. Esto encaja con el análisis previo: la muestra importa directamente APIs de keylogging, escritura de archivos, captura de pantalla y persistencia.
- El directorio de resources está en .rsrc y es pequeño, de solo 480 bytes. Esto sugiere que no hay recursos embebidos grandes, como otro ejecutable, configuración cifrada o payload adicional.
- El directorio de debug está presente y apunta a .rdata.
- La ausencia de TLS es importante: no hay evidencia de callbacks TLS usados para ejecutar código antes del entry point, una técnica común en algunos malware para evasión o antianálisis.
- La ausencia de security confirma que el binario no está firmado digitalmente.

9. Los imports



PEStudio marca con una x varios imports porque los considera relevantes o potencialmente sospechosos:

API importada	Posible significado
`GetAsyncKeyState`	Captura o monitorización de pulsaciones de teclado
`WriteFile`	Escritura de datos en fichero
`CopyFileW`	Copia de ficheros
`GetCurrentProcess`	Obtención del proceso actual
`GetCurrentProcessId`	Identificación del proceso
`GetCurrentThreadId`	Identificación del hilo actual

Imports con mayor valor analítico:

Categoría	Imports	Conclusión
Keylogging	`GetAsyncKeyState`	Confirma captura de teclado por polling.
Escritura de datos	`CreateFileA`, `WriteFile`, `SetFilePointer`, `CloseHandle`	Compatible con escritura de `keylogs.txt` y generación de `screenshot.bmp`.
Instalación local	`CopyFileW`	Confirma copia del malware a una ruta persistente.
Ocultación básica	`AllocConsole` junto con imports vistos previamente como `FindWindowA`/`ShowWindow`	Compatible con ocultación de ventana de consola.
Runtime / soporte	`GetCurrentProcessId`, `GetCurrentThreadId`, `GetSystemTimeAsFileTime`, `InitializeSListHead`	Principalmente artefactos normales de MSVC/CRT.

La tabla de imports es coherente con el comportamiento ya identificado por `Tria.ge`: La muestra importa APIs necesarias para capturar pulsaciones (`GetAsyncKeyState`), escribir datos en disco (`CreateFileA, WriteFile, SetFilePointer`), copiarse a una ruta persistente (`CopyFileW`) y gestionar su ejecución como aplicación de consola. Estos imports respaldan la clasificación de la muestra como un keylogger con persistencia local y capacidad de captura de pantalla.

10. Los strings

```
strings -a MASTER_Malware_Lab1.exe > MASTER_Malware_Lab1-Strings.txt
```

donde:

- `-a`: Escanea todo el fichero.
- `-n 6`: Sólo muestra strings de 6 o más caracteres. Reduce mucho ruido.
- `-t x`: Muestra el offset en hexadecimal. Muy útil para saltar a esa zona en Ghidra, IDA, x64dbg o PE-bear

Los strings de la muestra

Analizando los strings parece que esta muestra, comprobamos que:

- Implementa un keylogging local.
- Hace captura de pantalla.
- Se copia a sí mismo.
- Persistencia por clave Run de usuario.

10.1 Hallazgos principales de los strings

Capacidad probable	Evidencia en strings
Keylogger	Ruta `C:\Users\Public\Public\keylogs.txt` y etiquetas de teclas como `**ENTER**`, `**SPACE**`, `**BACKSPACE**`, `**F1**-**F12**`, `**LEFT_SHIFT**`, `**RIGHT_CONTROL**`
Captura de pantalla	Rutas `C:\Users\Public\Public\Screens\screenshot`, `.bmp`, `screenshot.bmp`
Persistencia	Comando `REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run"`

Capacidad probable	Evidencia en strings
Copia o instalación local	Ruta `C:\Users\Public\Public\svchost.exe`
Ocultación de ventana de consola	String `ConsoleWindowClass` junto con imports `FindWindowA` y `ShowWindow`
Antianálisis básico	Import `IsDebuggerPresent`
Comunicación de red	No se observan URLs, dominios, IPs ni imports de red claros

La cadena `C:\Users\Public\Public\keylogs.txt`, seguida de una lista extensa de teclas especiales (`ENTER`, `SPACE`, `BACKSPACE`, flechas, `F1-F12`, `WIN`, `SHIFT`, `CONTROL`, etc.). Esto encaja directamente con un keylogger que traduce códigos de tecla a texto legible antes de escribirlos en un fichero.

También aparecen rutas para screenshots en `C:\Users\Public\Public\Screens` y un fichero `screenshot.bmp`, lo que sugiere funcionalidad de captura de pantalla. Esa hipótesis se evidencia con el uso de imports gráficos como `GetSystemMetrics`, `GetDC`, `CreateCompatibleDC`, `CreateCompatibleBitmap`, `BitBlt` y `GetDIBits`.

La persistencia parece implementarse mediante una clave Run de usuario:

- Aparece `REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V`.
- Junto con el nombre Firewall y la ruta `C:\Users\Public\Public\svchost.exe`.
- Esto sugiere que el malware podría copiarse como `svchost.exe` en una ruta pública y registrar una entrada de inicio automático llamada `Firewall`.

10.2 Imports visibles en la tabla de importación

Import	Interpretación
`GetAsyncKeyState`	Captura estado de teclas; API típica en keyloggers simples
`WriteFile` / `CreateFileA` / `SetFilePointer`	Escritura o modificación de ficheros, probablemente `keylogs.txt`
`CopyFileW` / `GetModuleFileNameW`	Posible copia del ejecutable actual a otra ruta
`_mkdir`	Creación de directorios como `C:\Users\Public\Public\Screens`
`system`	Ejecución de comandos del sistema, probablemente para el `REG ADD`
`FindWindowA` / `ShowWindow`	Localización y ocultación de ventana
`BitBlt` / `GetDIBits`	Captura de pantalla
`IsDebuggerPresent`	Posible chequeo básico de depurador

Las APIs importadas coinciden muy bien con las cadenas encontradas: `GetAsyncKeyState` para capturar teclas, `WriteFile` para registrar datos, `CopyFileW` para copiar el binario, y las APIs de GDI para capturar pantalla.

Nota: `IsDebuggerPresent` parece un indicador débil, porque puede aparecer por `runtime/CRT` o por lógica antidebug real. Para confirmarlo hay que ver sus `XREFs` en Ghidra/IDA.

10.3 Búsqueda en los strings de IoCs

```
└─$ strings -a -n 4 -t x MASTER_Malware_Lab1.exe | grep -Ei "hkcu|run|svchost|keylog|screen|http"
     4d !This program cannot be run in DOS mode.
   6b00 C:\Users\Public\Public\keylogs.txt
   6e04 C:\Users\Public\Public\Screens\screenshot
   6e50 C:\Users\Public\Public\Screens
   6eb8 C:\Users\Public\Public\Screens\screenshot.bmp
   6ee8 C:\Users\Public\Public\svchost.exe
   6fa0 REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V
   8b86 VCRUNTIME140.dll
   8e94 api-ms-win-crt-runtime-l1-1-0.dll
```

Offset	String	Interpretación
0x004d	This program cannot be run in DOS mode.	DOS stub normal de ejecutables PE. No es malicioso por sí mismo.
0x6b00	C:\Users\Public\Public\keylogs.txt	Fichero donde probablemente almacena las pulsaciones capturadas. Indicador fuerte de keylogger.
0x6e04	C:\Users\Public\Public\Screens\screenshot	Prefijo o ruta base para capturas de pantalla.
0x6e50	C:\Users\Public\Public\Screens	Directorio donde almacena screenshots.
0x6eb8	C:\Users\Public\Public\Screens\screenshot.bmp	Ruta completa de una captura BMP. Indicador fuerte de screenshotting.
0x6ee8	C:\Users\Public\Public\svchost.exe	Posible copia del malware con nombre legítimo para camuflaje.
0x6fa0	REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V	Persistencia en el inicio del usuario actual mediante clave Run.
0x8b86	VCRUNTIME140.dll	Runtime normal de Visual C++. No es IoC malicioso.
0x8e94	api-ms-win-crt-runtime-l1-1-0.dll	Runtime normal de Windows/Visual C++. No es IoC malicioso.

10.4 Conclusión del análisis de strings

El análisis de strings revela múltiples indicadores funcionales asociados a comportamiento de keylogger. La muestra contiene una ruta explícita para el almacenamiento de pulsaciones en `C:\Users\Public\Public\keylogs.txt`, junto con etiquetas de teclas especiales como `ENTER`, `SPACE`, `BACKSPACE`, teclas de función, modificadores y flechas.

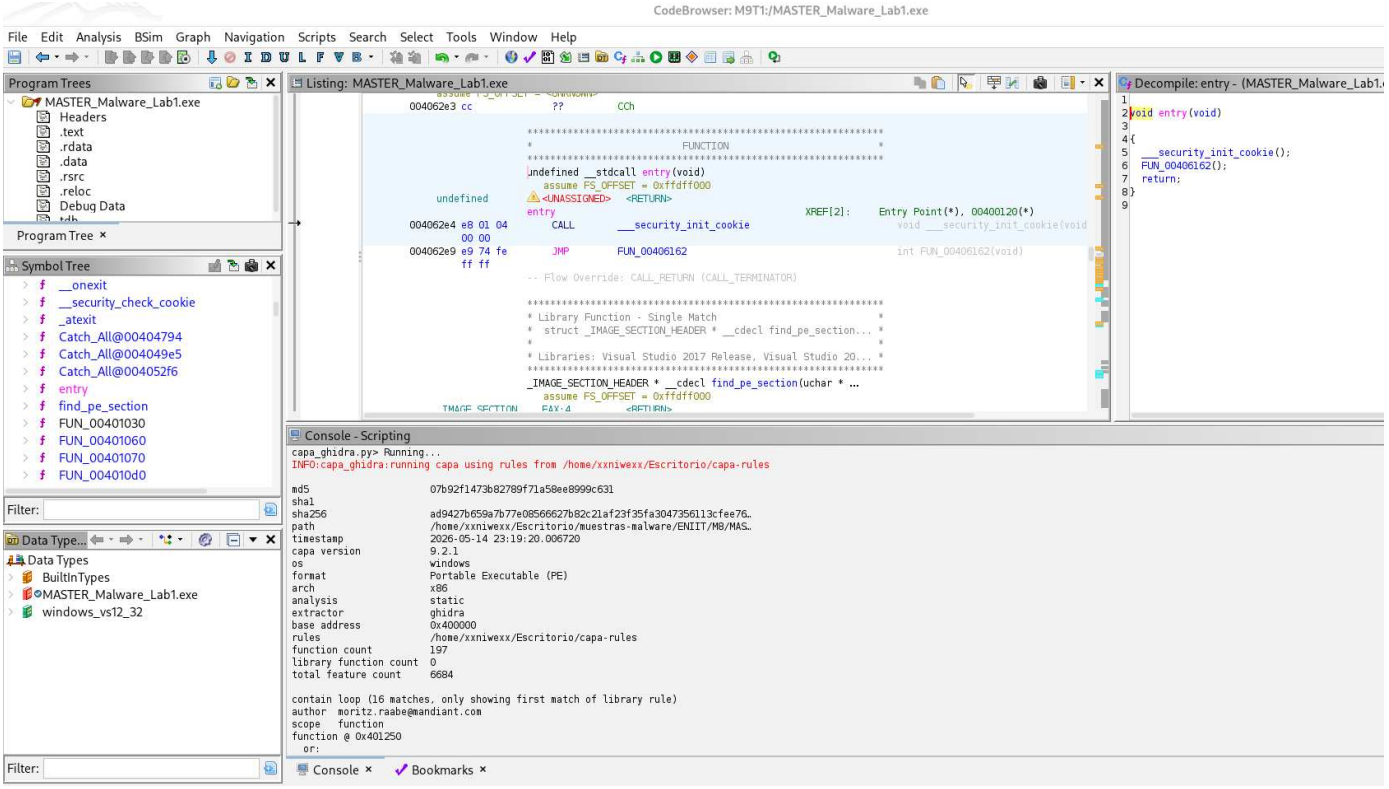
Además, se identifican rutas relacionadas con capturas de pantalla en formato BMP, así como una posible rutina de persistencia mediante `REG ADD` sobre `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`, usando el nombre `Firewall` y una copia del binario como `svchost.exe`. Estos hallazgos, combinados con imports como `GetAsyncKeyState`, `WriteFile`, `CopyFileW`, `BitBlt` y `GetDIBits`, permiten inferir capacidades de keylogging, screenshotting y persistencia de usuario.

---

11. Aplicamos Capa Rules

Nota: **Instalación del plugin Capa en Ghidra**

En Ghidra aplicamos las reglas Capa usando el plugin anteriormente instalado:




```
capa_ghidra.py> Running...
INFO:capa_ghidra:running capa using rules from /home/xxniwexx/Escritorio/capa-rules

md5                     07b92f1473b82789f71a58ee8999c631
sha1
sha256                  ad9427b659a7b77e08566627b82c21af23f35fa3047356113cfee76…
path                    /home/xxniwexx/Escritorio/muestras-malware/ENIIT/M8/MAS…
timestamp               2026-05-14 23:19:20.006720
capa version            9.2.1
os                      windows
format                  Portable Executable (PE)
arch                    x86
analysis                static
extractor               ghidra
base address            0x400000
rules                   /home/xxniwexx/Escritorio/capa-rules
function count          197
library function count  0
total feature count     6684

contain loop (16 matches, only showing first match of library rule)
author  moritz.raabe@mandiant.com
scope   function
function @ 0x401250
  or:
    characteristic: loop @ 0x401250

create or open file (library rule)
author  michael.hunhoff@mandiant.com, joakim@intezer.com
scope   instruction
mbc     File System::Create File [C0016]
instruction @ 0x40594E
  or:
    api: CreateFile @ 0x40594E
```

```
delay execution (3 matches, only showing first match of library rule)
author      michael.hunhoff@mandiant.com, @ramen0x3f
scope       basic block
mbc         Anti-Behavioral Analysis::Dynamic Analysis Evasion::Delayed
            Execution [B0003.003]
references  https://docs.microsoft.com/en-us/windows/win32/sync/wait-functions,
            https://github.com/LordNoteworthy/al-khaser/blob/master/al-khaser/T…
basic block @ 0x402B20 in function 0x402A00
  or:
    and:
      os: windows
      or:
        api: Sleep @ 0x402B22

log keystrokes via polling
namespace  collection/keylog
author     michael.hunhoff@mandiant.com
scope      function
att&ck     Collection::Input Capture::Keylogging [T1056.001]
mbc        Collection::Keylogging::Polling [F0002.002]
function @ 0x402A00
  or:
    api: GetAsyncKeyState @ 0x402B31

capture screenshot
namespace  collection/screenshot
author     moritz.raabe@mandiant.com, @_re_fox, michael.hunhoff@mandiant.com
scope      function
att&ck     Collection::Screen Capture [T1113]
mbc        Collection::Screen Capture::WinAPI [E1113.m01]
function @ 0x405AD0
  or:
    and:
      or:
        api: GetDC @ 0x405ADA
      or:
        api: BitBlt @ 0x405B2A
      api: CreateCompatibleDC @ 0x405AE6
      api: CreateCompatibleBitmap @ 0x405AFC

contains PDB path
namespace  executable/pe/pdb
author     moritz.raabe@mandiant.com
scope      file
regex: /:\\.*\.pdb/
  - "C:\\Users\\Admin\\Desktop\\MALWARE
Labs\\MASTER_Malware_Lab1\\Release\\MASTER_Malware_Lab1.pdb" @ file+0x408D1C

copy file
namespace  host-interaction/file-system/copy
author     moritz.raabe@mandiant.com, michael.hunhoff@mandiant.com
scope      function
mbc        File System::Copy File [C0045]
function @ 0x402FA0
  or:
    api: CopyFile @ 0x40323A

create directory
namespace  host-interaction/file-system/create
author     moritz.raabe@mandiant.com, michael.hunhoff@mandiant.com
scope      function
mbc        File System::Create Directory [C0046]
function @ 0x402FA0
  or:
    api: _mkdir @ 0x402FEF, 0x403009

read file on Windows
namespace  host-interaction/file-system/read
author     moritz.raabe@mandiant.com, anushka.virgaonkar@mandiant.com
```

```
scope      function
mbc        File System::Read File [C0051]
function @ 0x4039D0
  or:
    and:
      os: windows
      or:
        api: fread @ 0x403AAB, 0x403ADC

write file on Windows (4 matches)
namespace  host-interaction/file-system/write
author     william.ballenthin@mandiant.com, anushka.virgaonkar@mandiant.com
scope      function
mbc        File System::Writes File [C0052]
function @ 0x4038E0
  or:
    and:
      os: windows
      or:
        api: fwrite @ 0x4039A7
function @ 0x403E80
  or:
    and:
      os: windows
      or:
        api: fwrite @ 0x403F87
function @ 0x404400
  or:
    and:
      os: windows
      optional:
        basic block:
          or:
            number: 0x2 = FILE_WRITE_DATA @ 0x404457
      or:
        api: fwrite @ 0x404486
function @ 0x405850
  or:
    and:
      os: windows
      optional:
        basic block:
          or:
            number: 0x40000000 = GENERIC_WRITE @ 0x405946
            number: 0x2 = FILE_WRITE_DATA @ 0x405940
            match: create or open file @ 0x40594E
              or:
                api: CreateFile @ 0x40594E
      or:
        api: WriteFile @ 0x405979, 0x40599B, 0x4059F7, 0x405A2E, and 1 more...

find graphical window
namespace  host-interaction/gui/window/find
author     moritz.raabe@mandiant.com
scope      instruction
att&ck     Discovery::Application Window Discovery [T1010]
instruction @ 0x4032BD
  or:
    api: FindWindow @ 0x4032BD

hide graphical window
namespace  host-interaction/gui/window/hide
author     michael.hunhoff@mandiant.com
scope      basic block
att&ck     Defense Evasion::Hide Artifacts::Hidden Window [T1564.003]
basic block @ 0x4032B0 in function 0x402FA0
  and:
    number: 0x0 = SW_HIDE @ 0x4032B6, 0x4032C3, 0x4032DE, 0x4032EC, and 5 more...
    api: ShowWindow @ 0x4032C6
```

```
execute command
namespace  host-interaction/process/create
author     @mr-tz
scope      function
mbc        Process::Create Process [C0017]
function @ 0x405580
  or:
    api: system @ 0x405760

terminate process (3 matches)
namespace  host-interaction/process/terminate
author     moritz.raabe@mandiant.com, mehunhoff@google.com,
           anushka.virgaonkar@mandiant.com
scope      function
mbc        Process::Terminate Process [C0018]
function @ 0x406162
  or:
    api: exit @ 0x4062D6
function @ 0x406542
  or:
    and:
      or:
        api: TerminateProcess @ 0x406562
function @ 0x406CAF
  or:
    api: exit @ 0x406CAF

set registry value
namespace  host-interaction/registry/create
author     moritz.raabe@mandiant.com, michael.hunhoff@mandiant.com
scope      function
mbc        Operating System::Registry::Set Registry Key [C0036.001]
function @ 0x405580
  or:
    and:
      match: host-interaction/process/create @ 0x405580
        or:
          api: system @ 0x405760
      regex: /add/i
        - "REG ADD \"HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\" /V " @ 0x4055D3
      or:
        regex: /reg(|.exe)/i
          - " /t REG_SZ /F /D " @ 0x4055F4
          - "REG ADD \"HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\" /V " @ 0x4055D3
        regex: /hkcu/i
          - "REG ADD \"HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\" /V " @ 0x4055D3

create thread (2 matches)
namespace  host-interaction/thread/create
author     moritz.raabe@mandiant.com, michael.hunhoff@mandiant.com,
           joakim@intezer.com, anushka.virgaonkar@mandiant.com
scope      basic block
mbc        Process::Create Thread [C0038]
basic block @ 0x4032B0 in function 0x402FA0
  or:
    and:
      os: windows
      or:
        api: _beginthreadex @ 0x40335D
basic block @ 0x403370 in function 0x402FA0
  or:
    and:
      os: windows
      or:
        api: _beginthreadex @ 0x40339E

parse PE header (2 matches)
namespace  load-code/pe
author     moritz.raabe@mandiant.com
scope      function
```

```
att&ck     Execution::Shared Modules [T1129]
function @ 0x406424
  and:
    os: windows
    and:
      mnemonic: cmp @ 0x406439, 0x406447, 0x406458, 0x406478
      or:
        number: 0x4550 = IMAGE_NT_SIGNATURE (PE) @ 0x406447
      or:
        number: 0x5A4D = IMAGE_DOS_SIGNATURE (MZ) @ 0x406434
function @ 0x4068D2
  and:
    os: windows
    and:
      mnemonic: cmp @ 0x4068E3, 0x4068ED, 0x4068FA, 0x406900, and 1 more...
      or:
        number: 0x4550 = IMAGE_NT_SIGNATURE (PE) @ 0x4068ED
      or:
        number: 0x5A4D = IMAGE_DOS_SIGNATURE (MZ) @ 0x4068DE

persist via Run registry key
namespace  persistence/registry/run
author     moritz.raabe@mandiant.com, mehunhoff@google.com
scope      function
att&ck     Persistence::Boot or Logon Autostart Execution::Registry Run Keys /
           Startup Folder [T1547.001]
mbc        Persistence::Registry Run Keys / Startup Folder [F0012]
function @ 0x405580
  and:
    or:
      match: set registry value @ 0x405580
        or:
          and:
            match: host-interaction/process/create @ 0x405580
              or:
                api: system @ 0x405760
            regex: /add/i
              - "REG ADD \"HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\" /V " @ 0x4055D3
            or:
              regex: /reg(|.exe)/i
                - " /t REG_SZ /F /D " @ 0x4055F4
                - "REG ADD \"HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\" /V " @
0x4055D3
              regex: /hkcu/i
                - "REG ADD \"HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\" /V " @
0x4055D3
    or:
      regex: /Software\\Microsoft\\Windows\\CurrentVersion\\Run/i
        - "REG ADD \"HKCU\\SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run\" /V " @ 0x4055D3



Script /home/xxniwexx/ghidra_scripts/capa_ghidra.py called exit with code 0
capa_ghidra.py> Finished!
```

11.1 Capacidades destacadas identificadas

Capacidad detectada	Evidencia capa	Confianza
Keylogging por polling	`log keystrokes via polling` en `0x402A00`, uso de `GetAsyncKeyState`	Alta
Captura de pantalla	`capture screenshot` en `0x405AD0`, uso de `GetDC`, `BitBlt`, `CreateCompatibleDC`, `CreateCompatibleBitmap`	Alta
Escritura de ficheros	`write file on Windows`, múltiples funciones con `fwrite` y `WriteFile`	Alta

Capacidad detectada	Evidencia capa	Confianza
Creación/apertura de ficheros	`CreateFile` en `0x40594E`	Alta
Copia del ejecutable	`copy file` en `0x402FA0`, uso de `CopyFile`	Alta
Creación de directorios	`_mkdir` en `0x402FA0`	Alta
Persistencia	`persist via Run registry key` en `0x405580`	Alta
Ejecución de comandos	`system` en `0x405580`	Alta
Ocultación de ventana	`ShowWindow` con `SW_HIDE` en `0x402FA0`	Alta
Creación de hilos	`_beginthreadex` en `0x402FA0`	Media-alta
Antianálisis por delay	`Sleep` en `0x402A00`	Baja-media; probablemente usado para el bucle de keylogging
Antidebug	No aparece como regla capa destacada, aunque antes vimos `IsDebuggerPresent` como import	Pendiente de confirmar

11.2 Funciones relevantes identificadas

Dirección	Capacidad	Interpretación
`0x402A00`	Keylogging / loop / Sleep	Probable bucle principal de captura de teclado mediante `GetAsyncKeyState`
`0x402FA0`	Copia, creación de directorios, ocultación de ventana, creación de hilos	Probable función de inicialización/instalación
`0x4032B0`	Ocultación de ventana	Usa `ShowWindow(..., SW_HIDE)`
`0x4032BD`	Búsqueda de ventana	Usa `FindWindow`, probablemente para localizar la consola
`0x4038E0`	Escritura de fichero	Usa `fwrite`
`0x4039D0`	Lectura de fichero	Usa `fread`
`0x403E80`	Escritura de fichero	Usa `fwrite`
`0x404400`	Escritura de fichero	Usa `fwrite`
`0x405580`	Persistencia / comando / registro	Construye o ejecuta `REG ADD HKCU...\Run` mediante `system`
`0x405850`	Escritura con WinAPI	Usa `CreateFile` y varias llamadas a `WriteFile`
`0x405AD0`	Captura de pantalla	Usa APIs GDI para capturar pantalla
`0x406424`, `0x4068D2`	Parseo PE	Posible código de runtime/librería; no necesariamente malicioso
`0x406162`, `0x406542`, `0x406CAF`	Terminación de proceso	Probablemente parte de CRT/runtime o manejo de errores

11.3 Interpretación de comportamiento

La muestra parece seguir un flujo similar a este:

Fase	Comportamiento probable	Evidencia
1	Inicialización del binario	Entry point llama a `__security_init_cookie` y luego a la lógica principal
2	Ocultación de ventana	`FindWindow` + `ShowWindow(SW_HIDE)`
3	Creación de estructura en disco	`_mkdir`, rutas `C:\Users\Public\Public` y `Screens`
4	Copia de la muestra	`CopyFile` hacia `C:\Users\Public\Public\svchost.exe`

Fase	Comportamiento probable	Evidencia
5	Persistencia	`REG ADD HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`
6	Creación de hilos	`_beginthreadex`, probablemente para ejecutar keylogger y screenshotter en paralelo
7	Captura de teclas	`GetAsyncKeyState` dentro de un bucle
8	Escritura de logs	`fwrite`, `WriteFile`, `CreateFile`
9	Captura de pantalla	`GetDC`, `CreateCompatibleDC`, `CreateCompatibleBitmap`, `BitBlt`, `GetDIBits`
10	Guardado de evidencia robada	`keylogs.txt` y `screenshot.bmp`

11.4 Correspondencia con MITRE ATT&CK

Técnica	ID	Evidencia
Keylogging	`T1056.001`	`GetAsyncKeyState` en `0x402A00`
Screen Capture	`T1113`	APIs GDI en `0x405AD0`
Registry Run Keys / Startup Folder	`T1547.001`	`HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`
Hidden Window	`T1564.003`	`ShowWindow` con `SW_HIDE`
Application Window Discovery	`T1010`	`FindWindow`

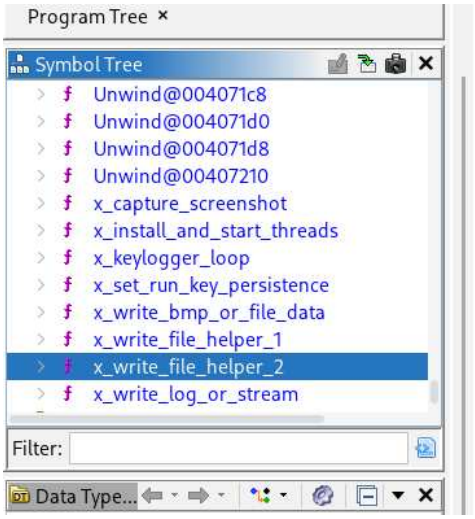
11.5 Matices importantes

El hallazgo `delay execution` no debe interpretarse automáticamente como evasión avanzada. En esta muestra, como aparece dentro de la función `0x402A00` junto al `keylogging`, es muy probable que `Sleep` se use para espaciar el `polling` de teclas y evitar consumo excesivo de CPU. Puede ser compatible con evasión, pero aquí parece más funcional que antianálisis.

La persistencia sí es muy sólida: capa detecta `set registry value` y `persist via Run registry key` en la misma función `0x405580`, con la cadena `REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V` y ejecución vía `system`.

11.6 Funciones que vamos a renombrar en Ghidra

Dirección	Nombre sugerido
`0x402A00`	`x_keylogger_loop`
`0x402FA0`	`x_install_and_start_threads`
`0x405580`	`x_set_run_key_persistence`
`0x405850`	`x_write_bmp_or_file_data`
`0x405AD0`	`x_capture_screenshot`
`0x4038E0`	`x_write_log_or_stream`
`0x4039D0`	`x_read_file_helper`
`0x403E80`	`x_write_file_helper_1`
`0x404400`	`x_write_file_helper_2`





11.7 Orden de las funciones que vamos a revisar en Ghidra

Prioridad	Función	Qué confirmar
1	`0x402FA0`	Si copia el binario, crea directorios, oculta ventana y lanza hilos
2	`0x405580`	Comando exacto de persistencia y nombre de la clave Run
3	`0x402A00`	Bucle de `GetAsyncKeyState`, mapeo de teclas y escritura en `keylogs.txt`
4	`0x405AD0`	Captura de pantalla y ruta de salida
5	`0x405850`	Estructura del BMP o escritura del screenshot
6	XREFs a `C:\Users\Public\Public\svchost.exe`	Confirmar si se copia ahí con `CopyFile`
7	XREFs a `Firewall`	Confirmar si ese es el nombre de la entrada de persistencia

11.8 Conclusión de las reglas Capa

El análisis estático mediante capa integrado en Ghidra identifica capacidades consistentes con un `keylogger`. La función `0x402A00` contiene lógica de captura de pulsaciones mediante `GetAsyncKeyState`, asociada a la técnica `MITRE ATT&CK T1056.001`. Además, la función `0x405AD0` implementa captura de pantalla mediante `APIs GDI` como `GetDC`, `CreateCompatibleDC`, `CreateCompatibleBitmap` y `BitBlt`, correspondiente a `T1113`. La muestra también presenta persistencia mediante clave Run de usuario en `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`, ejecutada a través de system en la función `0x405580`, lo que corresponde a `T1547.001`. Asimismo, se observan capacidades de copia de fichero, creación de directorios, ocultación de ventana y creación de hilos. En conjunto, los resultados de capa corroboran los indicadores obtenidos previamente con `strings` y permiten clasificar la muestra como un `keylogger` con captura de pantalla y persistencia local básica.

---

12. Funciones

12.1 Función `0x402FA0` | `x_install_and_start_threads`

Analizamos ahora la Función 0x402FA0.

Creación de directorios

Al principio carga `_access` y `_mkdir`. Primero comprueba si existe:

```
C:\Users\Public\Public
```

donde:

- Llama a `_access`, se comprueba `EAX`, y si no existe crea el directorio con `_mkdir`.

Después repite, y comprueba si existe:

```
C:\Users\Public\Public\Screens
```

donde:

- Llama a `_access`, se comprueba `EAX`, y si no existe crea el directorio con `_mkdir`.

Directorios:

Ruta	Uso probable
`C:\Users\Public\Public`	Directorio base de instalación
`C:\Users\Public\Public\Screens`	Directorio para capturas de pantalla

Obtiene la ruta del ejecutable actual

Luego llama a:

```
GetModuleFileNameW(NULL, local_21c, 0x104)
```

donde:

- Esto obtiene la ruta completa del ejecutable que se está ejecutando. Como el primer argumento es `NULL`, Windows devuelve la ruta del proceso actual.

Copia el malware como svchost.exe

```
PUSH 0x1
PUSH u"C:\Users\Public\Public\svchost.exe"
PUSH EAX
CALL CopyFileW
```

`CopyFileW` tiene esta firma:

```
BOOL CopyFileW(
  LPCWSTR lpExistingFileName,
  LPCWSTR lpNewFileName,
  BOOL bFailIfExists
);
```

donde:

- Como en `x86` los argumentos se apilan en orden inverso, eso significa:

```
CopyFileW(current_path, L"C:\\Users\\Public\\Public\\svchost.exe", TRUE);
```

Interpretación:

Argumento	Valor
Origen	Ruta del ejecutable actual
Destino	`C:\Users\Public\Public\svchost.exe`
`bFailIfExists`	`TRUE`

El detalle de `TRUE` es muy relevante: si svchost.exe ya existe en esa ruta, no lo sobrescribe. Eso encaja con una instalación de primera ejecución.

Prepara la ruta del screenshot

Después construye la cadena:

```
C:\Users\Public\Public\Screens\screenshot.bmp
```

Hay llamadas a funciones auxiliares C++ como `FUN_004044D0` y `FUN_00406071`. Por el contexto, parecen construir/copiar un `std::string` o buffer dinámico con la ruta del screenshot.

Oculta la consola

```
CALL AllocConsole
PUSH 0x0
PUSH "ConsoleWindowClass"
CALL FindWindowA
PUSH 0x0
PUSH EAX
CALL ShowWindow
```



donde:

- `SW_HIDE` vale `0`, así que la intención es ocultar la ventana de consola.
- La muestra es un ejecutable de consola, pero quiere ejecutarse sin mostrar una ventana visible al usuario.

Configura persistencia

Luego construye dos strings relevantes:

```
C:\Users\Public\Public\svchost.exe
Firewall
```

Y llama a:

```
CALL set_run_key_persistence
```

Por los strings que ya habíamos visto, esta función construye un comando para generar persistencia en el sistema. Pudiera ser algo parecido a:

```
REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D
C:\Users\Public\Public\svchost.exe
```

Elemento	Valor
Clave	`HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`
Nombre del valor	`Firewall`
Datos del valor	`C:\Users\Public\Public\svchost.exe`
Alcance	Usuario actual
Privilegios admin	No necesarios normalmente

Esto establece persistencia para que el malware se ejecute al iniciar sesión el usuario. La construcción de `svchost.exe`, `Firewall` y la llamada a `set_run_key_persistence` aparecen juntas antes de lanzar los hilos.

Captura screenshot inicial

Después aparece:

```
CALL capture_screenshot
```

donde:

- Esta llamada ocurre antes de crear los hilos.
- Por eso parece que la muestra toma una captura inicial durante la fase de instalación.

Tendremos que investigar si la ruta preparada anteriormente corresponde a la salida del screenshot::

```
capture_screenshot(screenshot_path);
```

Creación del hilo del keylogger

Este bloque es clave:

```
PUSH 0x4
CALL operator_new
MOV [EAX], keylogger_loop
...
PUSH FUN_00405380
...
CALL _beginthreadex
```

donde:

- La muestra reserva 4 bytes, guarda ahí un puntero a función, concretamente:

  ```
  MOV dword ptr [EAX], keylogger_loop
  ```

- Luego pasa esa estructura a `_beginthreadex` mediante un wrapper, `FUN_00405380`.

- `FUN_00405380` probablemente es un wrapper generado por C++ para invocar la función guardada dentro del objeto.

Crea un segundo hilo

Luego repite el patrón:

```
CALL operator_new
MOV [EAX], FUN_00402C40
...
CALL _beginthreadex
```

donde:

- Este segundo hilo ejecuta FUN_00402C40.

Resumen:

Hilo	Función lanzada	Hipótesis
Hilo 1	x_keylogger_loop / 0x402A00	Keylogging continuo
Hilo 2	FUN_00402C40	Rutina auxiliar pendiente de analizar

Espera a que los hilos terminen

Después de crear ambos hilos, llama a funciones de `MSVCP`:

```
_Thrd_id
_Thrd_join
```

donde:

- La función principal queda bloqueada esperando a los hilos. Como `keylogger_loop` contiene un bucle infinito, lo normal es que `install_and_start_threads` nunca llegue a su limpieza final ni al `RET`, salvo que el hilo falle, termine por excepción o el proceso sea cerrado.

Volvemos a renombrar la función

```
x_setup_persistence_start_keylogger
```

Conclusiones

La función `0x402FA0` es la rutina de instalación e inicialización principal del malware. Hace estas tareas:

- Crea directorios de trabajo.
- Obtiene la ruta del ejecutable actual.
- Se copia como `svchost.exe`.
- Prepara una ruta para screenshots.
- Oculta la ventana de consola.
- Configura persistencia en `HKCU\...\Run`.
- Toma una captura de pantalla inicial.
- Lanza dos hilos: el keylogger y otra rutina auxiliar.
- Espera a que esos hilos terminen.

Resumen funcional:

Bloque	Dirección aproximada	Acción
Preparación C++ / SEH	`0x402FA0 - 0x402FCE`	Prólogo, cookie de seguridad, manejo de excepciones C++
Crear directorios	`0x402FCE - 0x40300E`	Crea `C:\Users\Public\Public` y `C:\Users\Public\Public\Screens` si no existen
Obtener ruta propia	`0x40300E - 0x40306D`	Usa `GetModuleFileNameW(NULL, buffer, 0x104)`
Preparar string de ruta actual	`0x40306D - 0x40321E`	Convierte/manipula la ruta del ejecutable actual
Copiar malware	`0x40321E - 0x40323A`	Copia el binario a `C:\Users\Public\Public\svchost.exe`
Preparar ruta screenshot	`0x403240 - 0x4032AE`	Construye/copia `C:\Users\Public\Public\Screens\screenshot.bmp`
Ocultar consola	`0x4032B0 - 0x4032C6`	`AllocConsole`, `FindWindowA`, `ShowWindow(SW_HIDE)`
Preparar persistencia	`0x4032CC - 0x40331E`	Crea strings `svchost.exe` y `Firewall`, llama a `set_run_key_persistence`
Screenshot inicial	`0x403323 - 0x403328`	Llama a `capture_screenshot`
Lanzar hilo keylogger	`0x40332D - 0x40336A`	Crea hilo para `keylogger_loop`
Lanzar segundo hilo	`0x403370 - 0x4033AB`	Crea hilo para `FUN_00402C40`
Esperar hilos	`0x4033B1 - 0x40342D`	Hace `join` de los hilos
Limpieza	`0x403435 - 0x403545`	Libera memoria, destruye strings, restaura SEH y retorna

12.2 Función 0x405580 - x_set_run_key_persistence

Analizamos ahora la función 405580, que es la rutina que construye y ejecuta el comando de persistencia mediante `REG ADD`.

En términos funcionales hace:

```
REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D
C:\Users\Public\Public\svchost.exe
```

Es decir, crea una entrada de inicio automático en el registro del usuario actual.

Resumen de la función 405580

Elemento	Valor
Función	`0x405580 - x_set_run_key_persistence`
Propósito	Crear persistencia mediante clave `Run`
Técnica	Ejecuta comando `REG ADD` usando `system()`
Clave modificada	`HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`
Nombre del valor	`Firewall`
Datos del valor	`C:\Users\Public\Public\svchost.exe`
Privilegios requeridos	No requiere administrador, porque usa `HKCU`
MITRE ATT&CK	`T1547.001 - Registry Run Keys / Startup Folder`

Qué recibe como parámetros

Desde `0x402FA0`, esta función recibe dos strings construidos justo antes:

Parámetro	Valor probable	Uso
Primer `std::string`	`Firewall`	Nombre del valor en la clave `Run`
Segundo `std::string`	`C:\Users\Public\Public\svchost.exe`	Ruta del ejecutable persistente

El `RET 0x30` del final indica que la función limpia `0x30 bytes` de argumentos, es decir, `48 bytes`. Esto encaja con dos objetos `std::string` de `MSVC` de `0x18 bytes` cada uno.

```
00405841  RET 0x30
```

Por eso la función no recibe simples punteros `char *`, sino probablemente dos `std::string` pasados por valor.

Construye el inicio del comando

La cadena fija más importante aparece aquí:

```
004055d3  PUSH s_REG_ADD_"HKCU\SOFTWARE\Microsoft...
```

Corresponde a::

```
REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V
```

Después la función concatena el primer parámetro, que por contexto es:

```
Firewall
```

Por tanto, en esta fase construye algo equivalente a:

```
REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall
```

La comprobación inicial con `0x7fffffff`, la longitud del string y `0x40` sirve para evitar overflow o una longitud inválida al concatenar la cadena fija de `REG ADD`.

Añade el tipo y el dato del registro

Luego aparece esta cadena:

```
004055f4  PUSH s_/t_REG_SZ_/F_/D_00408788
```

Que corresponde a:

```
 /t REG_SZ /F /D
```

Significado:

Fragmento	Significado
`/t REG_SZ`	Crea un valor de tipo string
`/F`	Fuerza la operación sin pedir confirmación
`/D`	Especifica los datos del valor

Después concatena el segundo parámetro, que por contexto es:

```
C:\Users\Public\Public\svchost.exe
```

Así se obtiene el comando completo:

```
REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D
C:\Users\Public\Public\svchost.exe
```

Ejecuta el comando con system

La ejecución ocurre en este bloque:

```
00405754  CMP ...
00405758  LEA EAX, [local_2c]
0040575b  CMOVNC EAX, [local_2c]
0040575f  PUSH EAX
00405760  CALL dword ptr [API-MS-WIN-CRT-RUNTIME...]
```

Por el resultado de capa y por el contexto, esa llamada corresponde a:

```
system(command.c_str());
```

Es decir, la muestra no usa directamente APIs de registro como `RegOpenKeyExA` o `RegSetValueExA`. En su lugar, construye un comando de consola y lo ejecuta con `system()`. Esto es más simple, pero también más ruidoso desde el punto de vista forense, porque puede generar ejecución de `cmd.exe` y `reg.exe`.

Limpieza de objetos std::string

Después de ejecutar `system`, la función libera o destruye varios objetos `std::string` temporales:

```
00405793  CALL FUN_00406063
...
004057d7  CALL FUN_00406063
...
0040581f  CALL FUN_00406063
```

También reinicializa campos internos de strings, por ejemplo:

```
MOV dword ptr [...], 0x0
MOV dword ptr [...], 0xf
MOV byte ptr [...], 0x0
```

Esto es típico de `std::string` en `MSVC`:

Campo	Interpretación probable
`0x0`	longitud del string
`0xf`	capacidad SSO, small string optimization
primer byte `0x00`	string vacío

No es comportamiento malicioso por sí mismo; es limpieza normal de objetos C++.

Esta función confirma persistencia clara

Indicador	Tipo
`HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`	Clave de persistencia
`Firewall`	Nombre del valor de registro
`C:\Users\Public\Public\svchost.exe`	Ejecutable persistente
`REG ADD`	Técnica usada para escribir en registro
`system()`	Ejecución de comando externo

Conclusiones

La función `0x405580`, renombrada como `x_set_run_key_persistence`, implementa la persistencia de la muestra mediante la clave `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`. La función recibe dos objetos `std::string`, correspondientes al nombre del valor de registro y a la ruta del ejecutable persistente. Construye dinámicamente un comando `REG ADD`, añade el nombre `Firewall`, especifica el tipo `REG_SZ` y establece como dato la ruta `C:\Users\Public\Public\svchost.exe`. Finalmente ejecuta el comando mediante `system()`. Esta técnica permite que el malware se ejecute automáticamente al inicio de sesión del usuario actual, sin requerir privilegios de administrador.

12.3 Función 0x402A00 - x_keylogger_loop

Analizamos ahora la Función 402A00.

Prólogo de la función

Vemos el prólogo de la función que prepara el entorno de ejecución de la función y reserva espacio para sus variables locales.

```
00402a00  PUSH EBP
00402a01  MOV  EBP, ESP
...
00402a11  SUB  ESP, 0xb4
...
00402a21  PUSH EBX
00402a22  PUSH ESI
00402a23  PUSH EDI
```

donde:

- `PUSH EBP`: Guarda el valor anterior de `EBP`, es decir, el marco de pila de la función llamadora.
- `MOV EBP, ESP`: Crea un nuevo marco de pila para esta función. A partir de aquí, Ghidra puede referenciar variables como `[EBP - offset]`.
- `SUB ESP, 0xB4`: Reserva `0xB4` bytes en la pila para variables locales.
- `PUSH EBX`: Guarda `EBX` porque la función lo va a usar y debe restaurarlo antes de salir.
- `PUSH ESI`: Guarda `ESI` por el mismo motivo.
- `PUSH EDI`: Guarda `EDI` por el mismo motivo.

Inicialización del buffer/stream

Después reserva e inicializa una estructura local de `0xB0` bytes con `memset`, probablemente relacionada con un objeto C++ de salida, como un stream o helper de escritura:

```
00402a2e  PUSH 0xb0
00402a33  LEA  EAX, [EBP + local_c4]
00402a39  PUSH 0x0
00402a3b  PUSH EAX
00402a3c  CALL memset
```

Luego llama a funciones auxiliares relacionadas con `basic_ostream` / `basic_ios`, lo que encaja con escritura mediante `C++ streams`.

Entrada al bucle de keylogging

La función carga en `EDI` la dirección importada de `GetAsyncKeyState`:

```
00402b15  MOV EDI, [USER32.dll::GetAsyncKeyState]
```

Después entra en el bucle externo:

```
LAB_00402b20:
00402b20  PUSH 0xa
00402b22  CALL Sleep
00402b28  XOR EBX, EBX
```

Esto significa que cada iteración empieza con una pausa de 10 ms y reinicia `EBX` a `0`.

Bucle interno: Recorre códigos de tecla

El bucle interno usa `EBX` como contador de tecla:

```
LAB_00402b30:
00402b30  PUSH EBX
00402b31  CALL GetAsyncKeyState
00402b33  TEST AL, 0x1
00402b35  JZ LAB_00402c23
```

Aquí el malware llama a `GetAsyncKeyState(EBX)` para cada valor de `EBX`. El test `AL, 0x1` comprueba el bit bajo del resultado, que indica si la tecla fue presionada desde la última llamada a `GetAsyncKeyState`.

Luego incrementa `EBX` y sigue hasta `0xFF`:

```
00402c23  INC EBX
00402c24  CMP EBX, 0xff
00402c2a  JLE LAB_00402b30
```

Esto confirma que revisa el rango `0x00–0xFF`, típico de códigos virtual-key de Windows.

Procesamiento de teclas

Cuando `GetAsyncKeyState` detecta una pulsación, la función llama a:

```
00402b3d  CALL FUN_00401550
```

Esa función FUN_00401550 probablemente filtra o traduce códigos especiales. Si devuelve un valor distinto de cero, salta y no escribe el carácter directamente:

```
00402b42  TEST AL, AL
00402b44  JNZ LAB_00402c23
```

Esto encaja con la tabla de strings que ya vimos: `**ENTER**`, `**SPACE**`, `**BACKSPACE**`, `**TAB**`, `**F1**`, `**F12**`, `**LEFT_SHIFT**`, etc. Es decir, algunas teclas no se escriben como carácter bruto, sino como etiqueta textual. Si la tecla no fue tratada por esa función auxiliar, el código escribe el byte en `std::cout` y también en el `stream/local buffer`:

```
00402b59  MOV ECX, [std::cout]
00402b5f  MOV DL, BL
00402b61  CALL FUN_00404610
00402b66  MOV DL, BL
00402b68  LEA ECX, [local_c4]
00402b6e  CALL FUN_00404610
```

donde:

- Llama a la función FUN_00404610.
- Esto indica doble salida: consola y otro destino, probablemente el fichero de log.

¿Por qué no hay RET?

Esta función está diseñada como un bucle infinito de `keylogging`. La ausencia de RET se explica porque el flujo acaba en un salto incondicional hacia atrás:

```
00402c30  e9 eb fe ff ff    JMP LAB_00402b20
```

donde:

- Ese salto vuelve al comienzo del bucle externo, no a una rutina de salida.

Por tanto, el `RET` sería código inalcanzable. El compilador no necesita generar epílogo porque, desde su análisis, la función está pensada para no retornar.

Los bytes que aparecen después al final:

```
00402c35  CC
00402c36  CC
...
00402c3f  CC
```

no son el final lógico de la función. Son bytes `INT3` usados como relleno/alineación entre bloques o funciones. No están en una ruta ejecutable normal porque el salto en `0x402c30` vuelve al bucle.

Conclusiones:

La función `0x402A00` inicializa estructuras locales, prepara un flujo de salida/registro, y después entra en un bucle permanente que:

- Espera 10 ms con `Sleep(0xA)`.
- Recorre códigos de tecla desde `0x00` hasta `0xFF`.
- llama a `GetAsyncKeyState`.
- Si detecta una pulsación nueva, la procesa.
- Escribe el carácter o etiqueta en consola/fichero.
- Vuelve al inicio del bucle.

La función `0x402A00 - Keylogger_loop` implementa un bucle infinito de keylogging por polling. Tras inicializar estructuras locales y objetos de salida, carga la dirección de `GetAsyncKeyState` y entra en un ciclo permanente. En cada iteración ejecuta `Sleep(10)`, recorre los códigos de tecla de `0x00` a `0xFF`, consulta su estado con `GetAsyncKeyState` y registra las pulsaciones detectadas. La función no contiene epílogo ni instrucción `RET`, ya que al finalizar el recorrido de teclas ejecuta un salto incondicional hacia el inicio del bucle externo (`JMP LAB_00402b20`). Los bytes `0xCC` posteriores corresponden a relleno/alineación y no forman parte de una ruta normal de ejecución.

12.4 Función 0x405AD0 - x_capture_screenshot

La función 0x405AD0 implementa una captura de pantalla usando `APIs GDI` de Windows y después delega el guardado del resultado a `x_write_bmp_or_file_data`.

Resumen de la función 405AD0

Elemento	Valor
Función	`0x405AD0 - x_capture_screenshot`
Objetivo	Capturar pantalla
Técnica	GDI / `BitBlt`
Tamaño capturado	`0x780 x 0x438` → **1920 x 1080**
Formato probable	BMP
Función de guardado	`x_write_bmp_or_file_data`
Retorno	`AL`, probablemente éxito/error del guardado
MITRE ATT&CK	`T1113 - Screen Capture`

Obtiene el DC de la pantalla

```
00405ad8  PUSH 0x0
00405ada  CALL GetDC
```

Esto equivale a:

```
HDC hdcScreen = GetDC(NULL);
```

`GetDC(NULL)` obtiene el contexto de dispositivo de toda la pantalla.

Crea un DC compatible en memoria

```
00405ae2  PUSH ESI
00405ae6  CALL CreateCompatibleDC
```

Esto equivale a:

```
HDC hdcMem = CreateCompatibleDC(hdcScreen);
```

Este DC en memoria será el destino de la copia de pantalla.

Crea un bitmap compatible de 1920x1080

```
00405aec  PUSH 0x438
00405af1  PUSH 0x780
00405af8  PUSH ESI
00405afc  CALL CreateCompatibleBitmap
```

Esto equivale a:

```
HBITMAP hBitmap = CreateCompatibleBitmap(hdcScreen, 1920, 1080);
```

Donde:

Hex	Decimal	Significado
0x780	1920	Ancho
0x438	1080	Alto

Esto indica que la muestra no calcula dinámicamente la resolución, sino que usa una resolución fija de `1920x1080`.

Selecciona el bitmap en el DC de memoria

```
00405b02  PUSH EAX
00405b03  PUSH EBX
00405b07  CALL SelectObject
```

Esto equivale a:

```
HGDIOBJ oldObj = SelectObject(hdcMem, hBitmap);
```

Esto asocia el bitmap creado al DC en memoria para que pueda recibir la imagen copiada.

Captura la pantalla con BitBlt

```
00405b0d  PUSH 0xcc0020
00405b12  PUSH 0x0
00405b14  PUSH 0x0
00405b16  PUSH hdcScreen
00405b1b  PUSH 0x438
00405b20  PUSH 0x780
00405b25  PUSH 0x0
00405b27  PUSH 0x0
00405b29  PUSH hdcMem
00405b2a  CALL BitBlt
```

donde:

- `0x00CC0020` corresponde a `SRCCOPY`, operación estándar para copiar píxeles desde el DC origen al DC destino.

Copia la región superior izquierda de la pantalla, de tamaño 1920x1080, a un bitmap en memoria.

Guarda la captura como BMP

Después de `BitBlt`, aparece:

```
00405b30  MOV ECX, dword ptr [EBP + Stack[0x14]]
00405b36  PUSH EBX
00405b37  CALL x_write_bmp_or_file_data
```

Aquí `EBX` contiene el `hdcMem`, es decir, el `DC` de memoria que contiene la captura. La función `x_write_bmp_or_file_data` probablemente extrae los píxeles del bitmap y los escribe en disco como BMP.

Por el análisis previo, el destino esperado es:

```
C:\Users\Public\Public\Screens\screenshot.bmp
```

La función guarda el valor devuelto por `x_write_bmp_or_file_data` en BL:

```
00405b3f  MOV BL, AL
```

Y luego lo devuelve al final:

```
00405b68  MOV AL, BL
```

Por tanto, `AL` probablemente indica si la escritura del BMP fue exitosa.

Limpieza de recursos GDI

La función limpia correctamente los recursos usados:

```
00405b46  CALL SelectObject
00405b4f  CALL DeleteObject
00405b56  CALL DeleteDC
00405b61  CALL ReleaseDC
```

donde:

- Se evita dejar objetos GDI abiertos.

Sobre RET 0x18

La función termina con:

```
00405b6e  RET 0x18
```

Eso significa que la función limpia 24 bytes de argumentos de la pila al retornar.

Esto encaja con cómo se llama desde `0x402FA0`, donde antes de llamar a `x_capture_screenshot` se preparan 24 bytes en stack. En la práctica, parece que recibe una estructura o argumento relacionado con la ruta de salida del screenshot, probablemente el puntero/buffer que representa:

```
C:\Users\Public\Public\Screens\screenshot.bmp
```

Conclusiones

La función `0x405AD0`, renombrada como `x_capture_screenshot`, implementa la captura de pantalla mediante APIs GDI de Windows. Obtiene el contexto de dispositivo de la pantalla con `GetDC(NULL)`, crea un `DC` y un bitmap compatibles, copia una región fija de `1920x1080` píxeles mediante `BitBlt` usando la operación `SRCCOPY`, y posteriormente delega el guardado de la imagen a `x_write_bmp_or_file_data`. La función limpia los recursos `GDI` utilizados mediante `SelectObject`, `DeleteObject`, `DeleteDC` y `ReleaseDC`. Este comportamiento confirma la capacidad de captura de pantalla de la muestra, asociada a `MITRE ATT&CK T1113 - Screen Capture`.

12.5 Función 0x405850 - x_write_bmp_or_file_data

La función 0x405850 es la rutina que convierte una captura GDI en un fichero BMP y la escribe en disco.

Resumen funcional

Elemento	Valor
Función	`0x405850 - x_write_bmp_or_file_data`
Objetivo	Escribir screenshot en formato BMP
Entrada principal	Ruta de fichero + `HDC` con imagen capturada
Resolución usada	`1920 x 1080`
Formato	BMP
APIs clave	`CreateCompatibleBitmap`, `CreateCompatibleDC`, `BitBlt`, `GetDIBits`, `CreateFileA`, `WriteFile`
Retorno	`AL = 1` si éxito; probablemente `AL = 0` si falla
Técnica MITRE relacionada	`T1113 - Screen Capture`

Destacamos en esta función la parte de la creación del fichero BMP.

Creación del fichero BMP

```
00405939  PUSH 0x0
0040593b  PUSH 0x80
00405940  PUSH 0x2
00405942  PUSH 0x0
00405944  PUSH 0x0
00405946  PUSH 0x40000000
0040594b  PUSH dword ptr [EBP + local_28]
0040594e  CALL CreateFileA
```

Argumento	Valor	Significado
`GENERIC_WRITE`	`0x40000000`	Abrir para escritura
Share mode	`0`	Sin compartición
Creation disposition	`0x2`	`CREATE_ALWAYS`
Attributes	`0x80`	`FILE_ATTRIBUTE_NORMAL`

Escribe la cabecera BMP

Primero prepara la firma:

```
0040595f  MOV EAX, 0x4D42
00405964  MOV word ptr [EBP + local_1c], AX
```

donde:

- `0x4D42` en little-endian es: `BM`. Esto confirma que escribe un fichero BMP.

Después escribe `0x0E` bytes:

```
00405972  PUSH 0xE
00405979  CALL WriteFile
```

donde:

- `0x0E` son 14 bytes, el tamaño de `BITMAPFILEHEADER`.

Luego escribe `0x28` bytes:

```
00405997  PUSH 0x28
0040599b  CALL WriteFile
```

donde:

- `0x28` son 40 bytes, el tamaño de `BITMAPINFOHEADER`.

La estructura que se escribe es:

Parte	Tamaño
`BITMAPFILEHEADER`	14 bytes
`BITMAPINFOHEADER`	40 bytes
Tabla de colores, si aplica	variable
Datos de píxeles	`biSizeImage`

Indicadores que se confirman

Indicador	Valor
Formato escrito	BMP
Firma de fichero	`BM` / `0x4D42`
Ruta esperada	`C:\Users\Public\Public\Screens\screenshot.bmp`
API de creación	`CreateFileA`
Modo de apertura	`CREATE_ALWAYS`
Permiso	`GENERIC_WRITE`
Escritura	`WriteFile`
Extracción de píxeles	`GetDIBits`
Resolución	`1920 x 1080`

Conclusiones

La función `0x405850`, renombrada como `x_write_bmp_or_file_data`, implementa la escritura de la captura de pantalla en formato BMP. Recibe una ruta de salida y un HDC origen, crea un bitmap compatible de `1920x1080`, copia la imagen mediante `BitBlt`, extrae los píxeles con `GetDIBits` y genera manualmente un fichero BMP usando `CreateFileA`, `WriteFile` y `SetFilePointer`. La función escribe la cabecera `BITMAPFILEHEADER`, la estructura `BITMAPINFOHEADER`, una posible tabla de colores y los datos de píxeles, actualizando posteriormente la cabecera con el tamaño final del fichero y el offset de datos. Este comportamiento confirma que la muestra almacena capturas de pantalla en disco, probablemente en `C:\Users\Public\Public\Screens\screenshot.bmp`.

12.6 Función 0x402C40

Esta función es el segundo hilo worker que lanza la rutina `x_install_and_start_threads`. Su función principal es tomar capturas de pantalla periódicas y guardarlas con nombres numerados.

Resumen funcional

Elemento	Análisis
Función	`0x402C40 - FUN_00402C40`
Propósito	Generar nombres de screenshot y llamar periódicamente a `x_capture_screenshot`
Ruta base	`C:\Users\Public\Public\Screens\screenshot`
Extensión	`.bmp`
Intervalo	`0xC350` ms = **50000 ms = 50 segundos**
Función llamada	`x_capture_screenshot` en `0x402EFA`
Tipo de ejecución	Bucle persistente / hilo
Retorno normal	No se observa retorno normal. Funciona como bucle

Construye la ruta base del screenshot

Al inicio construye una cadena con:

```
00402c6d  PUSH 0x29
00402c6f  PUSH s_C:\Users\Public\Public\Screens\s_00408604
00402c97  CALL FUN_004044d0
```

donde:

- La cadena `C:\Users\Public\Public\Screens\screenshot` todavía sin número y sin extensión.

Espera 50 segundos

```
00402c9c  PUSH 0xc350
00402ca8  CALL Sleep
```

donde:

- Esto indica que el hilo no toma screenshots de forma continua, sino cada cierto intervalo.

Construye un nombre numerado

```
ADD CL, 0x30
```

donde:

- La función mantiene tres contadores locales y los convierte a caracteres ASCII sumando 0x30.

La extensión .bmp

```
00402e23  MOV EAX,[DAT_00408630] = 706D622Eh
```

donde:

- `0x706D622E` en little-endian corresponde a: `.bmp`.

Llama a x_capture_screenshot

```
00402efa  CALL x_capture_screenshot
```

donde:

- Comprobamos que esta función no captura directamente la pantalla. Prepara el nombre del fichero y delega la captura en `x_capture_screenshot`.

Vuelve a dormir y repite

```
00402f65  PUSH 0xc350
00402f6a  CALL Sleep
...
00402f82  JNZ LAB_00402cc3
```

donde:

- Esto vuelve al bloque donde construye el siguiente nombre de screenshot. Por tanto, es un bucle periódico.

Renombramos la función

La renombramos como:

```
x_screenshot_worker_thread
```

Relación con el flujo general del malware

Esta función completa el análisis de los dos hilos creados por [0x402FA0]:

Hilo	Función	Comportamiento
Hilo 1	0x402A00 - x_keylogger_loop	Captura teclas con `GetAsyncKeyState`
Hilo 2	0x402C40 - x_screenshot_worker_thread	Toma screenshots periódicos cada 50 segundos

Esto confirma que la muestra no sólo toma una captura inicial, sino que probablemente sigue tomando capturas durante la ejecución.

Conclusiones

La función `0x402C40` implementa un hilo encargado de realizar capturas de pantalla periódicas. Inicializa una ruta base en `C:\Users\Public\Public\Screens\screenshot`, construye nombres de fichero añadiendo un contador decimal de tres dígitos y la extensión `.bmp`, y llama a `x_capture_screenshot` para generar la captura. Entre iteraciones ejecuta `Sleep(0xC350)`, equivalente a `50 segundos`, por lo que la muestra realiza capturas de pantalla de forma periódica durante su ejecución. Esta función confirma la capacidad de screen capture recurrente de la muestra, complementando el keylogger ejecutado en el hilo `0x402A00`.

12.7 Función 0x401550

La función `0x401550` es una rutina auxiliar del `keylogger`. Su función es detectar teclas especiales y registrarlas con etiquetas legibles, en vez de dejar que el bucle principal escriba el carácter bruto.

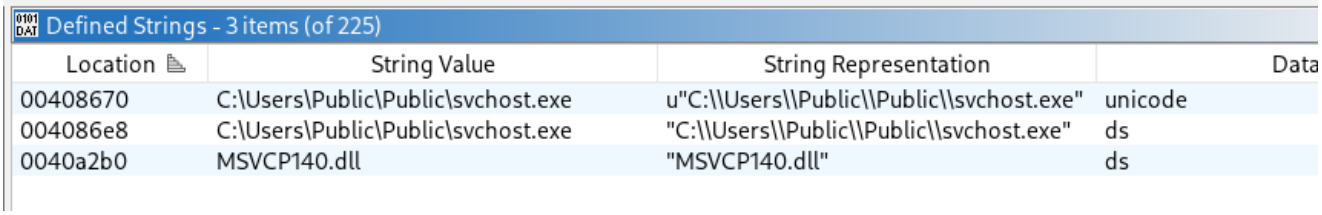
La función `0x401550` implementa el tratamiento de teclas especiales dentro del keylogger. Recibe un código de tecla virtual, lo procesa mediante una tabla `switch` y, para teclas como `SPACE, ENTER, BACKSPACE, LEFT_CLICK y RIGHT_CLICK`, escribe una etiqueta textual tanto en consola como en el mecanismo de registro usado por la muestra. Si la tecla es reconocida como especial, devuelve `1`, lo que evita que el bucle principal escriba el carácter bruto. Si no la reconoce, devuelve `0` y permite que `x_keylogger_loop` registre directamente el valor de la tecla.

---

13. Referencias Cruzadas

13.1 XREFs a svchost.exe

Vamos a localizar todas las XREFs a `C:\Users\Public\Public\svchost.exe`. Para ello en Ghidra: `Window → Defined Strings`. En el filtro de la ventana escribimos: `svchost`. Obtenemos:

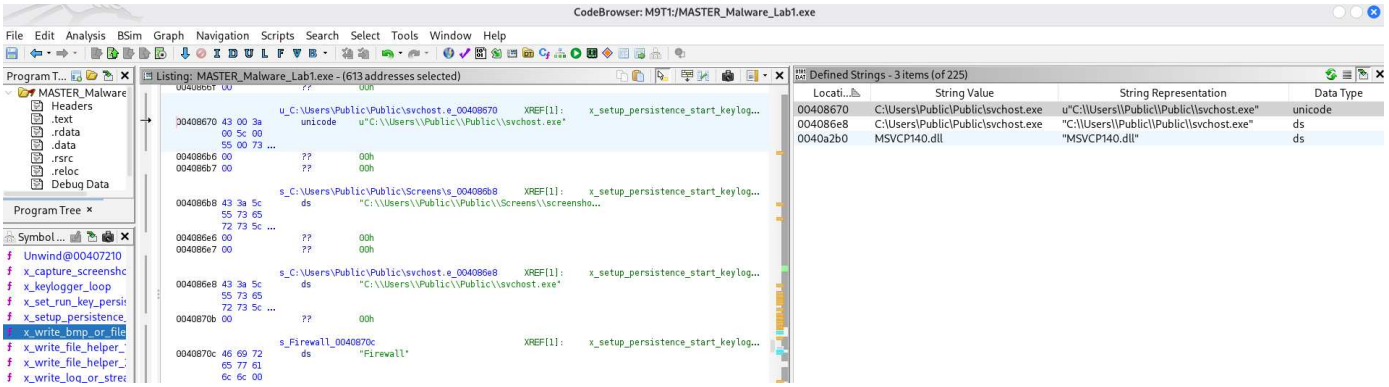


Defined Strings - 3 items (of 225)

Location	String Value	String Representation	Data
00408670	C:\Users\Public\Public\svchost.exe	u"C:\\Users\\Public\\Public\\svchost.exe"	unicode
004086e8	C:\Users\Public\Public\svchost.exe	"C:\\Users\\Public\\Public\\svchost.exe"	ds
0040a2b0	MSVCP140.dll	"MSVCP140.dll"	ds

La captura confirma que Ghidra ha identificado dos cadenas distintas relacionadas con `svchost.exe`:

Location	String	Tipo	Uso probable
`0x00408670`	`C:\Users\Public\Public\svchost.exe`	Unicode / UTF-16LE	Usada por APIs wide-char, especialmente `CopyFileW`
`0x004086E8`	`C:\Users\Public\Public\svchost.exe`	ASCII / `ds`	Usada para persistencia con `REG ADD` / `system()`
`0x0040A2B0`	`MSVCP140.dll`	ASCII / `ds`	Librería runtime de C++, no relacionada con persistencia

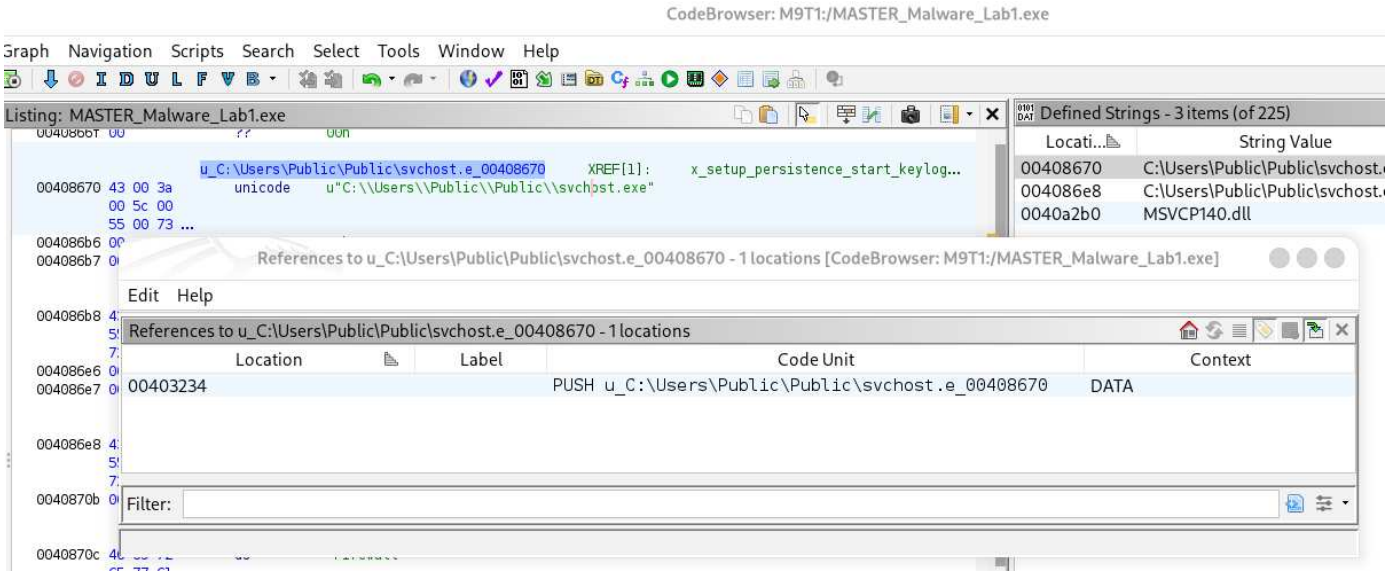


Cadena Unicode en `0x00408670`: Esta cadena se usa como ruta destino para copiar el malware.

Cadena ASCII en `0x004086E8`: Esta cadena se usa como ruta persistente registrada en la clave `Run`.

0x00408670

Mostramos las referencias de `0x00408670`: Sobre la cadena Unicode hacemos: `Right click → References → Show References to Address`:



donde:

- Esa XREF confirma que la cadena Unicode en `0x00408670` se usa en una única ubicación:

String	Dirección de string	XREF encontrada	Uso
`C:\Users\Public\Public\svchost.exe`	`0x00408670`	`0x00403234`	Argumento de `CopyFileW`

Esta XREF:

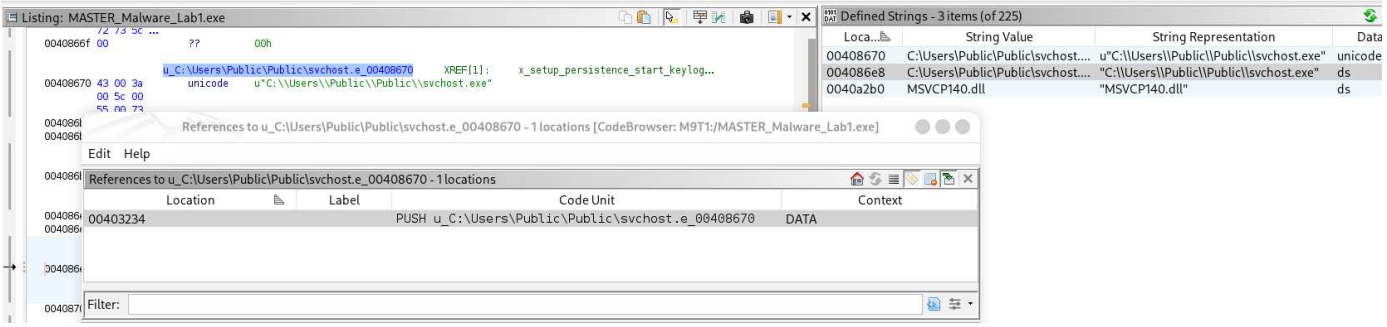
Elemento	Confirmación
Ruta destino	`C:\Users\Public\Public\svchost.exe`
API usada	`CopyFileW`
Tipo de string	Unicode / UTF-16LE
Acción	Copia del ejecutable actual
Sobrescritura	No sobrescribe si ya existe, porque `bFailIfExists = TRUE`
Finalidad probable	Instalación local / camuflaje como `svchost.exe`

```
                    LAB_0040321e                                    XREF[2]:     004031ba(j), 00403214(j)
0040321e 83 fa 08        CMP         EDX,0x8
00403221 c6 45 fc 03     MOV         byte ptr [EBP + local_8],0x3
00403225 6a 01           PUSH        0x1
00403227 8d 85 a8        LEA         EAX=>local_25c,[EBP + 0xfffffda8]
         fd ff ff
0040322d 0f 43 85        CMOVNC      EAX=>local_25c,dword ptr [EBP + 0xfffffda8]
         a8 fd ff ff
00403234 68 70 86        PUSH        u_C:\Users\Public\Public\svchost.e_00408670      = u"C:\\Users\\Public\\Public\\s...
         40 00
00403239 50              PUSH        EAX
0040323a ff 15 50        CALL        dword ptr [->KERNEL32.DLL::CopyFileW]            = 000098d4
         80 40 00
00403240 6a 2d           PUSH        0x2d
00403242 68 b8 86        PUSH        s_C:\Users\Public\Public\Screens\s_004086b8      = "C:\\Users\\Public\\Public\\Sc...
         40 00
00403247 8d 8d c0        LEA         ECX=>local_244,[EBP + 0xfffffdc0]
         fd ff ff
0040324d c7 85 d0        MOV         dword ptr [EBP + local_234],0x0
         fd ff ff
         00 00 00 00
00403257 c7 85 d4        MOV         dword ptr [EBP + local_230],0xf
         fd ff ff
         0f 00 00 00
00403261 c6 85 c0        MOV         byte ptr [EBP + local_244],0x0
         fd ff ff 00
```

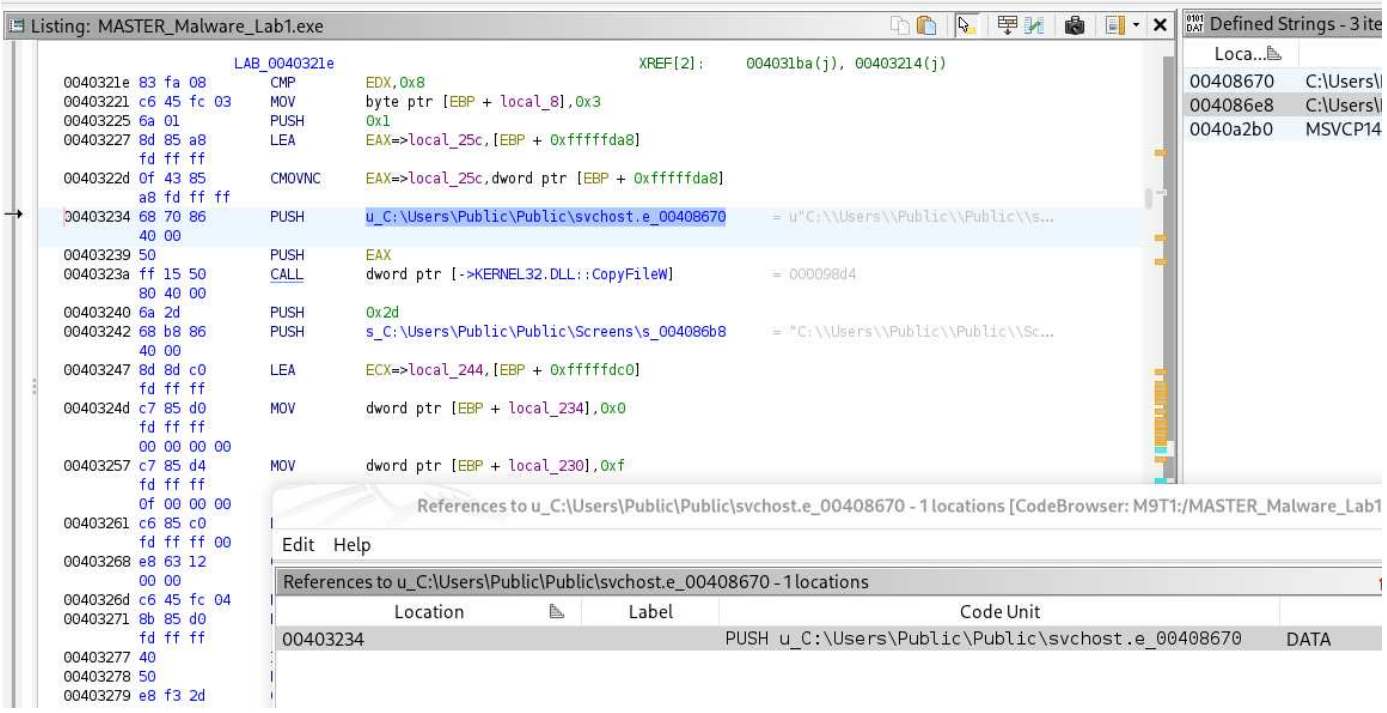
La referencia cruzada a la cadena Unicode `C:\Users\Public\Public\svchost.exe`, ubicada en `0x00408670`, aparece en la dirección `0x00403234`. En ese punto, la cadena se apila como argumento de la llamada a `CopyFileW`, donde se usa como ruta destino para copiar el ejecutable actual. Esto confirma que la muestra intenta instalar una copia de sí misma en `C:\Users\Public\Public\svchost.exe`, usando un nombre asociado a un proceso legítimo de Windows para camuflaje.

0x004086E8

Repetimos el proceso para ver las referencias a `0x004086E8`:



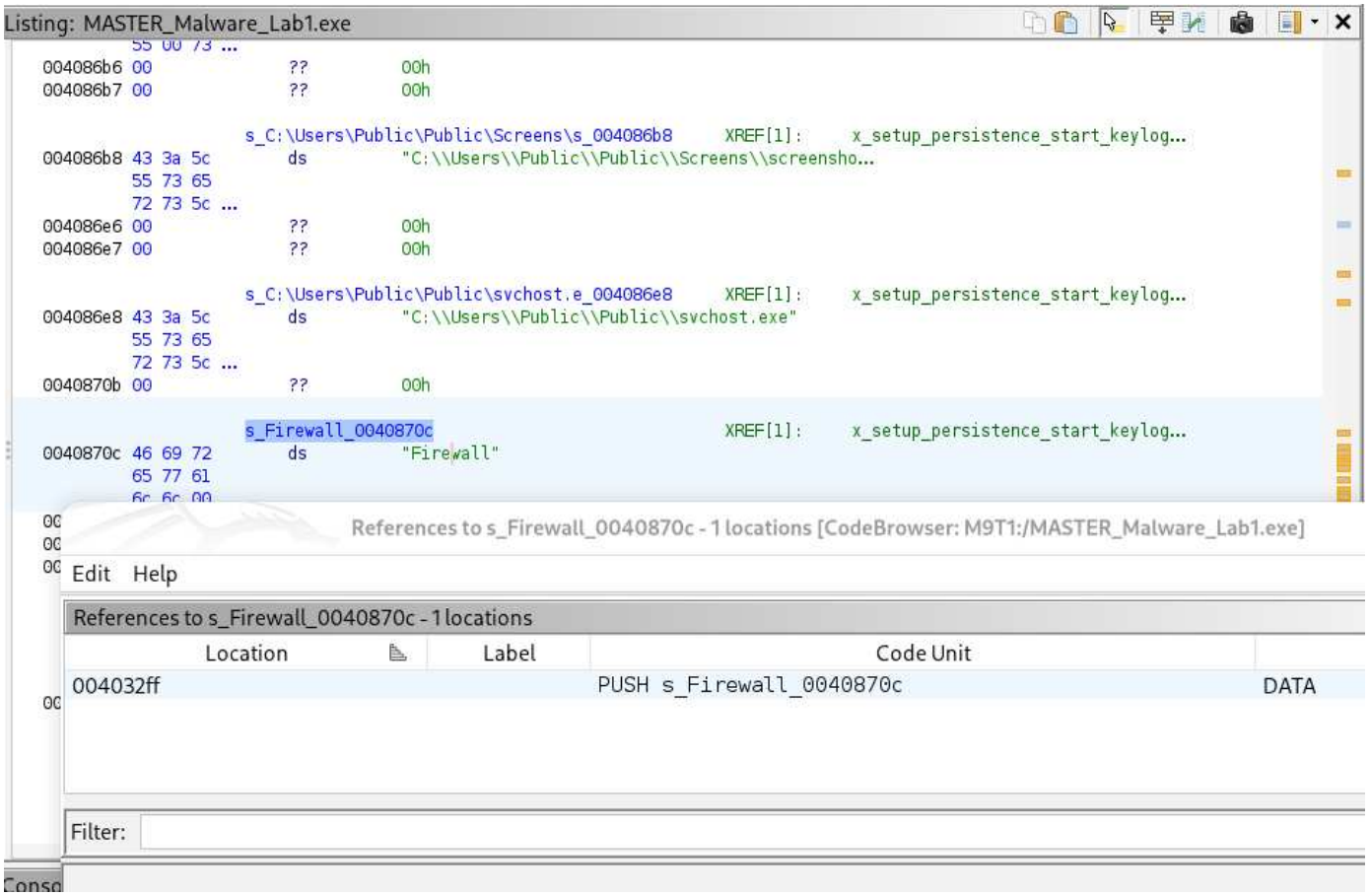
Vemos que esta cadena ASCII se usa para construir el comando de persistencia:



Elemento	Valor
String referenciada	`C:\Users\Public\Public\svchost.exe`
Dirección de la string	`0x004086E8`
Tipo	ASCII
Uso	Argumento para la rutina de persistencia
Función relacionada	`x_set_run_key_persistence`
Nombre de entrada Run	`Firewall`
Clave de persistencia	`HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`

La cadena ASCII `C:\Users\Public\Public\svchost.exe`, ubicada en `0x004086E8`, es referenciada en la rutina de inicialización antes de la llamada a `x_set_run_key_persistence`. Esta cadena se utiliza junto con el nombre `Firewall` para construir un comando `REG ADD` que registra el ejecutable copiado en `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`. Esto confirma que la ruta `C:\Users\Public\Public\svchost.exe` no sólo se usa como destino de copia, sino también como ejecutable persistente que se lanzará al inicio de sesión del usuario.

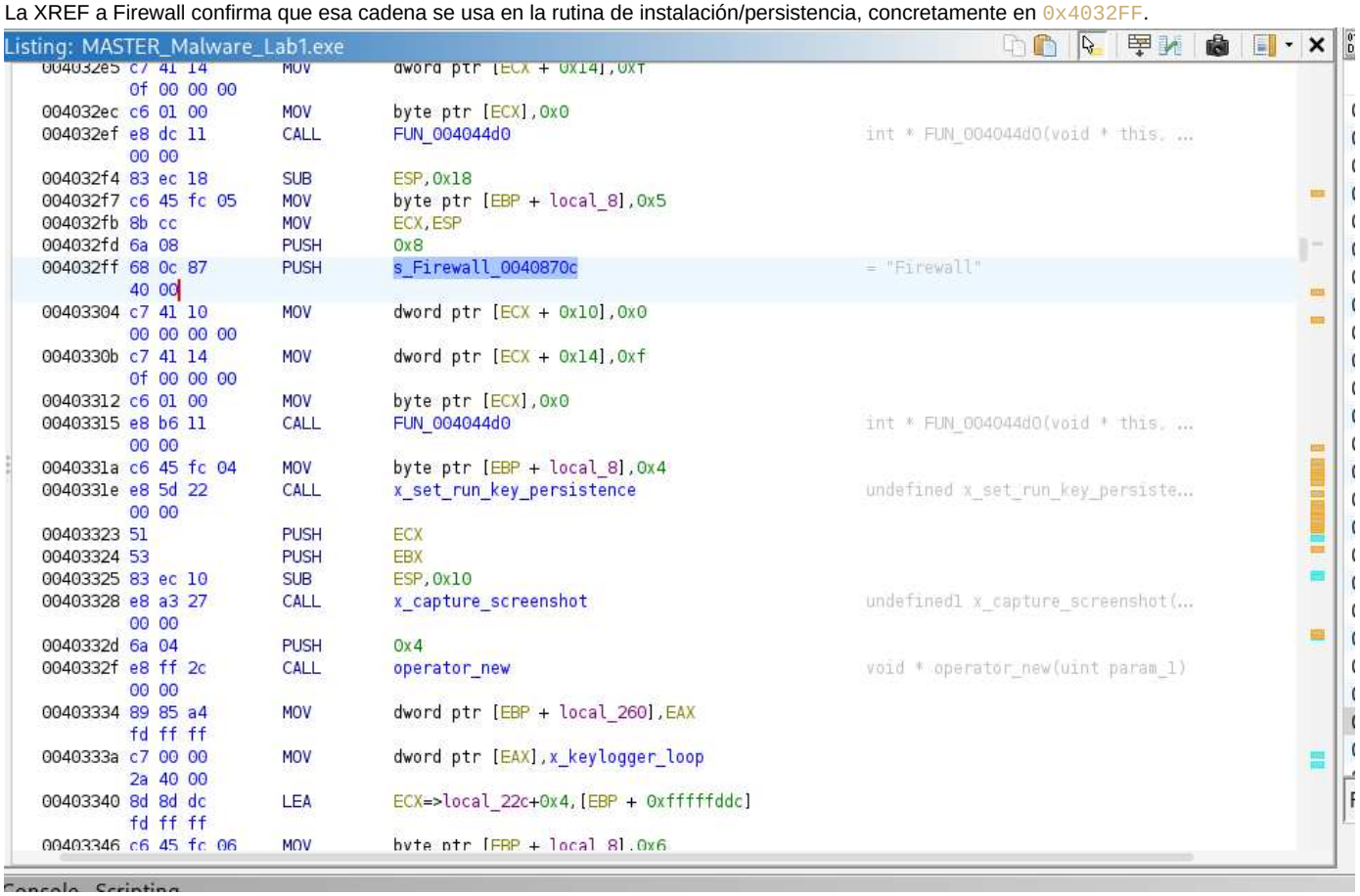
13.2 XREFs a Firewall



donde:

- Observamos que sólo hay una referencia directa a la cadena `Firewall`.

La XREF a Firewall confirma que esa cadena se usa en la rutina de instalación/persistencia, concretamente en `0x4032FF`.



donde:

- Observamos que la función `FUN_004044d0` parece ser un constructor o asignador de `std::string`. Recibe la cadena y su longitud.

Resultado de la XREF:

Elemento	Valor
String	`Firewall`
Dirección de la string	`0x0040870C`
Tipo	ASCII
XREF encontrada	`0x004032FF`
Función contenedora	`x_setup_persistence_start_keylog...` / `install_and_start_threads`
Uso	Nombre del valor de registro para persistencia

Interpretación: La muestra construye dos strings antes de llamar a `x_set_run_key_persistence`:

Dirección	String construida	Uso
`0x4032D9`	`C:\Users\Public\Public\svchost.exe`	Ruta del ejecutable persistente
`0x4032FF`	`Firewall`	Nombre del valor en la clave Run

Después llama a:

```
0040331e    CALL x_set_run_key_persistence
```

Por tanto, `Firewall` se usa como nombre del valor de registro, no como configuración real del firewall de Windows.

La persistencia quedaría conceptualmente así:

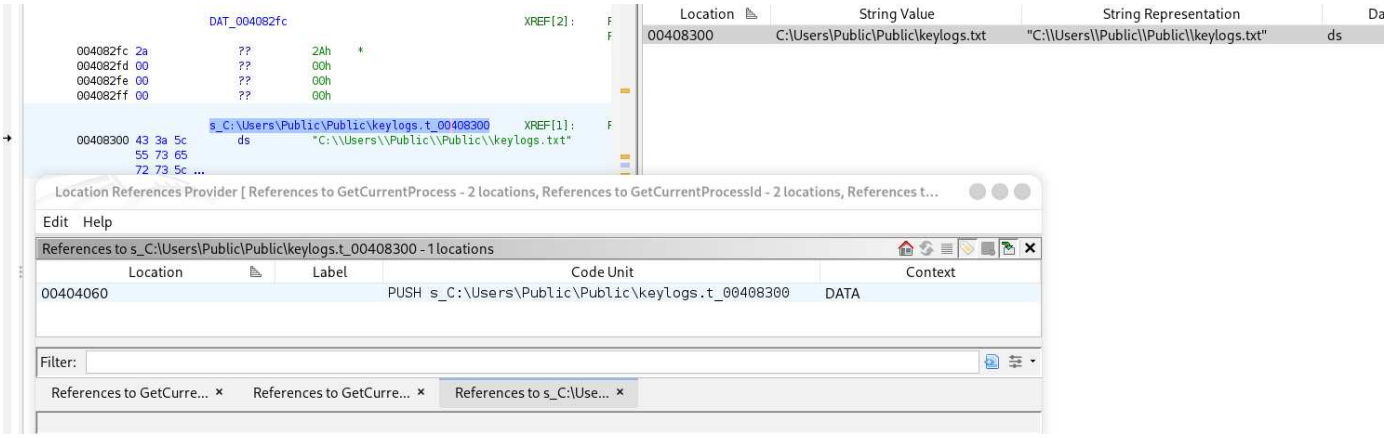
```
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
└── Firewall = C:\Users\Public\Public\svchost.exe
```

Campo	Valor
Hive	`HKCU`
Clave	`SOFTWARE\Microsoft\Windows\CurrentVersion\Run`
Nombre del valor	`Firewall`
Datos del valor	`C:\Users\Public\Public\svchost.exe`
Finalidad	Ejecutarse al inicio de sesión del usuario

Conclusión: La cadena `Firewall`, ubicada en `0x0040870C`, presenta una única referencia cruzada en `0x004032FF`. En dicha ubicación se construye un objeto `std::string` con el valor `Firewall`, inmediatamente antes de llamar a `x_set_run_key_persistence`. Esta cadena se utiliza como nombre del valor de registro creado en `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`, apuntando al ejecutable `C:\Users\Public\Public\svchost.exe`. Por tanto, `Firewall` funciona como nombre de persistencia y mecanismo de camuflaje, no como una funcionalidad legítima de firewall.

13.3 XREF a keylogs.txt

Analizamos las XREFs a `C:\Users\Public\Public\keylogs.txt`, para confirmar exactamente qué función abre/escribe el fichero de `keylogging`. En Ghidra abrimos: `Window → Defined Strings`. Filtramos por `keylog`. Doble clic en el string para ir a la parte del código donde aparece. Right click sobre la `string → References → Show References to ....keylog`:



Elemento	Valor
String	`C:\Users\Public\Public\keylogs.txt`
Dirección de la string	`0x00408300`
Tipo	ASCII / `ds`
XREF encontrada	`0x00404060`
Instrucción	`PUSH s_C:\Users\Public\Public\keylogs.txt_00408300`
Uso probable	Abrir/preparar el fichero de logs del keylogger

La XREF a `keylogs.txt` confirma que el binario contiene una ruta explícita para almacenar pulsaciones capturadas. La cadena se usa en `0x00404060`, probablemente dentro de una función auxiliar encargada de abrir o inicializar el fichero de log.

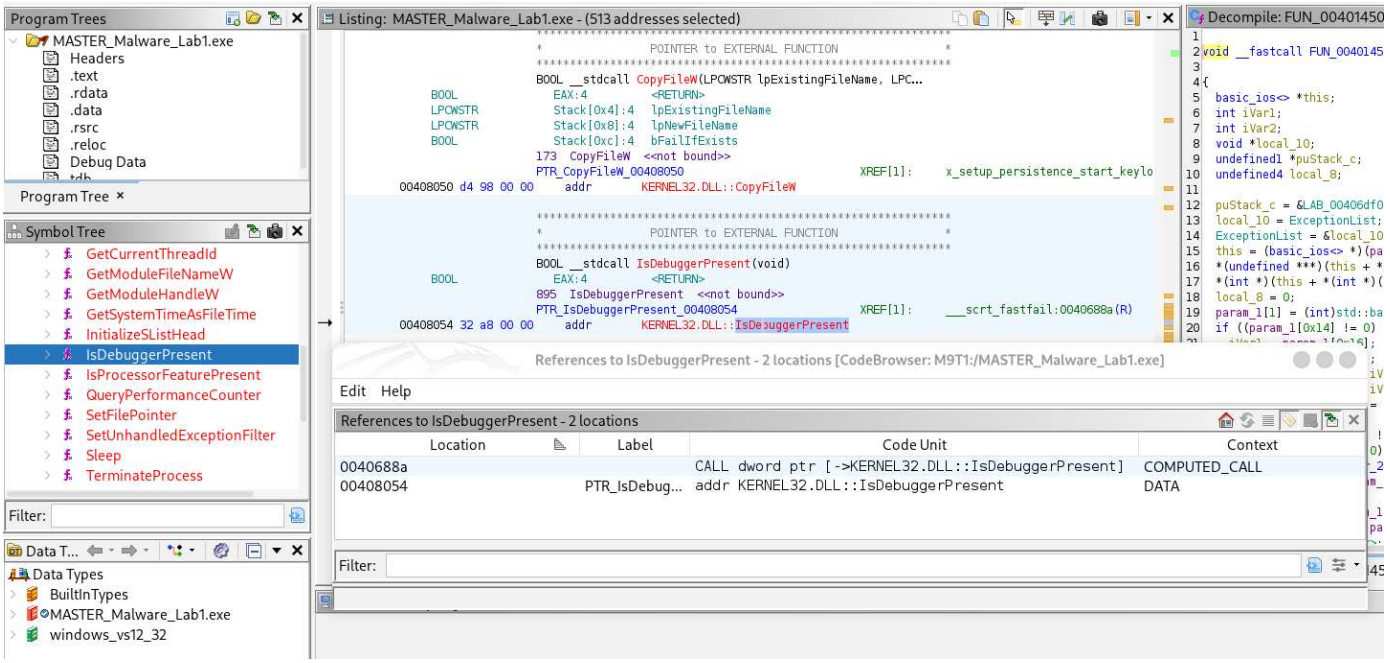
La función `FUN_00403FF0` referencia directamente la cadena `C:\Users\Public\Public\keylogs.txt` y la pasa como argumento a `std::_Fiopen`, confirmando que la muestra abre dicho fichero mediante el runtime C++ para registrar las pulsaciones capturadas. Esta función es invocada desde la rutina de escritura de strings del keylogger y desde el bucle principal de captura de teclas, por lo que el fichero `keylogs.txt` queda confirmado como destino local de los datos de keylogging.

---

14. Técnicas Anti-Análisis

Vamos a confirmar si `IsDebuggerPresent` se usa realmente. Para ello comprobaremos si tiene `XREFs` desde código ejecutable y qué hace el programa con su valor de retorno.

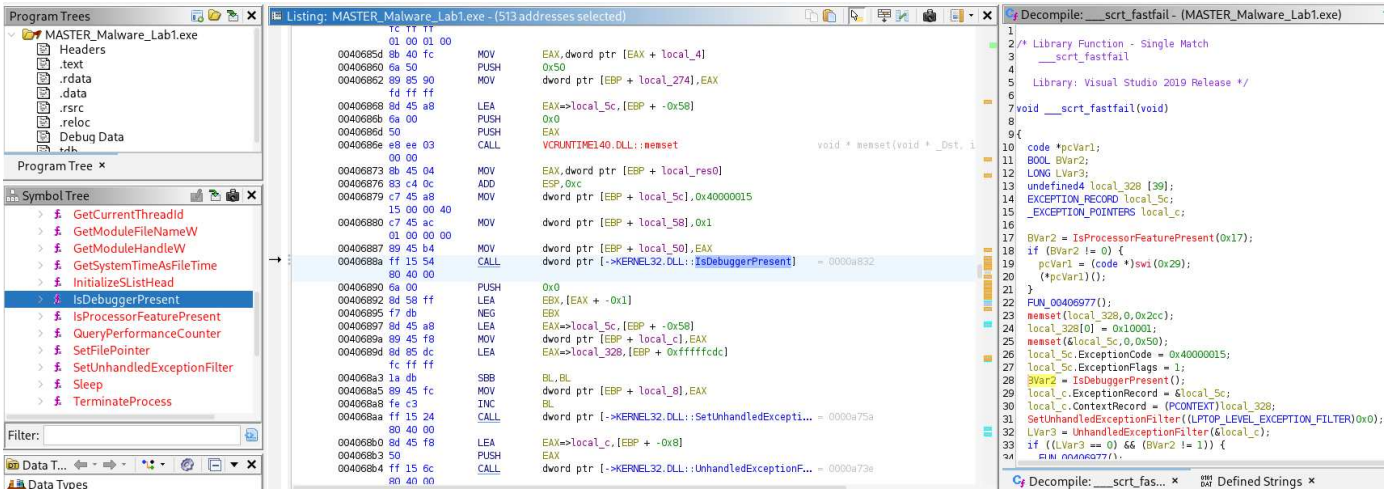
En Ghidra: `Symbol Tree → Imports → KERNEL32.dll → IsDebuggerPresent`. Sobre `IsDebuggerPresent`, hacemos doble clic para que muestre en el código dónde se usa. A continuación sobre el uso hacemos: `Click con el botón derecho → References → Show References to IsDebuggerPresent`:



donde:

- La captura confirma que `IsDebuggerPresent` sí tiene una `XREF` ejecutable, pero no parece formar parte de una lógica antidebug propia del malware.

Vemos el código donde aparece `IsDebuggerPresent`:

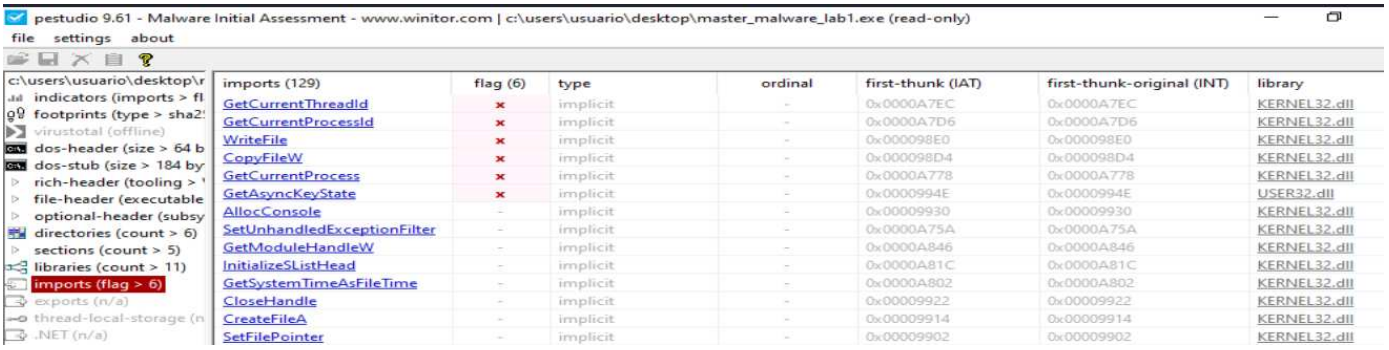


donde:

- Vemos que `IsDebuggerPresent` se ejecuta dentro de `__scrt_fastfail`, una rutina del runtime de Visual C++, no dentro de la lógica principal del malware.

Se verifican las referencias cruzadas a `IsDebuggerPresent`. Aunque la `API` está importada y presenta una llamada real en `0x0040688A`, dicha llamada se encuentra dentro de la función `__scrt_fastfail`, identificada por Ghidra como función de librería del runtime de Visual Studio 2019. El contexto incluye llamadas a `IsProcessorFeaturePresent`, `SetUnhandledExceptionFilter` y `UnhandledExceptionFilter`, lo que corresponde a manejo de fallos/excepciones del `CRT`. No se observan llamadas a `IsDebuggerPresent` desde las funciones principales del malware ni lógica de evasión condicionada por su resultado. Por tanto, no se considera una técnica antidebug propia de la muestra.

15. Finalizando el análisis estático



De los imports marcados por PEStudio como potencialmente peligrosos:

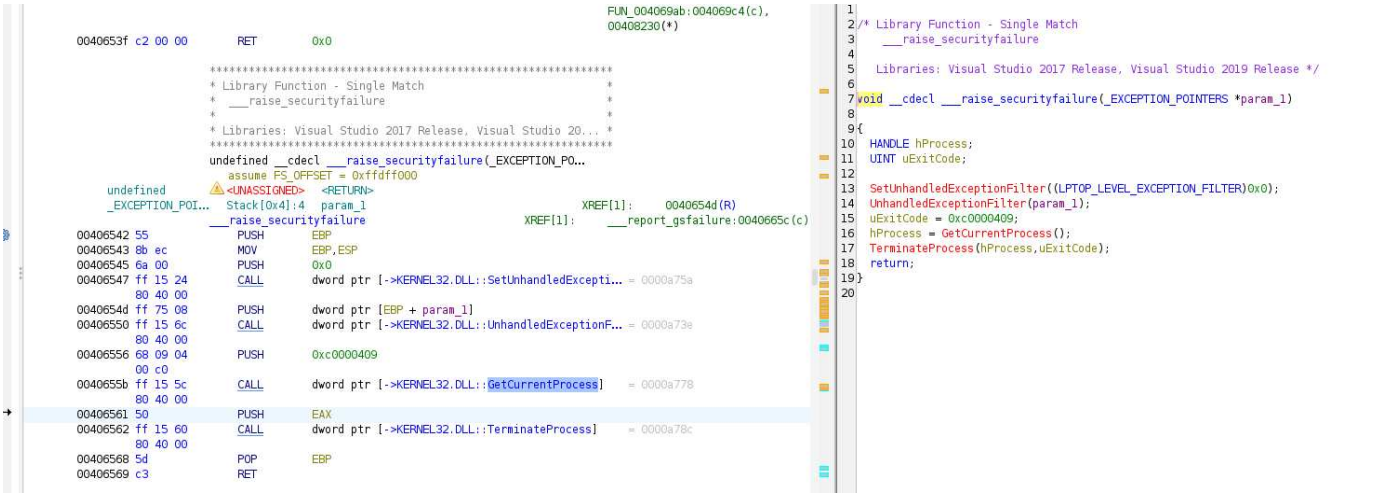
API importada	Resultado actual
`GetAsyncKeyState`	Confirmada en `0x402A00 - keylogger_loop`. Se usa para capturar teclas por polling.
`WriteFile`	Confirmada en `0x405850 - x_write_bmp_or_file_data`. Se usa para escribir el BMP de captura de pantalla.
`CopyFileW`	Confirmada en `0x402FA0 - install_and_start_threads`. Copia el binario a `C:\Users\Public\Public\svchost.exe`.

Nos quedan las APIs pendientes de analizar:

- `GetCurrentProcess`.
- `GetCurrentProcessId`.
- `GetCurrentThreadId`.

15.1 Import GetCurrentProcess

Vemos las `XREF` a esta API:



La API GetCurrentProcess aparece referenciada dentro de la función `___raise_securityfailure`, identificada como función de librería del runtime de Visual Studio. En este contexto, se utiliza para obtener el handle del proceso actual y pasarlo a `TerminateProcess` con el código `0xC0000409`, asociado a fallos de seguridad del runtime, como errores de `cookie /GS`. Por tanto, este import no se considera parte de la lógica maliciosa principal de la muestra, sino un artefacto del runtime de Microsoft Visual C++.

15.2 Import GetCurrentProcessId

La XREF a `GetCurrentProcessId` confirma que `GetCurrentThreadId` y `GetCurrentProcessId` no forman parte de la lógica maliciosa propia, sino del runtime de Visual Studio.

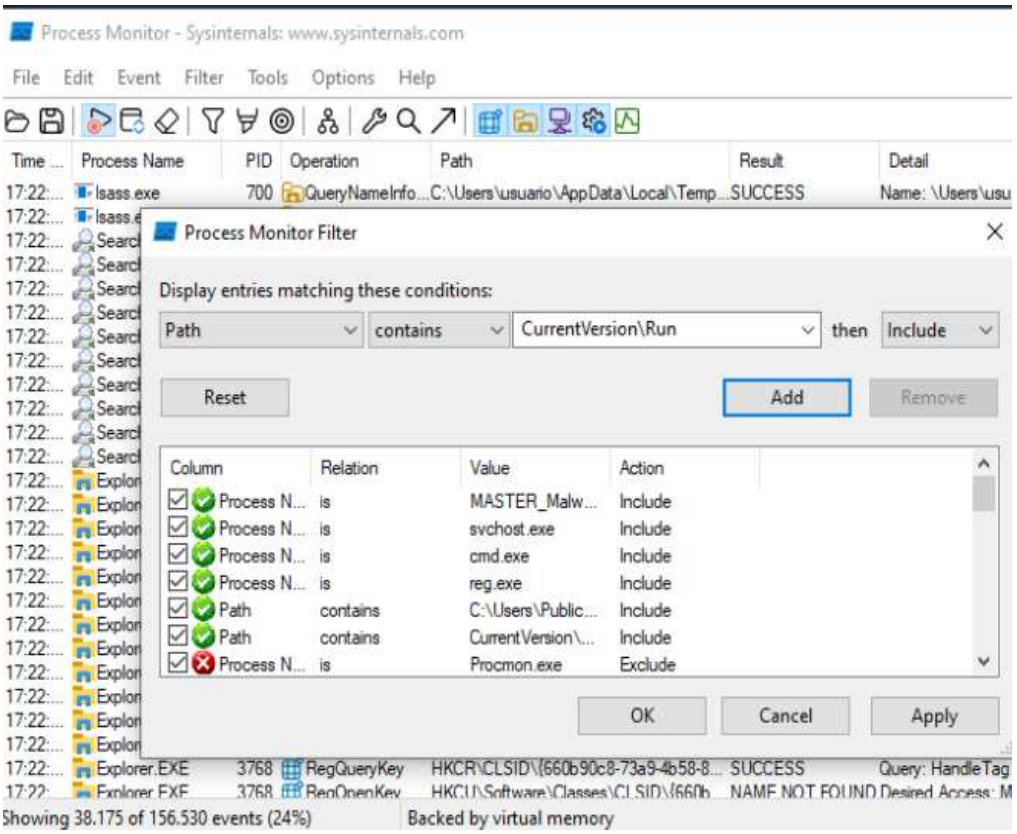
---

16. Análisis Dinámico

16.1 Preparamos la máquina virtual

Antes de ejecutar la muestra:

- Hacemos un snapshot con RegShot.
- Abrimos Procmon. Aplicamos los filtros:
  - Process Name is MASTER_Malware_Lab1.exe.
  - Process Name is svchost.exe.
  - Process Name is cmd.exe
  - Process Name is reg.exe
  - Path contains C:\Users\Public\Public.
  - Path contains CurrentVersion.
  - Operation is CreateFile.
  - Operation is WriteFile.
  - Operation is RegSetValue.
  - Operation is Process Create.



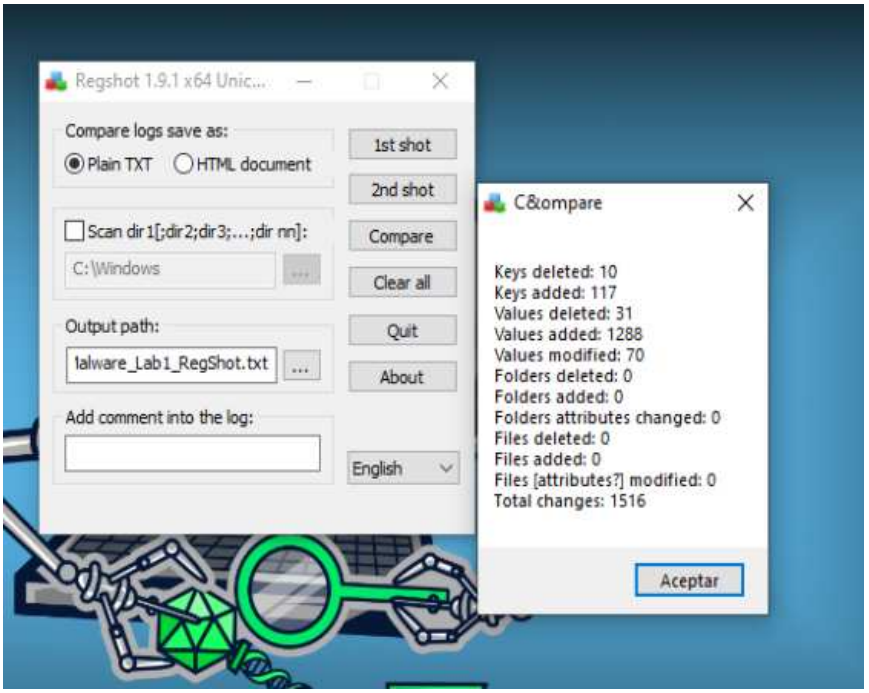
- Iniciamos la captura con Procmon.
- Abrimos Process Explorer.

16.2 Ejecutamos la muestra

- Escribimos algunas palabras en Notepad. Pulsamos Enter, Backspace, Space, F1....
- Esperamos unos minutos.
- Observamos si se crean los screenshots.

16.3 Paramos la muestra

- Finalizamos el proceso desde Process Explorer para que no se creen más screenshots.
- Paramos la captura en Procmon. En Process Monitor: Para detener la captura: `Ctrl + E`.
- Guardamos la captura realizada por Procmon - Process Monitor: `File → Save`. Formato recomendado: `Native Process Monitor Format (PML)`.
- Hacemos un segundo snapshot con RegShot.
    - Guardamos cada snapshot.
    - Guardamos la comparación que hace RegShot: `Compare → Output → Guardar en formato .txt y web`.



16.4 Comprobaciones

Persistencia en el registro

Comprobaremos con regedit: `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`.



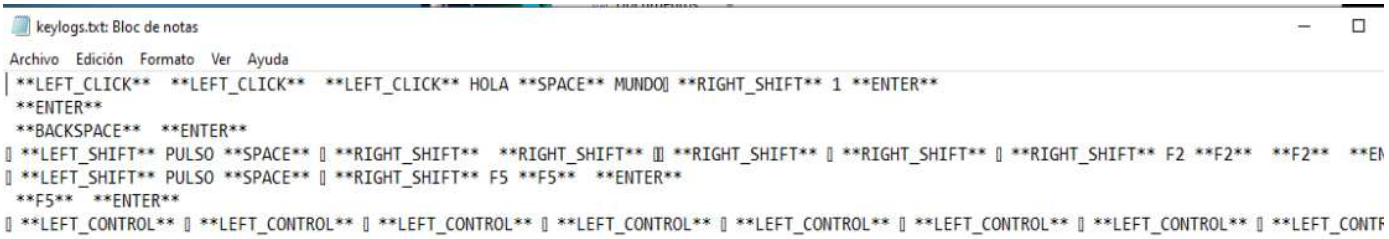
donde:

- Confirmamos dinámicamente la persistencia por clave `Run`.
- El malware ha creado una entrada de inicio automático en el contexto del usuario actual. Esto significa que, cuando el usuario inicie sesión, Windows intentará ejecutar: `C:\Users\Public\Public\svchost.exe`.

Esta evidencia valida dinámicamente la función de persistencia identificada en el análisis estático y confirma la técnica MITRE ATT&CK `T1547.001 - Registry Run Keys / Startup Folder`.

Revisamos `keylogs.txt`

Abrimos el fichero `C:\Users\Public\Public\keylogs.txt`. Comprobamos su contenido:



donde:

- Comprobamos que el fichero contiene texto y etiquetas de teclas especiales registradas durante la ejecución de la muestra.

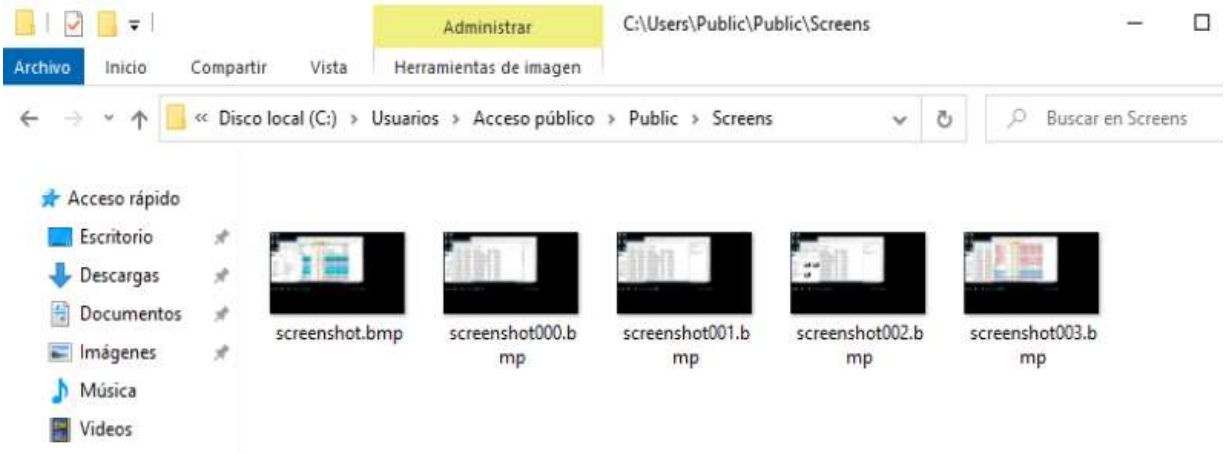
Esto valida lo que vimos en estático:

```
GetAsyncKeyState
 → keylogger_loop
 → handle_special_key
 → append_string_to_keylog
 → C:\Users\Public\Public\keylogs.txt
```

El malware no sólo detecta caracteres normales, sino también teclas especiales y eventos de ratón. Las etiquetas como `**ENTER**, **SPACE**, **BACKSPACE**, **LEFT_CONTROL** y **LEFT_CLICK**` coinciden con los strings y el mapeo de teclas que analizamos en `FUN_00401550`. Esta evidencia confirma que la muestra captura pulsaciones mediante polling y las almacena localmente en disco.

Revisamos carpeta de screenshots

Comprobamos si realiza las capturas de pantalla: `C:\Users\Public\Public\Screens`.



Esto valida lo que vimos en estático:

```
0x402FA0 - setup/install
 ├── x_capture_screenshot → screenshot.bmp
 └── lanza hilo 0x402C40
       └── screenshot_worker_thread
            └── screenshot000.bmp, screenshot001.bmp, screenshot002.bmp...
```

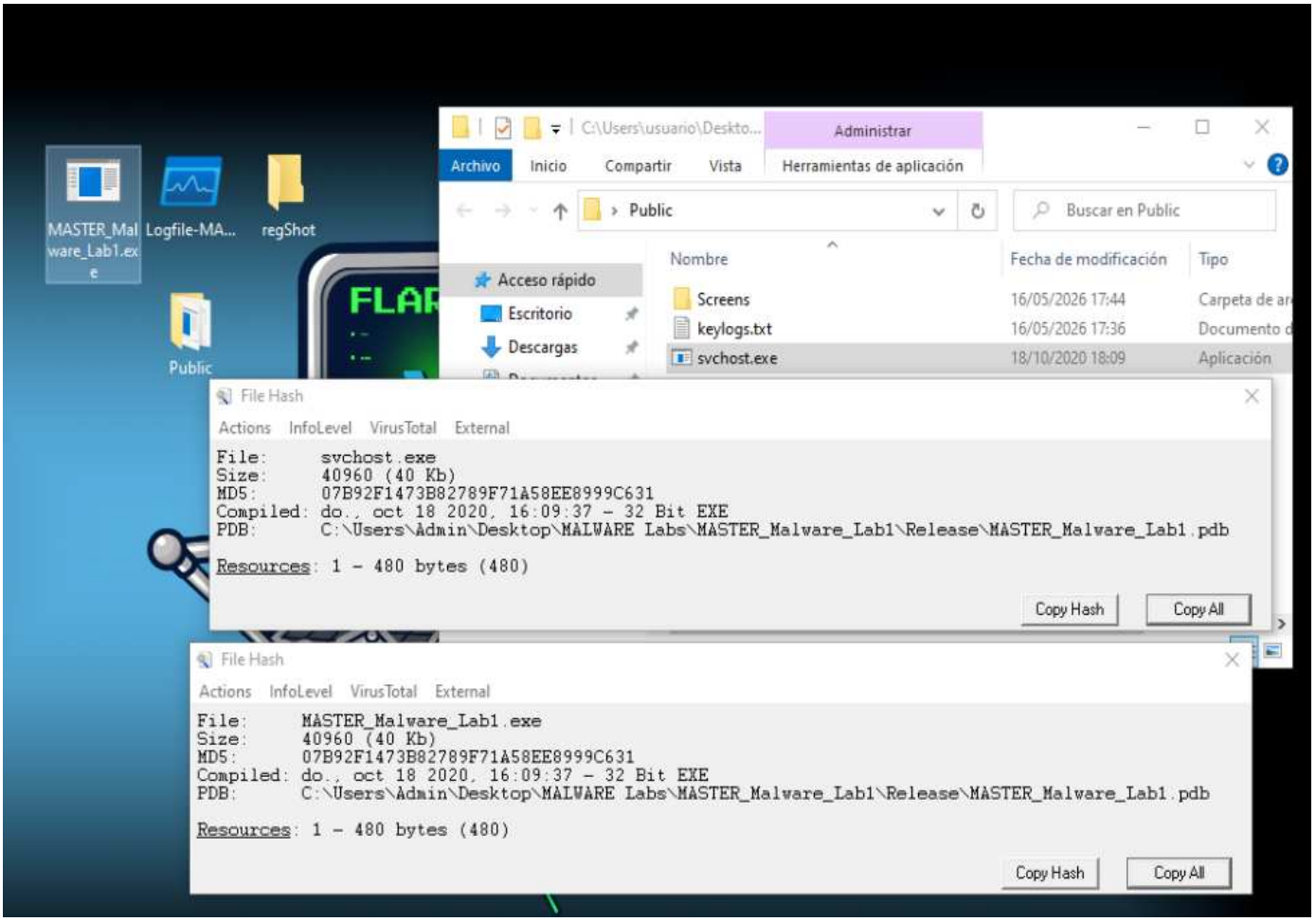
donde:

- Este comportamiento valida el análisis estático de las funciones `x_capture_screenshot`, `x_write_bmp_or_file_data` y `x_screenshot_worker_thread`, confirmando que la muestra realiza capturas de pantalla iniciales y periódicas.

La muestra no sólo toma una captura inicial, sino que mantiene un hilo activo que genera capturas sucesivas con nombres numerados. Esto confirma dinámicamente la funcionalidad de screen capture asociada a MITRE ATT&CK: `T1113 - Screen Capture`.

Confirmamos que `svchost.exe` copiado es la misma muestra del malware

Comparamos los hashes para confirmar que es una copia exacta.



Se confirma:

Fichero	Ubicación	MD5	Interpretación
`MASTER_Malware_Lab1.exe`	Escritorio	`07B92F1473B82789F71A58EE8999C631`	Muestra original
`svchost.exe`	`C:\Users\Public\Public\svchost.exe`	`07B92F1473B82789F71A58EE8999C631`	Copia generada por la muestra

Esta evidencia valida dinámicamente lo que vimos en estático con `CopyFileW`:

```
CopyFileW(
    ruta_actual_del_malware,
    L"C:\\Users\\Public\\Public\\svchost.exe",
    TRUE
);
```

Esto confirma que la muestra utiliza `CopyFileW` para instalar una copia local del binario bajo un nombre aparentemente legítimo.

Analizamos la comparación de los Shots realizados por RegShot

Del fichero de la comparacion de RegShot

Destacamos:

```
HKU\S-1-5-21-929642734-3731995757-1771841547-
1001\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\Firewall: "C:\Users\Public\Public\svchost.exe"
```

Eso equivale a:

```
HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\Firewall
```

donde:

- `HKU\S-1-5-21-...\1001` es el hive del usuario actual.
- El valor Firewall apunta al binario copiado `C:\Users\Public\Public\svchost.exe`.

Evidencia que aporta RegShot:

Evidencia	Interpretación
`Run\Firewall` añadido	Persistencia al inicio de sesión
Ruta `C:\Users\Public\Public\svchost.exe`	Ejecutable persistente
Hive `HKU\S-1-5-21-...\1001`	Corresponde al usuario analizado
No es `HKLM`	No requiere privilegios de administrador
Nombre `Firewall`	Camuflaje como componente legítimo/defensivo

El informe generado con RegShot confirma la creación de una entrada de persistencia en el hive del usuario actual. Se observó el valor Firewall bajo `HKU\S-1-5-21-929642734-3731995757-1771841547-1001\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`, apuntando a `C:\Users\Public\Public\svchost.exe`. Esta ruta corresponde a la copia del malware creada durante la ejecución. Dado que la clave se encuentra en el contexto de usuario, la persistencia no requiere privilegios administrativos. Este resultado valida dinámicamente la función estática `x_set_run_key_persistence`.

Revisamos los procesos

Hallazgos relevantes en Procmon:

Evidencia en el PML	Interpretación
`C:\Users\usuario\Desktop\MASTER_Malware_Lab1.exe`	Ejecución de la muestra original
`cmd.exe`	La muestra lanza comandos mediante `cmd.exe`
`reg.exe`	Se usa `reg.exe` para crear persistencia
`REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D C:\Users\Public\Public\svchost.exe`	Comando exacto de persistencia
`C:\Users\Public\Public\svchost.exe`	Copia local del malware
`HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\Firewall`	Valor de persistencia creado
`C:\Users\Public\Public\keylogs.txt`	Fichero de salida del keylogger
`C:\Users\Public\Public\Screens`	Directorio de capturas

Evidencia en el PML	Interpretación
`screenshot.bmp`, `screenshot000.bmp`, `screenshot001.bmp`, `screenshot002.bmp`, `screenshot003.bmp`	Capturas de pantalla generadas

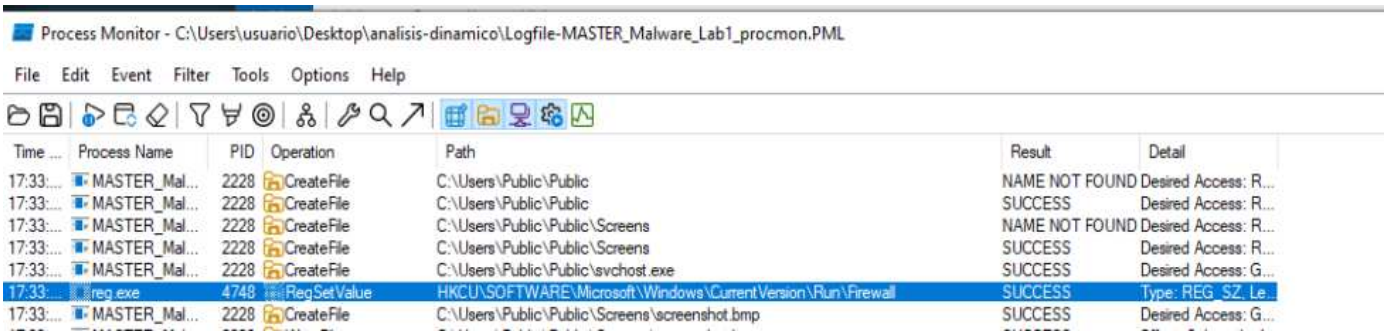
Confirmaciones dinámicas:

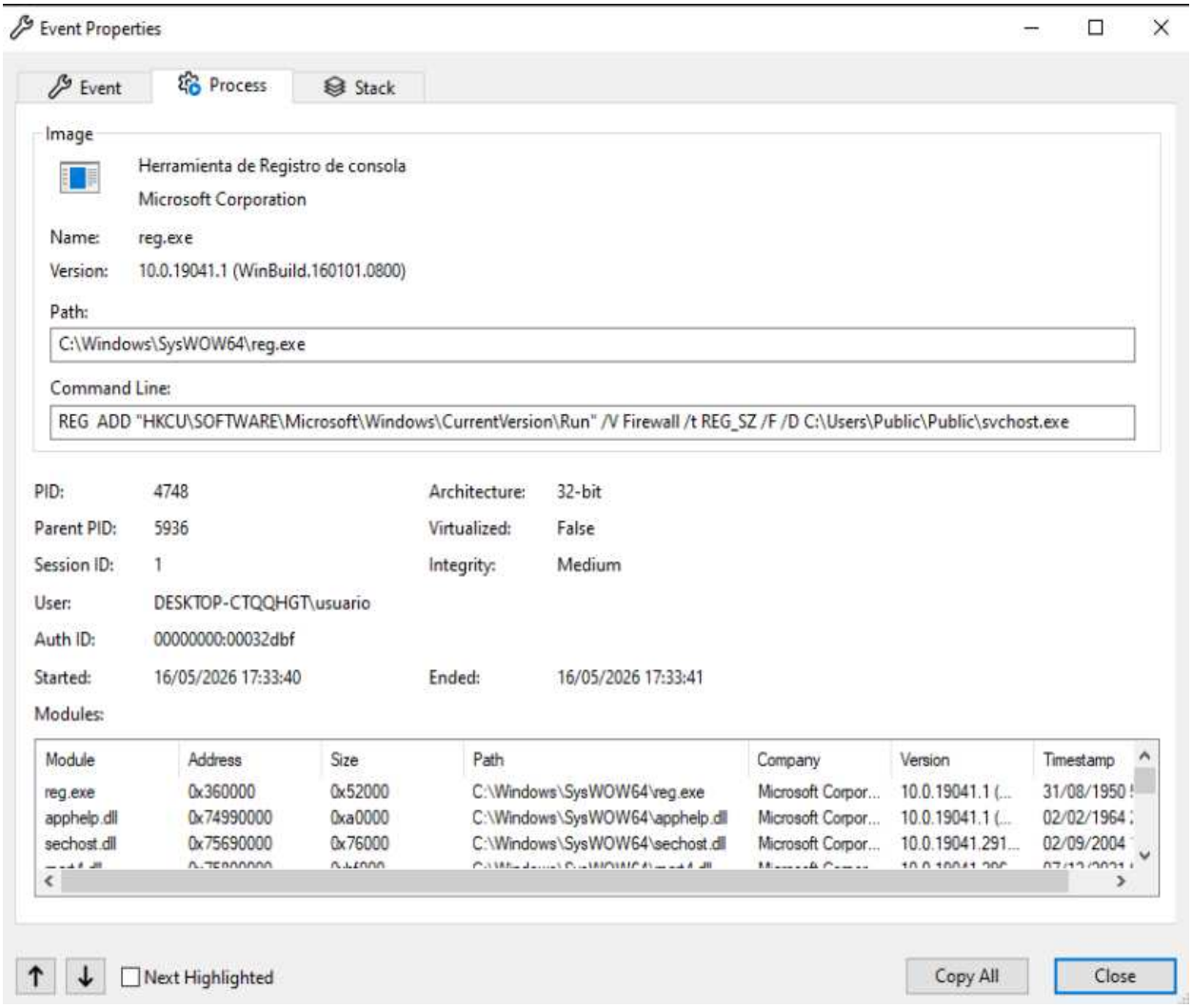
Capacidad	Evidencia
Persistencia	Ejecución de `REG ADD` con valor `Firewall` bajo `HKCU\...\Run`
Instalación local	Uso de `C:\Users\Public\Public\svchost.exe`
Keylogging	Acceso repetido a `C:\Users\Public\Public\keylogs.txt`
Captura de pantalla	Creación/acceso a BMPs en `C:\Users\Public\Public\Screens`

La presencia del comando completo:

```
REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D
C:\Users\Public\Public\svchost.exe
```

es especialmente importante porque conecta directamente el análisis estático de `x_set_run_key_persistence` con el comportamiento real observado.

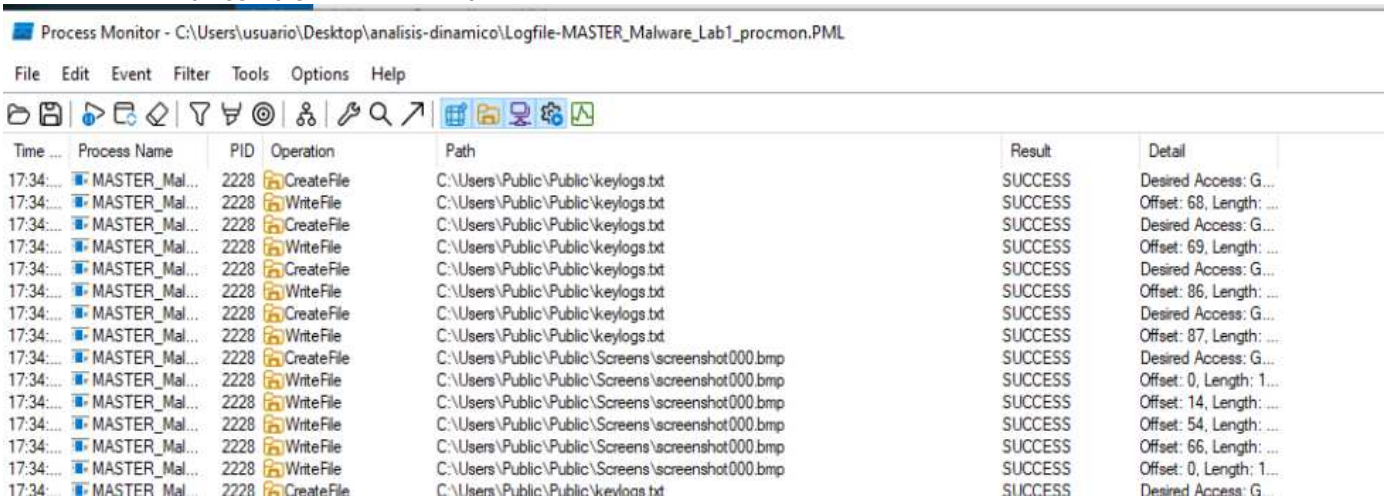




Process Monitor confirmó la creación de persistencia mediante el proceso reg.exe. La muestra ejecutó el comando `REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D C:\Users\Public\Public\svchost.exe`, creando correctamente el valor `Firewall` bajo `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`. El evento `RegSetValue` obtuvo resultado `SUCCESS`, por lo que la persistencia queda confirmada dinámicamente. El uso de `HKCU` indica persistencia a nivel de usuario, sin requerir privilegios administrativos.

Se documenta la técnica MITRE ATT&CK `T1547.001 - Registry Run Keys / Startup Folder`.

La escritura del keylogger y generación de capturas BMP:



donde:

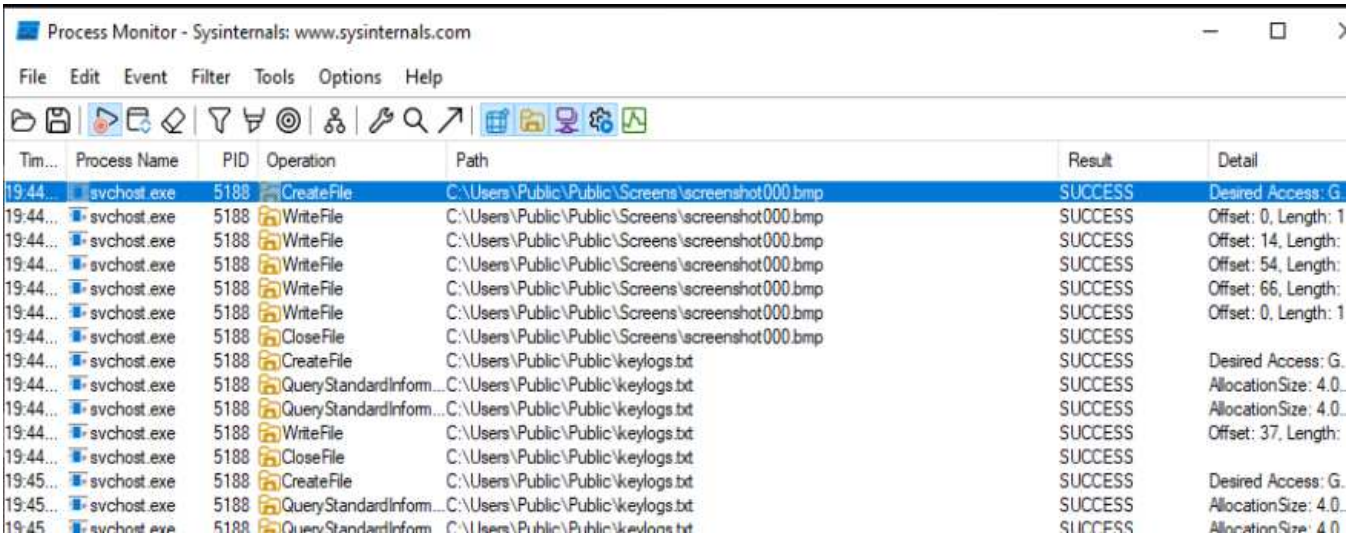
- Procmon confirma que MASTER_Malware_Lab1.exe escribe dinámicamente en `C:\Users\Public\Public\keylogs.txt`, validando la funcionalidad de keylogging observada en el análisis estático y en el contenido del fichero.
- Se confirma la creación de capturas de pantalla en formato BMP.

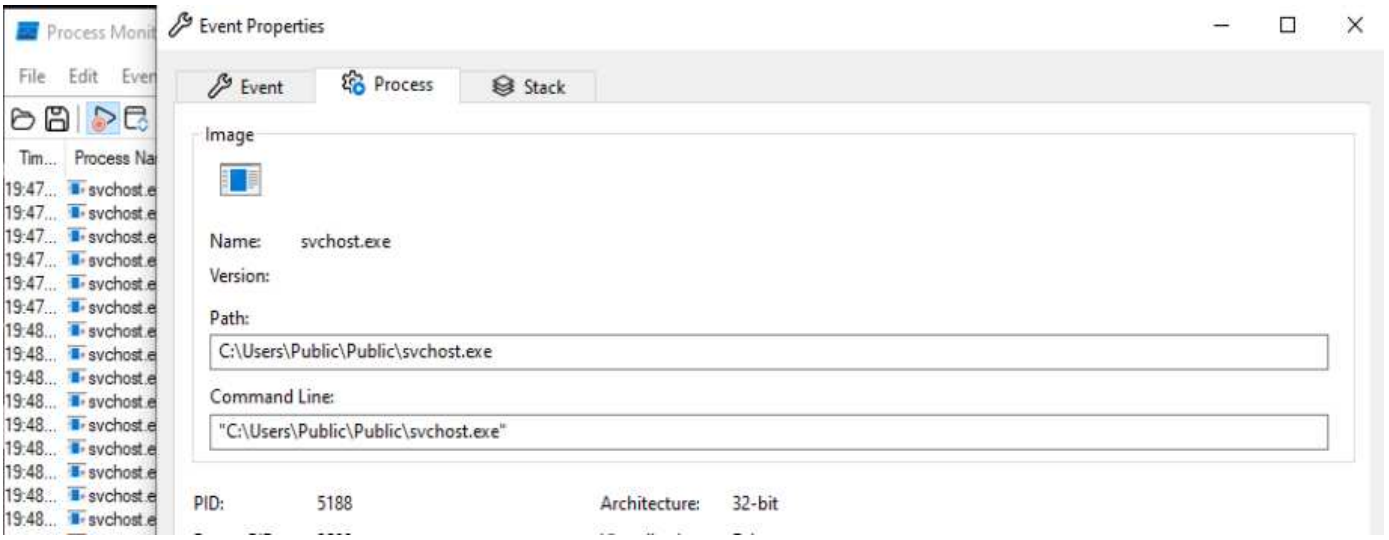
Esta evidencia valida dinámicamente las capacidades de keylogging y captura de pantalla identificadas durante el análisis estático.

16.5 Validamos la persistencia tras reinicio

Reiniciamos la máquina virtual para comprobar si tras reiniciar sigue ejecutándose: `C:\Users\Public\Public\svchost.exe`. Para ello abrimos Procmon y aplicamos los filtros:

Campo	Condición	Valor	Acción
`Process Name`	`is`	`svchost.exe`	`Include`
`Path`	`contains`	`C:\Users\Public\Public`	`Include`







donde:

- Se comprueba que, tras el reinicio, el proceso activo ya no aparece como `MASTER_Malware_Lab1.exe`, sino como: `svchost.exe`.
- Estas evidencias confirman que la persistencia se ejecuta correctamente tras el reinicio y que la copia del malware continúa activa bajo el nombre `svchost.exe`.
- También comprobamos que continúa tomando capturas de pantalla y haciendo keylogging.

---

17. Recopilación de Técnicas MITRE usadas

Técnicas principales:

Prioridad	Técnica	Motivo
1	**T1056.001 - Keylogging**	Funcionalidad principal: captura de teclado y ratón.
2	**T1113 - Screen Capture**	Capturas iniciales y periódicas en BMP.
3	**T1547.001 - Registry Run Keys / Startup Folder**	Persistencia confirmada tras reinicio.
4	**T1036.005 - Masquerading**	Copia como `svchost.exe`.
5	**T1059.003 - Windows Command Shell**	Uso de `REG ADD` mediante shell/`reg.exe`.
6	**T1112 - Modify Registry**	Escritura de la clave Run.

Técnicas auxiliares o de soporte:

Técnica	Comentario
T1564.003 - Hidden Window	Aparece en estático por `FindWindowA` / `ShowWindow`; útil para explicar ocultación de la consola.
T1074.001 - Local Data Staging	Aplica porque guarda localmente `keylogs.txt` y screenshots, aunque no se observó exfiltración.
T1106 - Native API	Es técnicamente correcto por el uso de WinAPI, pero es una técnica muy genérica.

La muestra implementa capacidades de `keylogging`, captura de pantalla, persistencia por clave `Run` y camuflaje mediante copia local como `svchost.exe`. El comportamiento se corresponde principalmente con las técnicas `MITRE ATT&CK T1056.001 - Input Capture: Keylogging`, `T1113 - Screen Capture`, `T1547.001 - Registry Run Keys / Startup Folder`, `T1036.005 - Masquerading`, `T1059.003 - Windows Command Shell` y `T1112 - Modify Registry`. Adicionalmente, se observan comportamientos compatibles con `T1564.003 - Hidden Window`, `T1074.001 - Local Data Staging` y `T1106 - Native API`. No se identificaron indicadores de comunicación `C2` ni exfiltración de red durante el análisis.

---

18. Indicadores de Compromiso

Resumen de los IOCs:

Tipo	Indicador	Descripción
MD5	`07B92F1473B82789F71A58EE8999C631`	Hash de la muestra original y de la copia `svchost.exe`
SHA256	`AD9427B659A7B77E08566627B82C21AF23F35FA3047356113CFEE764A890AD92`	Hash SHA256 de la muestra
Ruta	`C:\Users\Public\Public\svchost.exe`	Copia persistente del malware
Ruta	`C:\Users\Public\Public\keylogs.txt`	Fichero de salida del keylogger
Ruta	`C:\Users\Public\Public\Screens\`	Directorio de capturas de pantalla
Fichero	`screenshot.bmp`, `screenshot000.bmp`, `screenshot001.bmp`	Capturas generadas por la muestra
Registro	`HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Firewall`	Clave de persistencia
Valor Run	`Firewall`	Nombre usado para camuflar la persistencia
PDB	`C:\Users\Admin\Desktop\MALWARE Labs\MASTER_Malware_Lab1\Release\MASTER_Malware_Lab1.pdb`	Ruta de depuración expuesta

19. Análisis dinamico avanzado: x32dbg

Resturamos la máquina virtual para comenzar en un entorno limpio. Ahora vamos hacer un análisis dinámico avanzado con x32dbg para:

- Confirmar el flujo real de ejecución.
- Ver los argumentos exactos de APIs críticas antes de que se ejecuten.
- Comprobar que no hay resolución dinámica, unpacking ni lógica oculta.

19.1 Persistencia:

Ponemos los siguientes breakpoints para comprobar dinámicamente la persistencia:

```
bp kernel32.GetModuleFileNameW
bp kernel32.CopyFileW
bp ucrtbase.system
bp kernelbase.CreateProcessA
bp kernelbase.CreateProcessW
bp 00405580
bp 00405760
```

donde:

Breakpoint	Qué confirma
`kernel32.GetModuleFileNameW`	Obtención de la ruta completa del ejecutable actual. Sirve para confirmar cuál es el fichero origen que posteriormente será copiado.
`kernel32.CopyFileW`	Copia del binario actual a `C:\Users\Public\Public\svchost.exe`. Permite comprobar los argumentos `lpExistingFileName`, `lpNewFileName` y `bFailIfExists`.
`ucrtbase.system`	Ejecución del comando construido para crear persistencia. Puede no resolverse si `ucrtbase.dll` aún no está cargada o si x32dbg no identifica el símbolo.
`kernelbase.CreateProcessA/W`	Creación real de procesos derivados, normalmente `cmd.exe` y posteriormente `reg.exe`, usados para ejecutar el comando `REG ADD`.
`00405580`	Entrada a la función `x_set_run_key_persistence`. Confirma que la muestra entra en la rutina encargada de preparar la persistencia.
`00405760`	Llamada exacta a `system()`. Es el breakpoint más fiable para capturar el comando `REG ADD` justo antes de su ejecución.

bp 00405580 devuelve el error memread: En Ghidra la función aparece como: `0x00405580`, pero esa dirección depende de que el binario esté cargado con base: `0x00400000`. La forma correcta es usar el RVA de la función. Vamos a calcular el RVA: `0x00405580 - 0x00400000 = 0x5580`. Y para la llamada a `system()`: `0x00405760 - 0x00400000 = 0x5760`.

El breakpoint directo en `0x00405580` falla con error `memread` ya que la imagen no está cargada todavía en esa dirección o porque el módulo ha sido reubicado. Para evitarlo, usaremos el RVA de la función (`0x5580`) relativo al módulo cargado en x32dbg, por ejemplo `MASTER_Malware_Lab1.exe+5580` o `svchost.exe+5580`.

Usaremos breakpoints relativos al módulo:

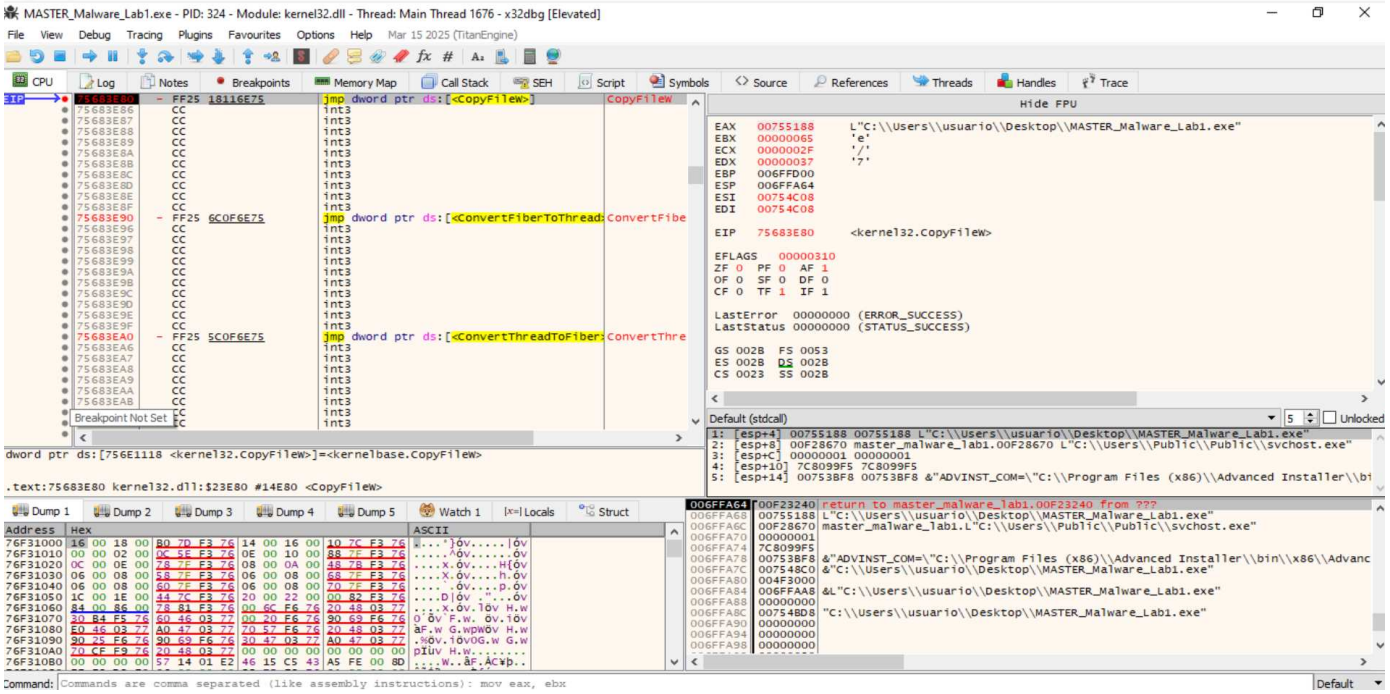
```
bp MASTER_Malware_Lab1.exe+5580
bp MASTER_Malware_Lab1.exe+5760
```

donde:

- `MASTER_Malware_Lab1.exe+5580` = 00F25580.
- `MASTER_Malware_Lab1.exe+5760` = 00F25760.

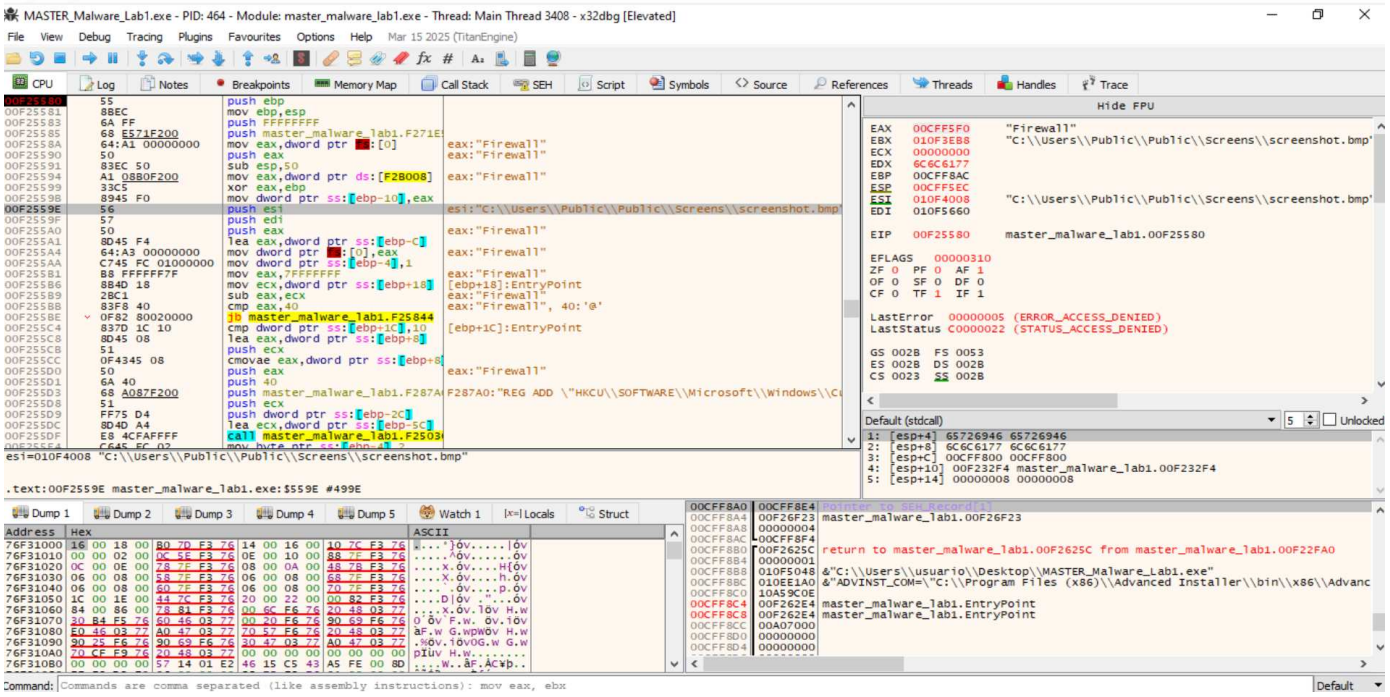
Corregimos la tabla de breakpoints para persistencia:

Breakpoint	Qué confirma
`kernel32.GetModuleFileNameW`	Obtención de la ruta completa del ejecutable actual. Sirve para confirmar el fichero origen que después será copiado.
`kernel32.CopyFileW`	Copia del binario actual a `C:\Users\Public\Public\svchost.exe`. Permite comprobar los argumentos `lpExistingFileName`, `lpNewFileName` y `bFailIfExists`.
`kernelbase.CreateProcessA/W`	Creación real de procesos derivados, normalmente `cmd.exe` y `reg.exe`, usados para ejecutar el comando `REG ADD`.
`MASTER_Malware_Lab1.exe+5580`	Entrada a la función `x_set_run_key_persistence`. Equivale al RVA `0x5580`, que en Ghidra aparece como `0x00405580`.
`MASTER_Malware_Lab1.exe+5760`	Llamada exacta a `system()`. Equivale al RVA `0x5760`, que en Ghidra aparece como `0x00405760`.



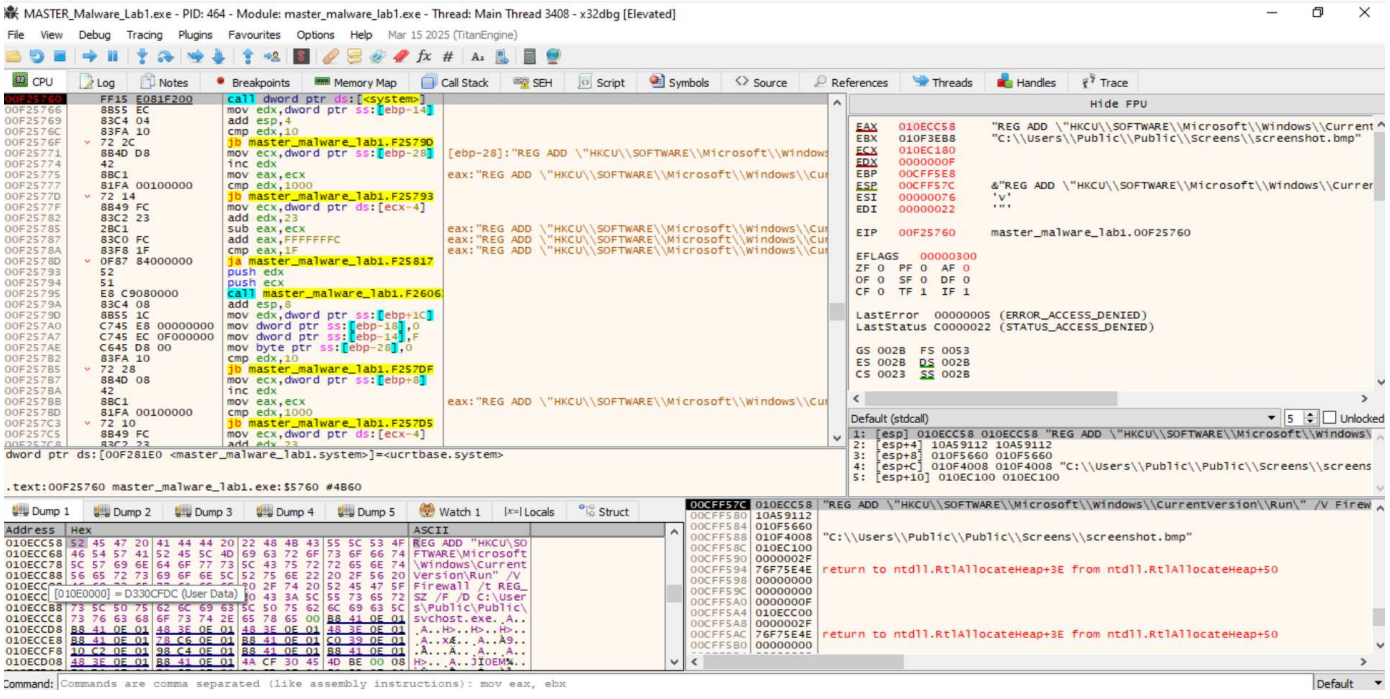
donde:

- Estamos parados en `kernel32.CopyFileW`.
- En x86/stdcall, `CopyFileW` recibe los argumentos en la pila así:
  - `[ESP+4] lpExistingFileName`: Fichero origen: la muestra original: `C:\Users\usuario\Desktop\MASTER_Malware_Lab1.exe`
  - `[ESP+8] lpNewFileName`: Fichero destino: copia persistente: `C:\Users\Public\Public\svchost.exe`.
  - `[ESP+C] bFailIfExists`: `bFailIfExists = TRUE`: `00000001`.
- Esta captura confirma dinámicamente que la muestra intenta copiarse a sí mismo.



donde:

- Estamos parados en `MASTER_Malware_Lab1.exe+5580`, en la función `0x00405580 - x_set_run_key_persistence`.
- En x86/stdcall vemos:
  - `[esp+4] = 65726946`: En little-endian eso corresponde a: `Fire`.
  - `[esp+8] = 6C6C6177`. En little-endian eso corresponde a: `wall`.
- Confirmanmos que la función recibe el nombre del valor `Run`: `Firewall`.

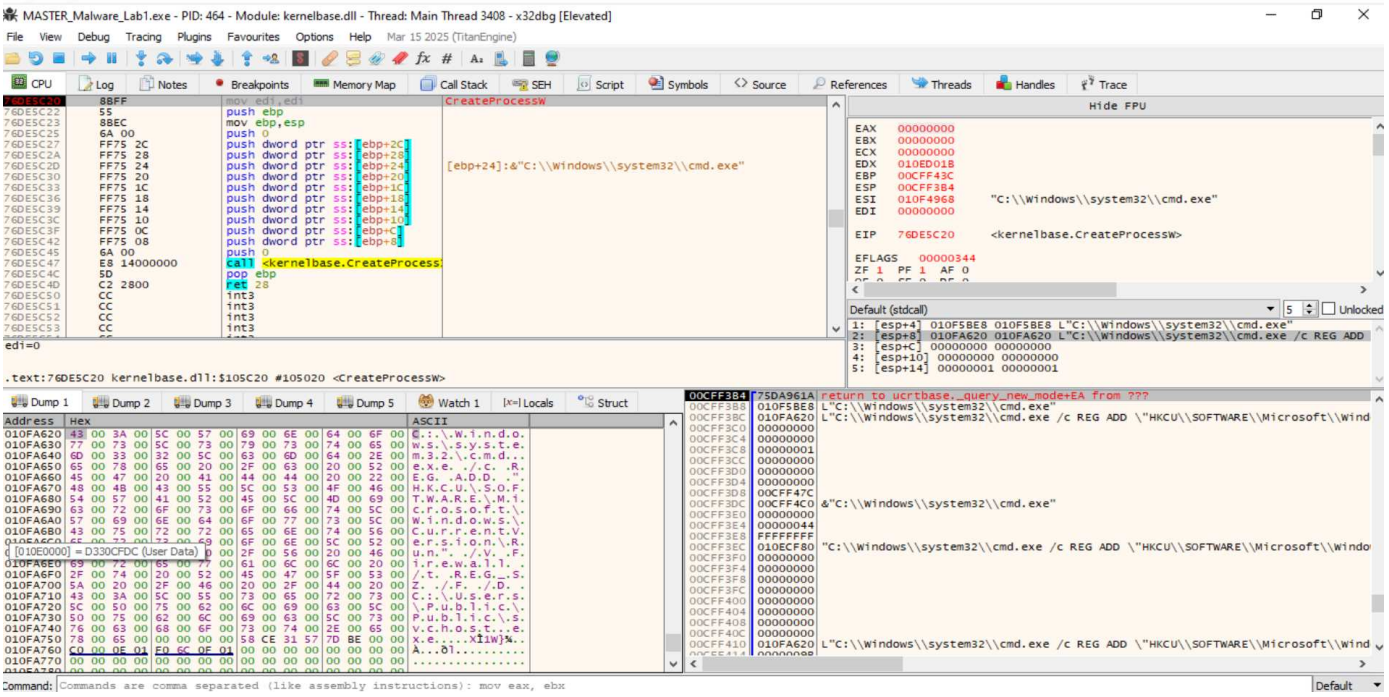


donde:

- Estamos parados en `MASTER_Malware_Lab1.exe+5760`, justo en la llamada a: `system(command);`.
- Vemos en la pila el argumento de `system()`:

```
[ESP] = 010ECC58 -> "REG ADD \"HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\" /V Firew..."
```

- Vemos en la direccion `010ECC58` este comando.
- Esta parada confirma dinámicamente que la función `x_set_run_key_persistence` ha construido el comando de genera la persistencia y que está a punto de ejecutarlo mediante `system()`.



donde:

- Estamos parados en el punto en el que `system()` ha lanzado realmente `cmd.exe` mediante `CreateProcessW`.
- `[ESP+4]`: Aplicación que se lanza: `C:\Windows\system32\cmd.exe`.
- `[ESP+8]`: Línea de comandos que se ejecuta: `C:\Windows\system32\cmd.exe /c REG ADD ...` .

En x32dbg se ha establecido un breakpoint en `kernelbase.CreateProcessW` para comprobar la creación real del proceso derivado de la llamada a `system()`. Al detenerse la ejecución, se observa que `CreateProcessW` recibe como aplicación `C:\Windows\system32\cmd.exe` y como línea de comandos `cmd.exe /c REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D C:\Users\Public\Public\svchost.exe`. Esto confirma dinámicamente que la muestra ejecuta `cmd.exe` para lanzar el comando de persistencia mediante `REG ADD`.

19.2 Keylogger

Vamos a demostrar ahora `keylogger`. Para ello limpiamos lo breakpoints y establecemos bp en:

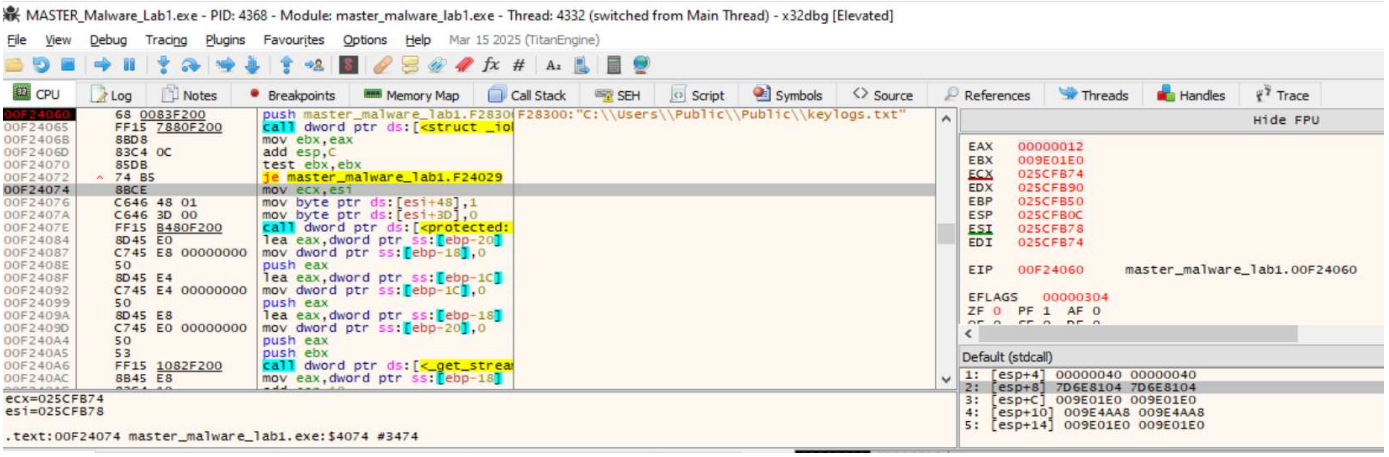
```
bp master_malware_lab1.exe+2A00
bp user32.GetAsyncKeyState
bp master_malware_lab1.exe+1550
bp master_malware_lab1.exe+3FF0
bp kernel32.CreateFileA
bp kernel32.WriteFile
```

Breakpoint	Qué confirma
`master_malware_lab1.exe+2A00`	Entrada al bucle principal del keylogger. Corresponde a `0x402A00 - x_keylogger_loop`.
`user32.GetAsyncKeyState`	Captura del estado de teclas mediante polling. Es la API clave del keylogger.
`master_malware_lab1.exe+1550`	Procesamiento de teclas especiales como `ENTER`, `SPACE`, `BACKSPACE`, `SHIFT`, etc.
`master_malware_lab1.exe+3FF0`	Rutina que abre/prepara `C:\Users\Public\Public\keylogs.txt`.
`kernel32.CreateFileA`	Creación/apertura real del fichero `keylogs.txt`.
`kernel32.WriteFile`	Escritura en disco del keylogger.

Para no extendernos demasiado, vamos a demostrar únicamente: La escritura en disco del keylogger. Usaremos la XREF directa a `keylogs.txt`

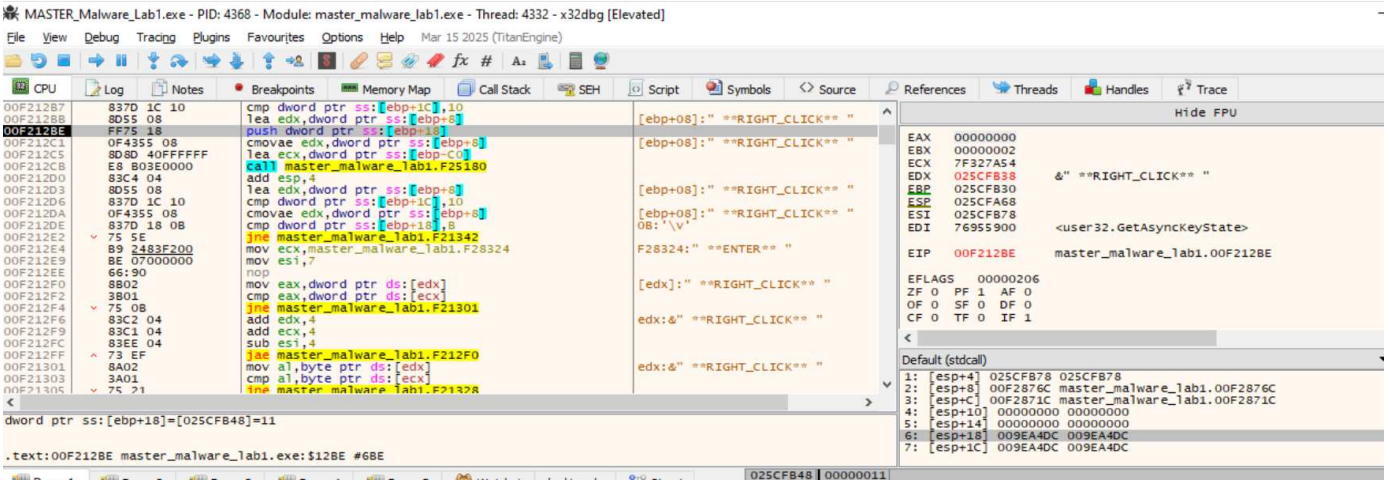
```
bp master_malware_lab1.exe+3FF0
bp master_malware_lab1.exe+4060
```

Breakpoint	Qué confirma
`master_malware_lab1.exe+3FF0`	Entrada a la función que prepara/abre el fichero de keylogging
`master_malware_lab1.exe+4060`	Uso directo de la string `C:\Users\Public\Public\keylogs.txt`



donde:

- Estamos parados en: `master_malware_lab1.exe+4060` que corresponde a la XREF que habíamos identificado en Ghidra: `0x00404060`.
- En este punto la muestra está preparando o abriendo el fichero donde almacenará las pulsaciones capturadas.

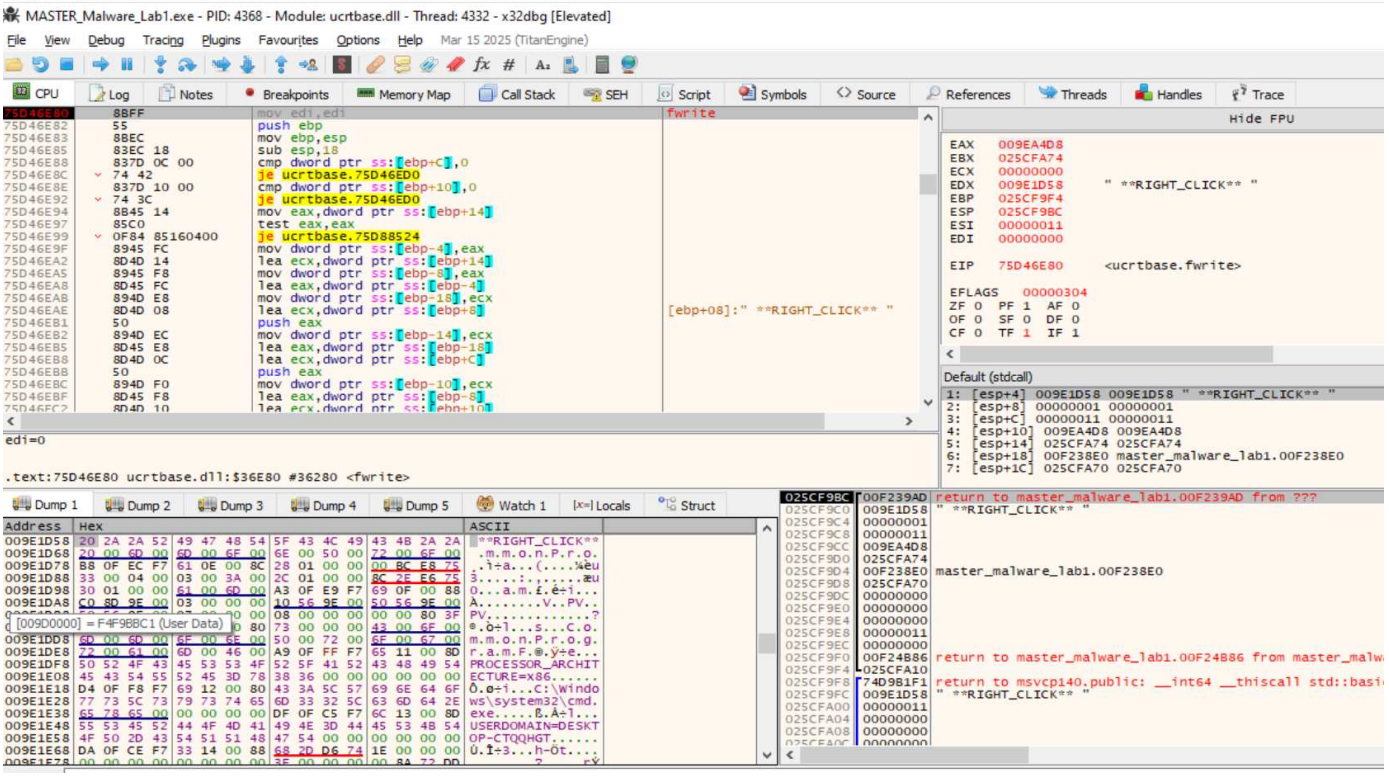


donde:

- Estamos dentro del flujo del keylogger.
- Vemos en `edx`: `&"**RIGHT_CLICK** "`

Ponemos un bp en `ucrtbase.fwrite`:

```
bp ucrtbase.fwrite
```



donde:

- Estamos parados en
- Vemos en la pila: | Argumento | Valor | Interpretación | | ---------- | ---------: | ------------------------------ | | `[ESP+4]` | `009E1D58` | Puntero al buffer | | `[ESP+8]` | `00000001` | Tamaño de cada elemento: 1 byte | | `[ESP+C]` | `00000011` | Número de elementos: 17 bytes | | `[ESP+10]` | `009EA4D8` | Stream/fichero de destino |

Esto demuestra que el keylogger está escribiendo una etiqueta de evento de ratón en su fichero de log.

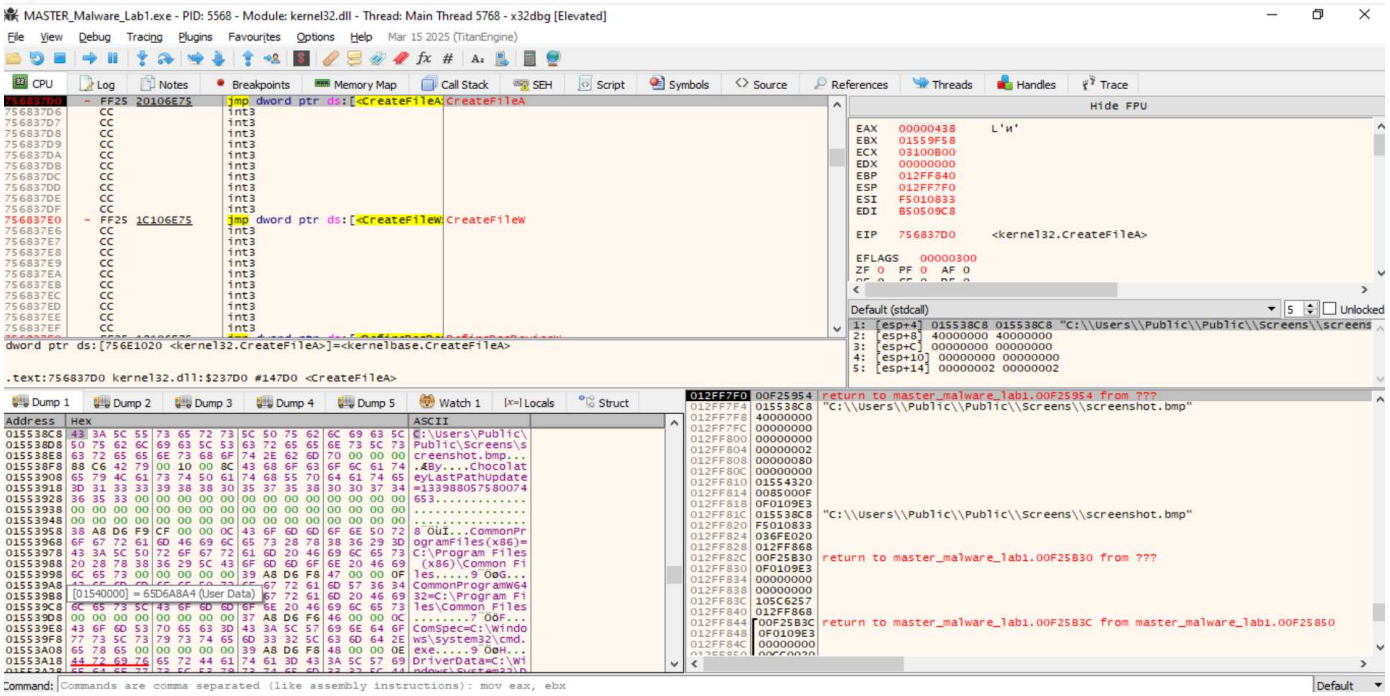
En x32dbg se ha establecido un breakpoint en `ucrtbase.fwrite` para observar la escritura del keylogger. Al detenerse la ejecución, el primer argumento de `fwrite` (`[ESP+4]`) apuntaba a la cadena `**RIGHT_CLICK**`, el segundo argumento indicaba un tamaño de elemento de `1` byte y el tercer argumento indicaba una longitud de `0x11` bytes. Esto confirma que la muestra escribe etiquetas de eventos capturados por el keylogger mediante `fwrite`. Esta evidencia, junto con la apertura previa de `C:\Users\Public\Public\keylogs.txt` mediante `_Fiopen`, confirma dinámicamente que los datos capturados se almacenan en el fichero `keylogs.txt`.

19.3 Captura de pantalla

Vamos a comprobar que crea y escribe un fichero BMP en `C:\Users\Public\Public\Screens\`. Para ello limpiamos lo breakpoints y establecemos bp en:

```
bp kernel32.CreateFileA
bp kernel32.WriteFile
```

Breakpoint	Qué confirma
`kernel32.CreateFileA`	Creación del fichero BMP en disco
`kernel32.WriteFile`	Escritura de la cabecera BMP y de los datos de imagen



donde:

- Estamos parados en `kernel32.CreateFileA`.
- En la pila vemos:

Stack	Valor observado	Significado
`[ESP+4]`	`C:\Users\Public\Public\Screens\screenshot.bmp`	Ruta del fichero que se va a crear
`[ESP+8]`	`0x40000000`	`GENERIC_WRITE`
`[ESP+C]`	`0x00000000`	Sin compartición
`[ESP+10]`	`0x00000000`	Sin atributos de seguridad especiales
`[ESP+14]`	`0x00000002`	`CREATE_ALWAYS`

Esto equivale a:

```
CreateFileA(
    "C:\\Users\\Public\\Public\\Screens\\screenshot.bmp",
    GENERIC_WRITE,
    0,
    NULL,
    CREATE_ALWAYS,
    FILE_ATTRIBUTE_NORMAL,
    NULL
);
```

En x32dbg se establece un breakpoint en `CreateFileA` para validar la creación del fichero de captura. Al detenerse la ejecución, el primer argumento de la llamada apuntaba a `C:\Users\Public\Public\Screens\screenshot.bmp`. Además, los argumentos indicaban apertura con `GENERIC_WRITE` y modo `CREATE_ALWAYS`, lo que confirma que la muestra crea un fichero BMP en disco dentro del directorio `C:\Users\Public\Public\Screens\`.

20. Conclusiones

El análisis estático y dinámico de la muestra `MASTER_Malware_Lab1.exe` confirma que se trata de un **malware de tipo keylogger con capacidad de captura de pantalla y persistencia local**. La muestra está desarrollada como un ejecutable **PE32 de consola para Windows**, sin evidencias claras de empaquetado u ofuscación relevante. Herramientas como Detect It Easy, PEStudio y FLOSS no mostraron indicios fuertes de packing ni cadenas ocultas decodificadas, por lo que gran parte de su funcionalidad pudo identificarse mediante strings, imports, XREFs y decompilación en Ghidra.

Durante el análisis estático se identificaron varias capacidades maliciosas principales. La función `0x402A00` implementa un bucle de keylogging basado en `GetAsyncKeyState`, mientras que la función `0x401550` procesa teclas especiales y las registra mediante etiquetas legibles como `**ENTER**, **SPACE**, **BACKSPACE**, **LEFT_SHIFT** o **LEFT_CONTROL**`. La función `0x401250` actúa como rutina auxiliar de escritura del keylogger, y las XREFs a `C:\Users\Public\Public\keylogs.txt` confirmaron que este fichero es el destino local de las pulsaciones capturadas.

También se confirmó la capacidad de captura de pantalla. Las funciones `0x405AD0` y `0x405850` utilizan APIs gráficas como `GetDC`, `CreateCompatibleDC`, `CreateCompatibleBitmap`, `BitBlt` y `GetDIBits` para obtener la imagen de pantalla y escribirla manualmente en formato BMP. Además, la función `0x402C40` implementa un hilo periódico que genera capturas numeradas como `screenshot000.bmp`, `screenshot001.bmp`, etc., con un intervalo aproximado de 50 segundos.

La persistencia se implementa mediante una clave `Run` del usuario actual. La función `0x405580` construye y ejecuta el comando:

```
REG ADD "HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run" /V Firewall /t REG_SZ /F /D
C:\Users\Public\Public\svchost.exe
```

Esto crea el valor `Firewall` en `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, apuntando a una copia del malware ubicada en `C:\Users\Public\Public\svchost.exe`. El uso de `HKCU` permite mantener persistencia sin requerir privilegios administrativos. El nombre `Firewall` y la copia como `svchost.exe` representan un intento de camuflaje usando nombres asociados a componentes legítimos de Windows.

El análisis dinámico validó las hipótesis estáticas. Se observó la creación de `C:\Users\Public\Public\svchost.exe`, cuyo hash coincidía con el de la muestra original, confirmando que se trata de una copia exacta del malware. También se confirmó la creación de `keylogs.txt`, cuyo contenido registraba texto introducido por el usuario y teclas especiales. La carpeta `C:\Users\Public\Public\Screens` contenía capturas `screenshot.bmp` y capturas numeradas, validando la funcionalidad de screen capture inicial y periódica.

Process Monitor confirmó la ejecución de `reg.exe` para crear la persistencia, con un evento `RegSetValue` exitoso sobre `HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run\Firewall`. RegShot confirmó la misma modificación del registro, y Regedit permitió verificar visualmente que el valor `Firewall` apuntaba a `C:\Users\Public\Public\svchost.exe`. Tras reiniciar la máquina, Procmon mostró que el proceso `svchost.exe` continuaba escribiendo en `keylogs.txt` y generando capturas en `Screens`, confirmando que la persistencia era funcional.

Desde el punto de vista de MITRE ATT&CK, las técnicas principales observadas son:

Técnica	Descripción	Estado
`T1056.001`	Input Capture: Keylogging	Confirmada
`T1113`	Screen Capture	Confirmada
`T1547.001`	Registry Run Keys / Startup Folder	Confirmada
`T1036.005`	Masquerading: Match Legitimate Resource Name or Location	Confirmada
`T1059.003`	Windows Command Shell	Confirmada
`T1112`	Modify Registry	Confirmada
`T1074.001`	Local Data Staging	Confirmada
`T1564.003`	Hidden Window	Evidencia estática, confianza media-alta

No se observaron indicadores claros de comunicación de red, `C2` o exfiltración remota. La muestra parece almacenar la información recolectada localmente en disco, concretamente en `keylogs.txt` y en ficheros BMP dentro de la carpeta Screens. Tampoco se confirmó antidebug propio de la muestra: aunque `IsDebuggerPresent` aparece importado y referenciado, su uso se localizó en rutinas del runtime de Visual C++ como `__scrt_fastfail`, por lo que no se considera una técnica antianálisis implementada directamente por el malware.

En conclusión, `MASTER_Malware_Lab1.exe` es una muestra orientada a la recolección local de información del usuario, principalmente mediante keylogging y capturas de pantalla, con persistencia en el registro y camuflaje básico como `svchost.exe`. El análisis dinámico confirmó las capacidades identificadas durante el análisis estático, por lo que el comportamiento malicioso principal queda validado con evidencias de fichero, registro, Procmon y x32dbg.